



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE GRADO

Quizzie - Plataforma multidispositivo para la realización y gestión dinámica de tests educativos

Realizado por
Francisco Gago Vázquez

Para la obtención del título de
Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por
Jesús Moreno León

En el departamento de
Lenguajes y Sistemas Informáticos

Convocatoria de octubre, curso 2025/26

Aquí la dedicatoria del trabajo

Agradecimientos

Quiero agradecer a X por...

También quiero agradecer a Y por...

Resumen

Incluya aquí un resumen de los aspectos generales de su trabajo, en español.

Palabras clave: Palabra clave 1, palabra clave 2, ..., palabra clave N

Abstract

This section should contain an English version of the Spanish abstract.

Keywords: Keyword 1, keyword 2, ..., keyword N

Índice general

1	Introducción	1
1.1.	Contexto y objeto del proyecto	1
1.2.	Motivación	2
1.3.	Objetivos del proyecto	2
1.3.1.	Objetivos académicos	2
1.3.2.	Objetivos técnicos y profesionales	3
1.3.3.	Objetivos de la plataforma	4
1.4.	Resumen del <i>stack</i> tecnológico	4
1.4.1.	Ecosistema del <i>backend</i>	5
1.4.2.	Ecosistema del <i>frontend</i>	5
1.5.	Estructura de la memoria	6
2	Acta de constitución	8
2.1.	Información del proyecto	8
2.2.	Descripción y propósito	8
2.3.	Requerimientos de alto nivel	9
2.4.	Marco legal y normativo	9
2.5.	Riesgos iniciales de alto nivel	10
2.5.1.	Riesgos de alcance	10
2.5.2.	Riesgos técnicos	11
2.5.3.	Riesgos temporales	11
2.6.	Lista de interesados (<i>Stakeholders</i>)	12
3	Gestión del proyecto	13
3.1.	Metodología del trabajo	13
3.2.	Línea base del alcance	14
3.2.1.	Estructura del Desglose del Trabajo (EDT)	14
3.2.2.	Diccionario de la EDT	15
3.3.	Línea base del cronograma	22
3.3.1.	Fases del proyecto	22
3.3.2.	Lista de actividades	23
3.3.3.	Lista de hitos	24
3.3.4.	Diagrama de Gantt	25
3.4.	Línea base de Costes	25
3.4.1.	Análisis de costes	25
3.4.2.	Costes de infraestructura según el número de usuarios	28
3.4.3.	Coste total de propiedad (TCO)	30
3.5.	Plan de gestión de cambios y riesgos	30
4	Estado del arte y fundamentación	32
4.1.	Análisis del mercado	32
4.2.	Factores diferenciadores	33

4.3.	Fundamentación tecnológica	34
4.3.1.	Alternativas consideradas y justificación de las decisiones . .	34
4.3.2.	Limitaciones tecnológicas	38
4.3.3.	Herramientas de soporte al desarrollo y control de calidad . .	38
4.4.	Conclusiones del estudio previo	40
5	Análisis de requisitos	42
5.1.	Historias de usuario	43
5.2.	Reglas de negocio y restricciones	45
5.2.1.	Reglas de negocio	45
5.2.2.	Restricciones	46
5.3.	Requisitos de información	47
5.4.	Requisitos funcionales	48
5.4.1.	Criterios de aceptación y definición de hecho	51
5.5.	Casos de uso	52
5.5.1.	Especificación detallada de Casos de Uso	53
5.6.	Requisitos no funcionales	59
5.7.	Trazabilidad de requisitos	60
6	Diseño y arquitectura	63
6.1.	Patrones arquitectónicos elegidos	63
6.1.1.	Arquitectura cliente-servidor desacoplada	64
6.1.2.	Arquitectura en capas	64
6.1.3.	Arquitectura basada en eventos	65
6.2.	Arquitectura global del sistema	67
6.3.	Justificación y decisiones de arquitectura	68
6.3.1.	Registros de decisiones de arquitectura (ADR)	68
6.3.2.	Trazabilidad entre RNF y tácticas arquitectónicas	70
6.4.	Estructura modular del código	71
6.4.1.	Organización del backend	72
6.4.2.	Organización del frontend	73
6.5.	Modelo de datos	73
6.5.1.	Módulo de gestión de usuarios	74
6.5.2.	Módulo de contenido	74
6.5.3.	Módulo de sesiones	75
6.6.	Diseño de interfaz de usuario	76
6.6.1.	Principios de diseño y guía de estilo	76
6.6.2.	Flujos funcionales y prototipos	76
6.7.	Seguridad	77
6.7.1.	Autenticación síncrona mediante JWT	77
6.7.2.	Gestión de roles y control de acceso (RBAC)	78
6.7.3.	Tokens de verificación para participantes	78
6.8.	Documentación de la API	79
6.8.1.	Integración con OpenAPI y Swagger	79
6.8.2.	Especificación de rutas fundamentales	80

7	Metodología técnica y de desarrollo	82
7.1.	Ecosistema base de desarrollo local	82
7.2.	Políticas del repositorio y flujo de trabajo	83
7.2.1.	Gestión de tareas mediante plantillas de <i>GitHub Issues</i>	83
7.2.2.	Estrategia de ramas	84
7.2.3.	Verificación local mediante <i>pre-commit</i>	85
7.2.4.	Estandarización de commits	86
7.2.5.	Consolidación y despliegue a producción	87
7.3.	Métricas de calidad y análisis estático de código	88
7.3.1.	Justificación y métricas objetivo (<i>Quality Gate</i>)	88
7.3.2.	Configuración y exclusiones técnicas	89
7.4.	Integración y Despliegue Continuo (CI/CD)	89
7.4.1.	Ciclo de vida y arquitectura	89
7.4.2.	Flujo de Integración Continua (CI)	90
7.4.3.	Análisis de calidad con SonarQube	91
7.4.4.	Estrategia de Despliegue Continuo (CD)	91
7.4.5.	Resumen de la estructura de CI/CD	92
7.5.	Gestión de configuración y entorno	93
7.5.1.	Categorización funcional de los secretos	93
7.5.2.	Entorno de desarrollo local	93
7.5.3.	Distribución de secretos por plataforma	94
8	Implementación y pruebas	95
8.1.	Componentes clave e implementación del sistema	95
8.1.1.	Comunicación en tiempo real mediante WebSockets	95
8.1.2.	Gestión de sala, estado, tiempo y aleatoriedad	98
8.1.3.	Seguridad, contexto y autenticación	102
8.2.	Estrategia de testing	103
8.3.	Casos de prueba y validación	103
8.4.	Pruebas de carga y rendimiento	104
9	Resultados y evaluación	105
9.1.	Grado de cumplimiento de la Línea Base del Alcance	105
9.2.	Tiempo real frente a Línea Base del Cronograma	105
9.3.	Gastos incurridos frente a Línea Base de Costes	105
9.4.	Desviaciones y gestión de cambios realizados	105
9.5.	Resultados de las pruebas y validación	105
9.6.	Retrospectiva técnica	105
10	Conclusiones	106
10.1.	Grado de cumplimiento de los objetivos	106
10.2.	Lecciones aprendidas y valoración personal	106
10.3.	Trabajos futuros y líneas de evolución	106
A	Prototipos de interfaz de usuario	107
B	Manual de usuario	118

C Manual de instalación	119
Bibliografía	120

Índice de figuras

3.1. Estructura de Desglose del Trabajo (EDT).	15
3.2. Diagrama de Gantt inicial del proyecto.	25
3.3. Costes mensuales de infraestructura según escenario de escalado. . .	29
3.4. Evolución de OPEX en función del volumen de usuarios.	29
5.1. Diagrama general de casos de uso del sistema.	52
6.1. Estados de la sala de juego en tiempo real.	65
6.2. Diagrama de secuencia de la transmisión de eventos en una sala. . .	66
6.3. Comparativa entre la arquitectura global y su implementación concreta.	67
6.4. Estructura preliminar de módulos y componentes del proyecto. . . .	72
6.5. Diagrama de clases conceptual y relaciones globales del dominio. . .	73
6.6. Entidades y atributos del módulo de usuarios.	74
6.7. Entidades y atributos del módulo de contenido.	75
6.8. Entidades y atributos del módulo de sesiones.	75
6.9. Flujo de autenticación del profesor mediante JWT.	78
6.10. Diagrama del proceso de verificación del participante.	79
6.11. Flujo de generación de documentación OpenAPI desde los esquemas Pydantic.	80
6.12. Secuencia del intercambio de eventos durante la sesión en tiempo real.	81
7.1. Ciclo de trabajo secuencial implementado en Quizzie.	83
7.2. Diagrama del flujo de ramas en el repositorio.	85
7.3. Procedimiento para integrar código en la rama <i>main</i>	87
7.4. Estructura del <i>pipeline</i> de CI/CD	90
7.5. Resumen detallado de la estructura de CI/CD.	92
A.1. Crear cuenta.	108
A.2. Verificar cuenta.	109
A.3. Iniciar sesión.	109
A.4. Crear cuestionario.	110
A.5. Listado de cuestionarios.	110
A.6. Configuración de la sala.	111
A.7. Sala de espera de la sala recién creada.	111
A.8. Ingresar código de sala.	112
A.9. Ingresar UVUS o nickname.	112
A.10. Aviso para registrar nuevo UVUS o nickname.	113
A.11. Sala de espera del alumno recién ingresado.	113
A.12. Sala de espera.	114
A.13. Cuenta atrás.	114
A.14. Tiempo de lectura de pregunta.	115
A.15. Pregunta en curso y envío de respuesta.	115
A.16. Resultados de la pregunta.	116

A.17.Podio de puntuaciones.	116
A.18.Distribución global de respuestas.	117
A.19.Verificación final de la sesión.	117

Índice de tablas

2.1. Datos de identificación del proyecto.	8
2.2. Tabla de riesgos de alcance.	10
2.3. Tabla de riesgos técnicos.	11
2.4. Tabla de riesgos temporales.	11
2.5. Lista de interesados y roles en el proyecto.	12
3.1. Datos del entregable 1.1.	15
3.2. Actividades del entregable 1.1.	16
3.3. Datos del entregable 1.2.	16
3.4. Actividades del entregable 1.2.	16
3.5. Datos del entregable 1.3.	16
3.6. Actividades del entregable 1.3.	17
3.7. Datos del entregable 2.1.	18
3.8. Actividades del entregable 2.1.	18
3.9. Datos del entregable 2.2.	18
3.10. Actividades del entregable 2.2.	18
3.11. Datos del entregable 3.1.	19
3.12. Actividades del entregable 3.1.	19
3.13. Datos del entregable 3.2.	19
3.14. Actividades del entregable 3.2.	19
3.15. Datos del entregable 3.3.	20
3.16. Actividades del entregable 3.3.	20
3.17. Datos del entregable 4.1.	20
3.18. Actividades del entregable 4.1.	20
3.19. Datos del entregable 4.2.	21
3.20. Actividades del entregable 4.2.	21
3.21. Datos del entregable 4.3.	21
3.22. Actividades del entregable 4.3.	21
3.23. Fases temporales del proyecto.	22
3.24. Actividades organizadas por su fase del cronograma.	23
3.25. Hitos de control del proyecto.	24
3.26. Resumen de Gastos de Capital (CAPEX).	26
3.27. Costes netos laborales agrupados por perfil profesional.	27
3.28. Resumen de Gastos Operativos mensuales (OPEX).	28
3.29. Comparativa de OPEX según escenario de escalado.	28
3.30. TCO proyectado a tres años.	30
4.1. Tabla comparativa del mercado.	33
4.2. Resumen del stack tecnológico de Quizzie.	34
4.3. Alternativas tecnológicas para la base de datos.	35
4.4. Alternativas tecnológicas para el ORM y validación.	35
4.5. Alternativas tecnológicas para el desarrollo del backend.	36
4.6. Alternativas para el lenguaje de programación frontend.	36

4.7. Alternativas tecnológicas para la interfaz de usuario.	37
4.8. Alternativas tecnológicas para el diseño de estilos visuales.	37
4.9. Alternativas para la infraestructura y el despliegue.	38
5.1. Objetivos funcionales de <i>Quizzie</i>	42
5.2. Historias de usuario asociadas al rol de Alumno.	43
5.3. Historias de usuario asociadas al rol de Profesor.	44
5.4. Requisitos de información.	47
5.5. Requisitos funcionales (1/3)	48
5.6. Requisitos funcionales (2/3)	49
5.7. Requisitos funcionales (3/3)	50
5.8. Especificación del caso de uso CU-01.	53
5.9. Especificación del caso de uso CU-02.	54
5.10. Especificación del caso de uso CU-03.	54
5.11. Especificación del caso de uso CU-04.	55
5.12. Especificación del caso de uso CU-05.	55
5.13. Especificación del caso de uso CU-06.	56
5.14. Especificación del caso de uso CU-07.	56
5.15. Especificación del caso de uso CU-08.	57
5.16. Especificación del caso de uso CU-09.	57
5.17. Especificación del caso de uso CU-10.	58
5.18. Especificación del caso de uso CU-11.	58
5.19. Requisitos no funcionales clasificados por dimensión.	59
5.20. Matriz de trazabilidad RI-OBJ.	60
5.21. Matriz de trazabilidad RNF-OBJ.	60
5.22. Matriz de trazabilidad RF-OBJ.	61
5.23. Matriz de trazabilidad unificada del sistema.	62
6.1. Registro de decisiones de arquitectura - ADR-01.	68
6.2. Registro de decisiones de arquitectura - ADR-02.	69
6.3. Registro de decisiones de arquitectura - ADR-03.	69
6.4. Registro de decisiones de arquitectura - ADR-04.	70
6.5. Matriz de trazabilidad entre RNF y tácticas arquitectónicas.	71
7.1. Clasificación de ramas, nomenclatura y rama de origen.	84
7.2. Tipos de <i>commits</i> según Conventional Commits.	86
7.3. Métricas objetivo y umbrales del <i>Quality Gate</i> en SonarCloud.	88
7.4. Variables de entorno y secretos por plataforma.	94

Índice de extractos de código

8.1. Gestor de conexiones en el <i>backend</i>	96
8.2. Conexión y enrutamiento de eventos WebSocket en React	97
8.3. Cálculo determinista del tiempo restante en el servidor	98
8.4. Gestión de desconexión imprevista del profesor	99
8.5. Algoritmo de barajado determinista anti-copia	99
8.6. Avance de pregunta y paso a siguiente fase	100
8.7. Validación presencial del participante mediante token QR	101
8.8. Gestión del contexto de la sala	102
8.9. Inyección de dependencia para la validación de tokens JWT	103

1. Introducción

A lo largo de mi trayectoria en la carrera, he asistido a multitud de clases teóricas y prácticas, y si algo he aprendido de ellas es que mantener un dinamismo activo en el aula marca la diferencia en el proceso de aprendizaje. Esta experiencia como estudiante, sumada al reto de diseñar un producto tecnológico completo y autónomo, me motivó a enfocar mi Trabajo de Fin de Grado en el desarrollo de *Quizzie*, una plataforma orientada a transformar las dinámicas de evaluación en el entorno educativo.

El objetivo de este capítulo inicial es asentar las bases del trabajo y ofrecer una hoja de ruta clara. Para ello, en las siguientes secciones se detalla el contexto y objeto del proyecto, las motivaciones personales y profesionales que impulsaron su creación, así como los objetivos académicos y técnicos que se pretenden alcanzar.

1.1. Contexto y objeto del proyecto

A raíz de la creciente digitalización en la enseñanza y la necesidad de mantener el interés de un alumnado, surge la necesidad de buscar soluciones que adapten la forma de evaluar en las aulas a la realidad de los estudiantes. En la primera reunión con el tutor de este proyecto, planteamos el reto de diseñar una herramienta que facilitara el trabajo de los profesores y que, al mismo tiempo, resultara entretenida y motivadora para los alumnos a la hora de comprobar lo aprendido mediante cuestionarios en clase.

La integración de este tipo de cuestionarios rápidos ofrece grandes ventajas en el día a día del aula. Para el alumno, rompe la monotonía de las clases teóricas convencionales introduciendo **aprendizaje dinámico**, elevando la participación y proporcionando *feedback* instantáneo sobre su nivel de comprensión. Por otro lado, para el profesor funciona como una herramienta de **seguimiento en tiempo real**: le permite medir de manera automatizada si el grupo está asimilando los conceptos explicados y, en caso de detectar fallos comunes, aclarar las dudas en ese mismo momento.

Sin embargo, el uso de las plataformas actuales en entornos educativos reales destapa un **problema de control crítico**. Al basarse generalmente en la difusión de un enlace o un código de acceso, es muy sencillo que los estudiantes compartan estas credenciales por grupos de mensajería. Esto provoca que alumnos que no han asistido a clase, o que se encuentran en su casa, puedan responder al cuestionario a distancia. Esta falta de **verificación de la presencia física** falsea las estadísticas de rendimiento que recibe el docente e impide usar estos test como herramientas de **evaluación formal**.

Como respuesta a este escenario, el objeto de este proyecto es el desarrollo de *Quizzie*. La aplicación está diseñada para mantener la sencillez y el dinamismo que

se busca en los cuestionarios en vivo, pero introduciendo una **capa de verificación** para los alumnos. A través de este mecanismo de control, *Quizzie* asegura que la participación se acote estrictamente a los alumnos presentes en el aula, evitando el **fraude a distancia** y consolidándose como un instrumento de evaluación válido y seguro.

1.2. Motivación

La elección de este proyecto nace, en gran medida, de mi propia experiencia como estudiante a lo largo de mi formación. He vivido en primera persona cómo la introducción de **cuestionarios interactivos** en mitad de una clase es capaz de transformar por completo el clima del aula, despertando el interés de los alumnos y fijando los conceptos de una manera mucho más eficaz que los métodos tradicionales. No obstante, también he sido testigo de las limitaciones de las herramientas actuales: el uso de pseudónimos informales, la posibilidad de participar de forma remota sin acudir a clase o la frustración del docente al no poder **validar la autenticidad** de los resultados. Ver cómo una dinámica con tanto potencial perdía rigor por falta de control o por limitaciones de las plataformas utilizadas fue el detonante que me impulsó a buscar esta solución.

El desarrollo de *Quizzie* representa la oportunidad perfecta para demostrar mi capacidad de afrontar un **proyecto de software completo** guiado por las metodologías y principios clásicos de la ingeniería del *software* descritos por autores de referencia como Pressman [1] y Sommerville [2]. Abordar este **ciclo de vida de extremo a extremo** no solo me permite consolidar mis conocimientos técnicos, sino también validar mi criterio al tomar decisiones justificadas ante problemas, convirtiendo este trabajo en el puente definitivo hacia mi madurez como ingeniero de *software*.

1.3. Objetivos del proyecto

Para guiar el desarrollo del proyecto y asegurar el cumplimiento de las expectativas tanto formativas como técnicas, he definido una serie de metas divididas en dos vertientes: académica y técnico-profesional.

1.3.1. Objetivos académicos

Metas orientadas a consolidar la formación recibida durante los estudios de grado:

- **Integrar y aplicar** de forma conjunta los conocimientos teóricos y prácticos adquiridos en las distintas asignaturas de la carrera, demostrando su aplicación real mediante la resolución de los problemas descritos en el

análisis de requisitos y su verificación final mediante un plan de pruebas sistemático con un índice de cobertura de código global superior al 80 %.

- **Adoptar metodologías de trabajo ágiles** en la gestión y planificación del proyecto, estableciendo un calendario estricto de iteraciones de desarrollo y midiendo las desviaciones temporales reales frente a la línea base del cronograma para mantener la desviación por debajo del 15 % mediante hitos de control definidos.
- **Fomentar la aplicación rigurosa de buenas prácticas** de ingeniería del *software*, estructurando el sistema según los principios de diseño y legibilidad del paradigma *Clean Code* [3] y articulando la solución bajo patrones de *Clean Architecture* [4], garantizando un historial de control de versiones libre de confirmaciones informales (100 % de cumplimiento de una política de commits semánticos en la rama principal) y asegurando que cada incremento de código supere automáticamente las directrices de formateo y estilo antes de su integración.

1.3.2. Objetivos técnicos y profesionales

Retos tecnológicos asumidos para el producto final y el desarrollo de competencias laborales:

- **Dominar nuevas tecnologías** y herramientas del ecosistema de desarrollo actual, diseñando e implementando una solución cliente-servidor desacoplada que separe con éxito la lógica de negocio del *backend* asíncrono de la representación en la interfaz reactiva del cliente.
- **Gestionar de manera autónoma el ciclo de vida completo** de un producto de *software* real, completando secuencialmente cada una de las fases temporales del cronograma y justificando documentalmente las decisiones de diseño mediante registros formales de decisiones arquitectónicas.
- **Diseñar e implementar un sistema de verificación** presencial robusto y eficiente, garantizando una tasa de fraude por suplantación digital remota de 0 % en las salas de cuestionarios cerradas gracias a la validación de tokens criptográficos de un solo uso.
- **Implementar una estrategia de pruebas exhaustiva** que asegure la calidad y fiabilidad del código, fijando un umbral objetivo de cobertura de código mínimo del 80 % y garantizando la ausencia de duplicidades, vulnerabilidades de seguridad y deuda técnica crítica mediante un flujo continuo de control de calidad.
- **Diseñar y configurar un flujo de trabajo automatizado** para la integración y despliegue continuos (CI/CD), fundamentado en los principios de entrega continua expuestos por Farley [5], asegurando que el 100 % de las modificaciones de código en el repositorio se verifiquen, compilen y desplieguen automáticamente en el entorno de producción únicamente si

superan satisfactoriamente los test de regresión y las metas de calidad preestablecidas.

1.3.3. Objetivos de la plataforma

Aspiraciones funcionales y operativas de *Quizzie*:

- **Evaluación docente simplificada:** Proveer al profesorado de un sistema eficiente para gestionar cuestionarios, reduciendo el tiempo necesario para configurar, lanzar y evaluar una sesión interactiva a menos de 5 minutos, y permitiendo la recogida automatizada de respuestas.
- **Dinamismo e interactividad en el aula:** Priorizar la fluidez de las sesiones en tiempo real en escenarios de alta demanda, asegurando una tasa de error de red inferior al 1 % y una transmisión bidireccional inmediata en el aula bajo simulaciones de carga de al menos 100 usuarios activos concurrentes.
- **Seguimiento continuo del alumnado:** Ofrecer herramientas analíticas para monitorizar el rendimiento individual y colectivo, generando estadísticas en tiempo real y permitiendo el volcado automático del histórico de calificaciones por grupos académicos estables.
- **Validación y presencialidad física:** Asegurar la legitimidad de las pruebas mediante mecanismos de verificación digital *in situ*, optimizando el proceso de captura y decodificación para que el tiempo de validación y firma del comprobante por cámara sea inferior a 2 segundos por participante.
- **Accesibilidad y agilidad de incorporación:** Optimizar la experiencia de usuario con flujos de acceso inmediato y sin necesidad de registro previo para los estudiantes, facilitando la entrada a las salas en menos de 5 segundos.
- **Compatibilidad y despliegue multidispositivo:** Garantizar que la plataforma sea completamente responsiva y accesible desde cualquier tipo de dispositivo (móviles, *tablets* u ordenadores), sin deformaciones ni pérdida de control sobre las acciones.
- **Garantía de privacidad y cumplimiento:** Blindar la privacidad de los datos académicos y el cumplimiento normativo (RGPD) [6], aplicando el principio de minimización de datos al prescindir de registros tradicionales para estudiantes y protegiendo el almacenamiento de credenciales docentes mediante algoritmos de hashing seguro.

1.4. Resumen del *stack* tecnológico

Para garantizar el cumplimiento de los objetivos técnicos del proyecto y asegurar un desarrollo robusto, eficiente y escalable, se ha seleccionado un conjunto de herramientas alineadas con los estándares de la industria actual. Esta sección ofrece una visión inicial del *stack* tecnológico empleado para la

construcción de *Quizzie*. Las ventajas técnicas de cada opción frente a otras alternativas del mercado se argumentarán en profundidad en el Capítulo 4, [Estado del arte y fundamentación](#).

El núcleo del sistema se divide claramente bajo una arquitectura cliente-servidor, donde el *backend* gestiona la lógica de negocio, la seguridad y la comunicación en tiempo real, mientras que el *frontend* ofrece una experiencia interactiva y fluida para profesores y alumnos.

1.4.1. Ecosistema del *backend*

El entorno se ha diseñado priorizando el alto rendimiento, la seguridad y la agilidad en el desarrollo. Las tecnologías y herramientas principales que componen esta capa son:

- **Lenguaje de programación:** Python 3.12 [7], aprovechando las últimas mejoras de rendimiento y tipado estático del lenguaje.
- **Framework de desarrollo:** FastAPI [8], seleccionado por su alta velocidad de ejecución, soporte nativo de operaciones asíncronas y validación automática de datos.
- **Comunicación en tiempo real:** Protocolo WebSocket [9], implementado para gestionar de manera bidireccional y eficiente el flujo de eventos y la sincronización de las salas en vivo.
- **Persistencia y ORM:** SQLAlchemy [10] en combinación con SQLite [11]. Al no requerir un gestor de base de datos externo complejo en esta fase, esta solución permite interactuar de forma nativa con la base de datos utilizando objetos de Python de manera eficiente.
- **Gestión de dependencias:** uv [12], utilizado como gestor de paquetes ultrarrápido para optimizar los tiempos de instalación y asegurar la reproducibilidad del entorno.

1.4.2. Ecosistema del *frontend*

La interfaz de usuario se ha planteado como una *Single Page Application* (SPA) responsiva para ofrecer la fluidez que requiere una aplicación con dinámicas lúdicas y en vivo:

- **Framework principal:** React [13], para la construcción de una interfaz basada en componentes reutilizables y reactivos.
- **Herramienta de construcción (Build Tool):** Vite [14], que proporciona un entorno de desarrollo ágil gracias a su velocidad de compilación y recarga en caliente.
- **Lenguajes:** TypeScript [15] como eje principal para dotar al código de tipado fuerte y robustez, apoyado puntualmente en JavaScript.

- **Enrutado y consumo de API:** React Router Dom [16] para la navegación interna de la aplicación y Axios [17] para realizar peticiones asíncronas y comunicarse con el *backend*.

1.5. Estructura de la memoria

Para facilitar la lectura y comprensión de este documento, la memoria se ha estructurado en diez capítulos principales y dos manuales adicionales organizados de la siguiente manera:

- **Capítulo 1: Introducción.** Define el punto de partida del proyecto, contextualizando el problema de la evaluación presencial frente al uso de herramientas digitales. Se exponen la motivación, los objetivos del proyecto y una visión general del *stack* tecnológico seleccionado.
- **Capítulo 2: Acta de constitución.** Formaliza el inicio del proyecto, detallando la descripción general del producto, los requerimientos de alto nivel, el marco legal y normativo (incluyendo cumplimiento de RGPD y licencias), los riesgos preliminares y los interesados o *stakeholders*.
- **Capítulo 3: Gestión del proyecto.** Describe la planificación organizativa inicial, incluyendo la metodología de trabajo, la línea base del alcance (a través de la EDT y su diccionario), la línea base del cronograma (con el diagrama de Gantt e hitos), la línea base de costes (estimación de presupuestos, Capex, Opex y TCO), así como los mecanismos para la gestión de riesgos y solicitudes de cambios.
- **Capítulo 4: Estado del arte y fundamentación.** Presenta un análisis comparativo de las soluciones y competidores existentes en el mercado de la gamificación educativa, identificando los factores diferenciales de *Quizzie*. Además, justifica detalladamente las decisiones tecnológicas del *stack* y las alternativas consideradas.
- **Capítulo 5: Análisis de requisitos.** Modela los requisitos del sistema empleando casos de uso, historias de usuario y reglas de negocio. Especifica tanto los requisitos funcionales como los no funcionales, y establece la matriz de trazabilidad que vincula los objetivos iniciales con las pruebas del sistema.
- **Capítulo 6: Diseño y arquitectura.** Detalla la arquitectura de *software* seleccionada (cliente-servidor estructurada en capas y basada en eventos), la modularización del código en el *frontend* y el *backend*, el modelo de datos relacional mediante diagramas de dominio, el diseño de la interfaz de usuario (*mockups* iniciales frente al acabado final), los mecanismos de seguridad (JWT y *tokens* de acceso) y la documentación de la API con OpenAPI/Swagger.
- **Capítulo 7: Metodología técnica (CI/CD).** Detalla los aspectos de ingeniería y DevOps integrados en el repositorio, abarcando las políticas de ramas y *commits* (Commitizen, *hooks* de *pre-commit*), el *pipeline* de integración y

despliegue continuo (GitHub Actions), la gestión de perfiles de configuración y secretos, y el control de calidad estática mediante SonarQube.

- **Capítulo 8: Implementación y pruebas.** Recoge las decisiones del desarrollo real mostrando fragmentos de código significativos. Describe la estrategia de *testing* (pruebas unitarias, de integración y extremo a extremo), los umbrales de cobertura y el diseño de las pruebas de carga y rendimiento de la plataforma.
- **Capítulo 9: Resultados y evaluación.** Expone el análisis tras la finalización del desarrollo, comparando los datos de ejecución reales con la planificación inicial (desviaciones en alcance, tiempos y costes). Asimismo, presenta los resultados de los casos de prueba ejecutados y una retrospectiva técnica enfocada en la **deuda técnica acumulada**.
- **Capítulo 10: Conclusiones.** Evalúa el **grado de consecución de los objetivos** generales e individuales planteados. Recoge las lecciones aprendidas durante el ciclo de vida del proyecto, una valoración personal sobre el proceso y las futuras líneas de evolución de la plataforma.

Adicionalmente, se incluyen dos manuales prácticos al final del documento como anexos independientes:

- **Prototipos de interfaz de usuario:** Conjunto de *mockups* y diagramas de flujo que ilustran la experiencia de usuario prevista, mostrando la navegación y las interacciones clave dentro de la plataforma.
- **Manual de instalación:** Guía técnica detallada para el despliegue del sistema tanto en un entorno local de desarrollo como en entornos de producción en la nube.
- **Manual de usuario:** Guía visual e instructiva orientada a profesores y alumnos para el correcto uso y manejo de la plataforma.

2. Acta de constitución

El desarrollo de *Quizzie* requiere fijar un marco formal que defina sus reglas y compromisos iniciales antes de comenzar cualquier fase de diseño o desarrollo técnico. El propósito de este capítulo es establecer la línea base del proyecto, delimitando su alcance inicial, sus interesados y las restricciones bajo las que se ejecutará. Este documento define el propósito estratégico de la aplicación, garantizando desde el primer momento la viabilidad técnica, la gestión de riesgos y el estricto cumplimiento del marco legal establecido.

2.1. Información del proyecto

La formalización de *Quizzie* como Trabajo de Fin de Grado requiere establecer, en primer lugar, los datos de identificación administrativa y académica que muestran su autoría y tutorización.

Nombre del proyecto	Quizzie
Descripción	Plataforma multidispositivo para la realización y gestión dinámica de tests educativos
Tipo de proyecto	Trabajo de Fin de Grado (TFG)
Titulación	Grado en Ingeniería Informática - Ingeniería del Software
Autor	Francisco Gago Vázquez
Tutor	Jesús Moreno León
Departamento	Lenguajes y Sistemas Informáticos (LSI)

Tabla 2.1: Datos de identificación del proyecto.

2.2. Descripción y propósito

Quizzie nace como una plataforma multidispositivo orientada a transformar la dinámica de evaluación en el entorno educativo. El propósito fundamental de su desarrollo es ofrecer una herramienta interactiva que agilice la creación, gestión y ejecución de tests en tiempo real. Frente a soluciones comerciales existentes, la aplicación pone el foco en la inmediatez, la robustez técnica y el control del flujo de evaluación por parte del profesor gracias a la verificación de la presencialidad de

los alumnos. El desglose de estas capacidades se materializará detalladamente en el capítulo de **análisis de requisitos**.

2.3. Requerimientos de alto nivel

El éxito de la plataforma depende del cumplimiento de una serie de condiciones críticas, atributos de calidad y restricciones de diseño iniciales. Estas directrices sientan las bases del desarrollo y actúan como los criterios de aceptación esenciales para considerar el producto como terminado y listo para su lanzamiento.

Para validar el correcto estado del sistema, este debe cumplir estrictamente con la siguiente lista de verificación:

- **Disponibilidad y despliegue en la nube:** La plataforma completa debe estar desplegada en un entorno de producción real y accesible a través de internet para sus usuarios.
- **Compatibilidad multiplataforma:** La interfaz debe ser completamente *responsive*, garantizando que los alumnos interactúen de forma fluida desde dispositivos móviles y el profesor desde equipos de escritorio.
- **Simplicidad de uso:** El diseño de las pantallas debe priorizar la usabilidad, permitiendo el acceso a las salas de cuestionarios y la creación de test con el menor número de interacciones posible.
- **Seguridad en las comunicaciones:** El intercambio de datos en tiempo real debe realizarse bajo protocolos seguros (HTTPS y WSS), asegurando el cifrado de la información en tránsito.
- **Privacidad de los datos:** El almacenamiento del histórico de calificaciones y datos académicos debe cumplir estrictamente con el marco legal de protección de datos vigente.

2.4. Marco legal y normativo

El desarrollo y uso de *Quizzie* se rige por la normativa vigente en materia de protección de datos, seguridad de la información y propiedad intelectual. Al tratarse de una herramienta orientada al entorno educativo, el diseño del sistema ha priorizado el principio de minimización de datos para mitigar los riesgos asociados al tratamiento de información sensible.

A continuación, se detallan los pilares normativos y legales de la aplicación:

- **Reglamento General de Protección de Datos (RGPD):** La plataforma aplica el principio de privacidad desde el diseño conforme al Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo [6]. Para los estudiantes, el acceso a las salas se realiza mediante un código PIN y su *uvus* (*Usuario Virtual de la Universidad de Sevilla*), prescindiendo de un registro tradicional y

limitando el almacenamiento de datos personales a este identificador institucional único. En el caso de los docentes, el registro obligatorio se limita a un correo electrónico y un alias, omitiendo identificadores directos como nombres, apellidos o DNI.

- **Seguridad y persistencia de calificaciones:** El histórico de resultados almacenado en el sistema solo vincula las calificaciones al *uvus* del alumno y cuenta con la previa verificación física del profesor. Esto garantiza la integridad del proceso de evaluación sin exponer la identidad real del estudiantado en la base de datos.
- **Propiedad intelectual y licenciamiento:** El código fuente y los entregables de este Trabajo de Fin de Grado se distribuyen bajo una licencia de código abierto MIT [18]. Esta elección garantiza la total transparencia del desarrollo, permite la auditoría pública del sistema y fomenta su reutilización o extensión por parte de la comunidad académica, eximiendo al autor de cualquier responsabilidad derivada de su uso.

2.5. Riesgos iniciales de alto nivel

El desarrollo de una plataforma orientada a la evaluación en tiempo real conlleva una serie de riesgos de alcance, técnicos y temporales. Su identificación temprana y categorización mediante matrices de probabilidad e impacto sigue las directrices del cuerpo de conocimiento del PMI [19], lo que permite evaluar la gravedad de cada amenaza y definir las estrategias de mitigación asociadas:

2.5.1. Riesgos de alcance

Desviaciones relacionadas con el volumen de funcionalidades implementadas y la correcta definición de los requisitos del sistema:

ID	Riesgo identificado	Estrategia de mitigación	Prob.	Impacto
R-01	Corrupción del alcance (<i>Scope Creep</i>) al querer añadir más modos de juego o analíticas complejas.	Definición estricta del Producto Mínimo Viable (MVP). Las mejoras secundarias se derivan a "Trabajos Futuros".	Medio	Alto
R-02	Ambigüedad en los requisitos iniciales que obligue a rehacer lógica de negocio ya programada.	Validación previa del modelo de datos e historias de usuario mediante diagramas de flujo antes de codificar.	Bajo	Medio

Tabla 2.2: Tabla de riesgos de alcance.

2.5.2. Riesgos técnicos

Incidencias tecnológicas ligadas a la arquitectura del sistema, conectividad de red e infraestructura empleada:

ID	Riesgo identificado	Estrategia de mitigación	Prob.	Impacto
R-03	Saturación del servidor o retardos (<i>lag</i>) ante múltiples alumnos conectados a la vez.	Diseñar un modelo de datos ligero para las tramas de WebSockets y realizar pruebas de carga tempranas.	Medio	Alto
R-04	Indisponibilidad, caídas o latencia elevada en los servicios en la nube de terceros (<i>Gemini</i> o <i>Render</i>).	Implementación de zonas de control de errores (<i>error boundaries</i>) y reintentos exponenciales en el cliente.	Bajo	Alto
R-05	Despliegue y red: fallos de conectividad o puertos bloqueados en la red wifi que impidan la comunicación.	Configurar el despliegue bajo protocolos seguros de producción y prever mecanismos de reconexión rápida.	Bajo	Alto
R-06	Incompatibilidades visuales menores o descuadres de la interfaz en ciertos navegadores o pantallas antiguas.	Uso de CSS responsivo estándar mediante <i>Flexbox/Grid</i> y testeo rápido en Google Chrome y Firefox.	Alto	Bajo

Tabla 2.3: Tabla de riesgos técnicos.

2.5.3. Riesgos temporales

Amenazas asociadas directa o indirectamente con el cumplimiento de los plazos y el calendario de entregas de la memoria y el software:

ID	Riesgo identificado	Estrategia de mitigación	Prob.	Impacto
R-07	Retraso en el calendario por la curva de aprendizaje en WebSockets o la API de <i>Gemini Pro</i> .	Reservar las primeras semanas para crear pruebas de concepto aisladas antes de programar el núcleo.	Medio	Alto
R-08	Falta de disponibilidad por la carga de trabajo de otras asignaturas simultáneas del curso.	Planificación mediante <i>sprints</i> semanales rígidos con objetivos mínimos viables para mantener el ritmo.	Alto	Medio
R-09	Retrasos menores en la redacción de la memoria final debido al foco exclusivo en el desarrollo de código.	Establecer bloques de tiempo semanales dedicados a documentar los módulos que se acaban de programar.	Medio	Bajo

Tabla 2.4: Tabla de riesgos temporales.

2.6. Lista de interesados (*Stakeholders*)

En este apartado se identifica a los actores, grupos e instituciones que interactúan con la plataforma o se ven afectados por su desarrollo, aplicando la metodología de análisis de interesados descrita por Sommerville [2] para definir sus roles, necesidades y expectativas principales.

Interesado	Rol en el proyecto	Intereses / expectativas
Autor	Desarrollar e implementar el sistema completo.	Diseñar una arquitectura robusta, cumplir los plazos de entrega y superar con éxito la defensa del TFG.
Tutor	Supervisar, guiar y evaluar el desarrollo del proyecto.	Garantizar el rigor metodológico, la calidad técnica del software y validar el cumplimiento de los objetivos.
Departamento LSI	Entidad académica de la Universidad responsable de la calificación del TFG.	Asegurar el cumplimiento de la normativa de los Trabajos de Fin de Grado y el correcto proceso de evaluación.
Cuerpo docente	Futuros usuarios (Profesores).	Disponer de una herramienta intuitiva que facilite la evaluación de los alumnos y asegure su presencialidad.
Alumnado	Futuros usuarios (Estudiantes).	Acceder de forma ágil y multidispositivo sin registros complejos para participar en las evaluaciones dinámicas.

Tabla 2.5: Lista de interesados y roles en el proyecto.

3. Gestión del proyecto

En la ingeniería de software, el éxito de un proyecto no depende únicamente de la calidad de su código, sino de una rigurosa gobernanza que garantice la viabilidad técnica, temporal y económica del proyecto. Para ello, se adopta un enfoque predictivo en la definición de las líneas base fundamentado en el estándar de gestión del PMBOK [19], el cual se complementa con las prácticas de desarrollo iterativo e incremental del Manifiesto Ágil [20] para la gestión del cambio.

3.1. Metodología del trabajo

El desarrollo de *Quizzie* se organiza mediante un flujo de trabajo ágil apoyado en el modelo de ramificación y gestión de versiones *GitHub Flow* [21], el cual centraliza el control de tareas en el repositorio para coordinar las integraciones y el despliegue de las *releases*.

- **GitHub:** Centraliza el almacenamiento del código fuente y la gestión de las tareas mediante las siguientes herramientas:
 - **Issues y Milestones:** Cada funcionalidad o corrección se registra como una *Issue* (tarea). Estas se agrupan en *Milestones* (hitos temporales) para asociar el avance del código con los plazos asignados en el cronograma.
 - **Ramas permanentes:** Se mantiene una separación estricta entre la rama *main* (exclusiva para versiones definitivas y estables) y la rama *develop* (entorno de integración donde se vuelcan las novedades).
 - **Ramas de tareas:** Para cada *issue* se abre una rama específica desde *develop* (ej. *feature/websocket-timer*). Una vez completada y probada la tarea de forma aislada, esta rama se fusiona (*merge*) de vuelta en *develop*.
 - **Releases:** Al completar un *Milestone* y consolidar los cambios en *main*, se genera una *Release* con su etiqueta de versión.
- **Clockify:** Se utiliza exclusivamente para cronometrar el tiempo real dedicado a cada tarea e hito, permitiendo controlar el esfuerzo invertido.
- **Outlook:** Canal formal para comunicaciones con el tutor y envío de entregables o documentación.

El avance del proyecto se valida de forma continua mediante tutorías. La dinámica de estas reuniones consiste en presentar y revisar el software funcionando o el texto redactado hasta esa fecha para recibir el visto bueno del tutor y, acto seguido, definir los siguientes pasos y prioridades a realizar para la próxima tutoría.

3.2. Línea base del alcance

La línea base del alcance define con precisión los límites del proyecto, asegurando que todos los esfuerzos se concentren exclusivamente en los entregables necesarios para cumplir con los objetivos académicos y técnicos. A continuación, se detallan los supuestos, restricciones y exclusiones que condicionan esta planificación.

- **Supuestos:** Se asume la disponibilidad continua de las infraestructuras en la nube (proveedores de alojamiento o bases de datos) durante las fases de desarrollo y despliegue, así como el acceso ilimitado a la documentación de las API y tecnologías que componen el *stack* tecnológico.
- **Restricciones:** El proyecto está sujeto a una restricción temporal vinculada a los plazos institucionales de entrega y defensa del Trabajo Fin de Grado. Asimismo, el desarrollo debe ceñirse al presupuesto simulado y a la capacidad operativa de un único desarrollador.
- **Exclusiones:** Queda fuera del alcance del proyecto el despliegue comercial en producción y el soporte para navegadores web obsoletos o sin compatibilidad con WebSockets modernos.

3.2.1. Estructura del Desglose del Trabajo (EDT)

Para organizar y estructurar el alcance total del proyecto de forma sistemática, se ha elaborado la siguiente Estructura de Desglose del Trabajo (EDT). Esta representación jerárquica descompone el proyecto en paquetes de trabajo y entregables específicos según las directrices de descomposición del estándar ISO/IEC/IEEE 21839 [22], los cuales se detallan en el [diccionario de la EDT](#).

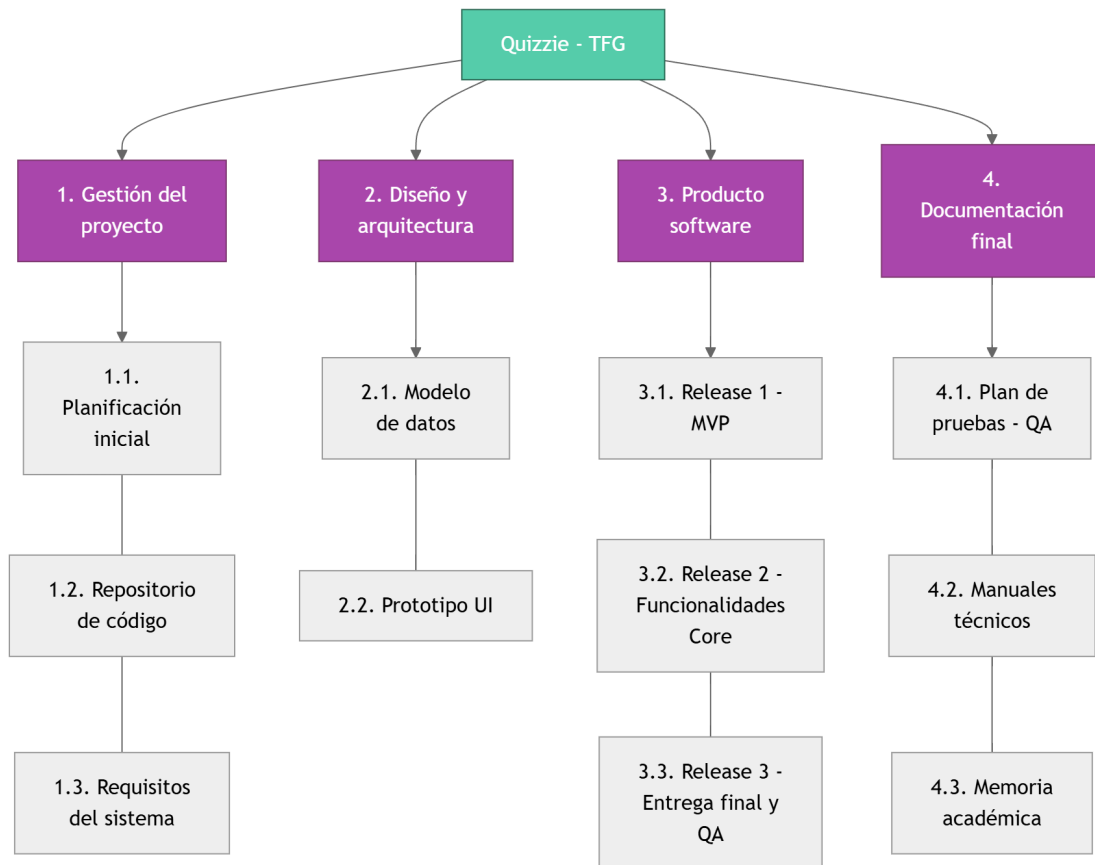


Figura 3.1: Estructura de Desglose del Trabajo (EDT).

3.2.2. Diccionario de la EDT

El diccionario de la EDT define formalmente el alcance de cada entregable y desglosa las actividades necesarias para su consecución, especificando los perfiles profesionales asignados y la estimación temporal.

Paquete 1.0: Gestión del proyecto

ID Entregable	1.1
Nombre	Planificación inicial
Descripción	Acta de constitución del proyecto y cronograma base estimado para fijar los hitos académicos.
Paquete	1. Gestión del proyecto

Tabla 3.1: Datos del entregable 1.1.

Actividad	Recurso	Horas
ACT-01	Project Manager	4h
ACT-02	Project Manager	4h

Tabla 3.2: Actividades del entregable 1.1.

ID Entregable	1.2
Nombre	Repositorio de código
Descripción	Configuración de GitHub con reglas de protección de ramas, flujos de trabajo basados en fusiones (<i>merges</i>) y plantillas estandarizadas para <i>issues</i> .
Paquete	1. Gestión del proyecto

Tabla 3.3: Datos del entregable 1.2.

Actividad	Recurso	Horas
ACT-13	Técnico	11h
ACT-14	Técnico	5h
ACT-15	Técnico	12h

Tabla 3.4: Actividades del entregable 1.2.

ID Entregable	1.3
Nombre	Requisitos del sistema
Descripción	Catálogo de requisitos funcionales y no funcionales, junto al modelado de casos de uso y flujos <i>core</i> de la aplicación.
Paquete	1. Gestión del proyecto

Tabla 3.5: Datos del entregable 1.3.

Actividad	Recurso	Horas
ACT-08	Analista	3h
ACT-09	Analista	4h

Tabla 3.6: Actividades del entregable 1.3.

Paquete 2.0: Diseño y Arquitectura

ID Entregable	2.1
Nombre	Modelo de datos
Descripción	Diseño conceptual, lógico y físico de la base de datos relacional, materializado en el diagrama de entidades del sistema.
Paquete	2. Diseño y Arquitectura

Tabla 3.7: Datos del entregable 2.1.

Actividad	Recurso	Horas
ACT-10	Analista	4h
ACT-11	Analista	4h
ACT-12	Programador Backend	2h

Tabla 3.8: Actividades del entregable 2.1.

ID Entregable	2.2
Nombre	Prototipo UI
Descripción	Diseños visuales y plantillas para las pantallas del sistema.
Paquete	2. Diseño y Arquitectura

Tabla 3.9: Datos del entregable 2.2.

Actividad	Recurso	Horas
ACT-03	Diseñador UI	2h
ACT-04	Diseñador UI	5h
ACT-05	Diseñador UI	5h

Tabla 3.10: Actividades del entregable 2.2.

Paquete 3.0: Producto software

ID Entregable	3.1
Nombre	Release 1 (MVP)
Descripción	Habilitación y despliegue del flujo completo base del sistema, permitiendo la creación de cuestionarios por parte del profesor y la realización de los mismos por los alumnos.
Paquete	3. Producto software

Tabla 3.11: Datos del entregable 3.1.

Actividad	Recurso	Horas
ACT-16	Programador Backend	25h
ACT-17	Programador Frontend	25h
ACT-18	Programador Backend	15h

Tabla 3.12: Actividades del entregable 3.1.

ID Entregable	3.2
Nombre	Release 2 (Funcionalidades Core)
Descripción	Implementación de la capa de comunicación en tiempo real mediante WebSockets y el motor de validación de datos del sistema.
Paquete	3. Producto software

Tabla 3.13: Datos del entregable 3.2.

Actividad	Recurso	Horas
ACT-21	Programador Backend	20h
ACT-22	Programador Frontend	17h
ACT-23	Programador Backend	12h

Tabla 3.14: Actividades del entregable 3.2.

ID Entregable	3.3
Nombre	Release 3 (Entrega final y QA)
Descripción	Integración de funcionalidades avanzadas de gestión, pulido general de la interfaz y despliegue del sistema.
Paquete	3. Producto software

Tabla 3.15: Datos del entregable 3.3.

Actividad	Recurso	Horas
ACT-24	Programador Backend	5h
ACT-25	Programador Frontend	5h
ACT-26	Técnico	3h

Tabla 3.16: Actividades del entregable 3.3.

Paquete 4.0: Documentación final

ID Entregable	4.1
Nombre	Plan de pruebas (QA)
Descripción	Diseño y ejecución de baterías de test de integración y <i>scripts</i> específicos de pruebas de carga.
Paquete	4. Documentación final

Tabla 3.17: Datos del entregable 4.1.

Actividad	Recurso	Horas
ACT-19	Tester	7h
ACT-20	Tester	8h

Tabla 3.18: Actividades del entregable 4.1.

ID Entregable	4.2
Nombre	Manuales técnicos
Descripción	Redacción del manual de usuario y del manual de despliegue.
Paquete	4. Documentación final

Tabla 3.19: Datos del entregable 4.2.

Actividad	Recurso	Horas
ACT-06	Analista	2h
ACT-07	Técnico	1h

Tabla 3.20: Actividades del entregable 4.2.

ID Entregable	4.3
Nombre	Memoria académica
Descripción	Redacción técnica, maquetación y consolidación del documento final del TFG junto con el diseño de las diapositivas para la defensa.
Paquete	4. Documentación final

Tabla 3.21: Datos del entregable 4.3.

Actividad	Recurso	Horas
ACT-27	Analista	70h
ACT-28	Diseñador UI	10h
ACT-29	Project Manager	10h

Tabla 3.22: Actividades del entregable 4.3.

3.3. Línea base del cronograma

A partir de los paquetes de trabajo presentes en la Estructura de Desglose del Trabajo (EDT), se ha planificado la distribución temporal del proyecto. El flujo de trabajo se estructura dividiendo el ciclo de vida en fases temporales que agrupan las actividades y culminan en hitos de control.

3.3.1. Fases del proyecto

El proyecto se divide en 10 fases temporales consecutivas y paralelas, que se han organizado en 4 bloques de trabajo fundamentales:

ID	Nombre	Descripción	Bloque
F-01	Planificación	Planificación inicial, estimación de tiempos y recursos.	Gestión y planificación
F-02	Documentación	Redacción de manuales y documentación técnica de soporte.	Documentación y cierre
F-03	Inicio	Configuración del entorno base y análisis de requisitos.	Gestión y planificación
F-04	CI/CD	Automatización de flujos de trabajo e integración continua.	Ciclo de vida y calidad
F-05	MVP	Construcción de la primera versión mínima funcional.	Desarrollo de código
F-06	Testing	Diseño y ejecución de la batería de pruebas del sistema.	Ciclo de vida y calidad
F-07	Core	Desarrollo de las características principales (WebSockets).	Desarrollo de código
F-08	QA	Estabilización de código y corrección de errores finales.	Desarrollo de código
F-09	Memoria	Consolidación, revisión final y entrega de la memoria.	Documentación y cierre
F-10	Defensa	Preparación del material de soporte y ensayos.	Documentación y cierre

Tabla 3.23: Fases temporales del proyecto.

3.3.2. Lista de actividades

Cada una de las actividades del cronograma conserva una relación directa con los paquetes de trabajo definidos en el diccionario de la EDT. Para garantizar el seguimiento y la trazabilidad temporal, a continuación se detallan todas las tareas del proyecto:

Fase	ID	Actividad
F-01 – Planificación	ACT-01	Redacción del acta de constitución
	ACT-02	Definición de hitos y estimación del cronograma inicial
	ACT-03	Elección de estilos, paleta de colores y componentes básicos
	ACT-04	Diseño de prototipos de las pantallas de gestión del profesor
	ACT-05	Diseño de la interfaz de respuesta para pantallas de alumnos
F-02 – Documentación	ACT-06	Elaboración del manual funcional como guía para usuarios
	ACT-07	Redacción del manual de despliegue técnico de infraestructura
	ACT-08	Elicitación y redacción de requisitos funcionales y no funcionales
	ACT-09	Modelado de diagramas de casos de uso y flujos esenciales
	ACT-10	Diseño del modelo conceptual y lógico de datos
	ACT-11	Modelado del diagrama de entidades relacional
	ACT-12	Generación del <i>script</i> de inicialización de tablas y poblado
F-03 – Inicio	ACT-13	Inicialización del repositorio en GitHub y estructura base
F-04 – CI/CD	ACT-14	Diseño de <i>templates</i> para <i>issues</i> y automatización base
	ACT-15	Desarrollo de <i>scripts</i> de despliegue e integración continuos
F-05 – MVP	ACT-16	Diseño y desarrollo de la persistencia base para cuestionarios
	ACT-17	Implementación de la interfaz de creación de tests y panel base
	ACT-18	Lógica de control y validación para la ejecución asíncrona
F-06 – Testing	ACT-19	Desarrollo y ejecución de test de integración de servicios
	ACT-20	Pruebas de componentes, de carga y <i>end-to-end</i>
F-07 – Core	ACT-21	Desarrollo de la infraestructura síncrona y canales WebSocket
	ACT-22	Implementación en frontend de la conexión reactiva a salas
	ACT-23	Programación del motor de validación y control de integridad
F-08 – QA	ACT-24	Implementación del módulo de autenticación y panel avanzado
	ACT-25	Desarrollo de vistas de historial y exportación de datos
	ACT-26	Puesta a punto, empaquetado, corrección de errores y despliegue
F-09 – Memoria	ACT-27	Redacción y estructuración técnica de la memoria en LaTeX
F-10 – Defensa	ACT-28	Diseño, desarrollo y maquetación de diapositivas de defensa
	ACT-29	Ensayos de oratoria y preparación de la defensa pública

Tabla 3.24: Actividades organizadas por su fase del cronograma.

3.3.3. Lista de hitos

Los hitos representan los puntos de control clave y logros de duración cero dentro del cronograma del proyecto. Su consecución suponen grandes avances en el desarrollo o la finalización de fases completas, y permiten evaluar el progreso general del proyecto.

Fase	ID	Hito de logro	Límite
F-01 – Planificación	H-01	Acta de constitución del proyecto redactada y firmada	21 Ene
	H-02	Líneas base del proyecto constituidas y aprobadas	28 Ene
F-02 – Documentación	H-03	Catálogo de requisitos, casos de uso y modelos de datos definidos	18 Feb
	H-04	Manuales técnicos de usuario y despliegue finalizados	14 Abr
F-03 – Inicio	H-05	Repositorio inicializado y entorno de desarrollo listo	11 Feb
F-04 – CI/CD	H-06	Flujo de CI/CD automatizado y plantillas de trabajo desplegadas	1 Mar
F-05 – MVP	H-07	Funcionalidades del MVP operativas (creación y respuesta de tests)	8 Mar
F-06 – Testing	H-08	Baterías de pruebas de software ejecutadas y superadas con éxito	26 Abr
F-07 – Core	H-09	Conexión y respuesta a cuestionarios en tiempo real operativos	29 Mar
	H-10	Motor de validación y control de integridad de respuestas completado	12 Abr
F-08 – QA	H-11	Vistas de historial de salas pasadas y exportación de datos funcionales	19 Abr
F-09 – Memoria	H-12	Memoria académica consolidada y entregada	17 May
F-10 – Defensa	H-13	Presentación y material de soporte maquetados para la defensa	5 Jun
	H-14	Defensa pública del proyecto ejecutada ante el tribunal	10 Jun

Tabla 3.25: Hitos de control del proyecto.

3.3.4. Diagrama de Gantt

El siguiente diagrama de Gantt[23] sintetiza de manera visual la distribución cronológica del proyecto, clave para el seguimiento del trabajo realizado y asegurar el cumplimiento de los plazos fijados.

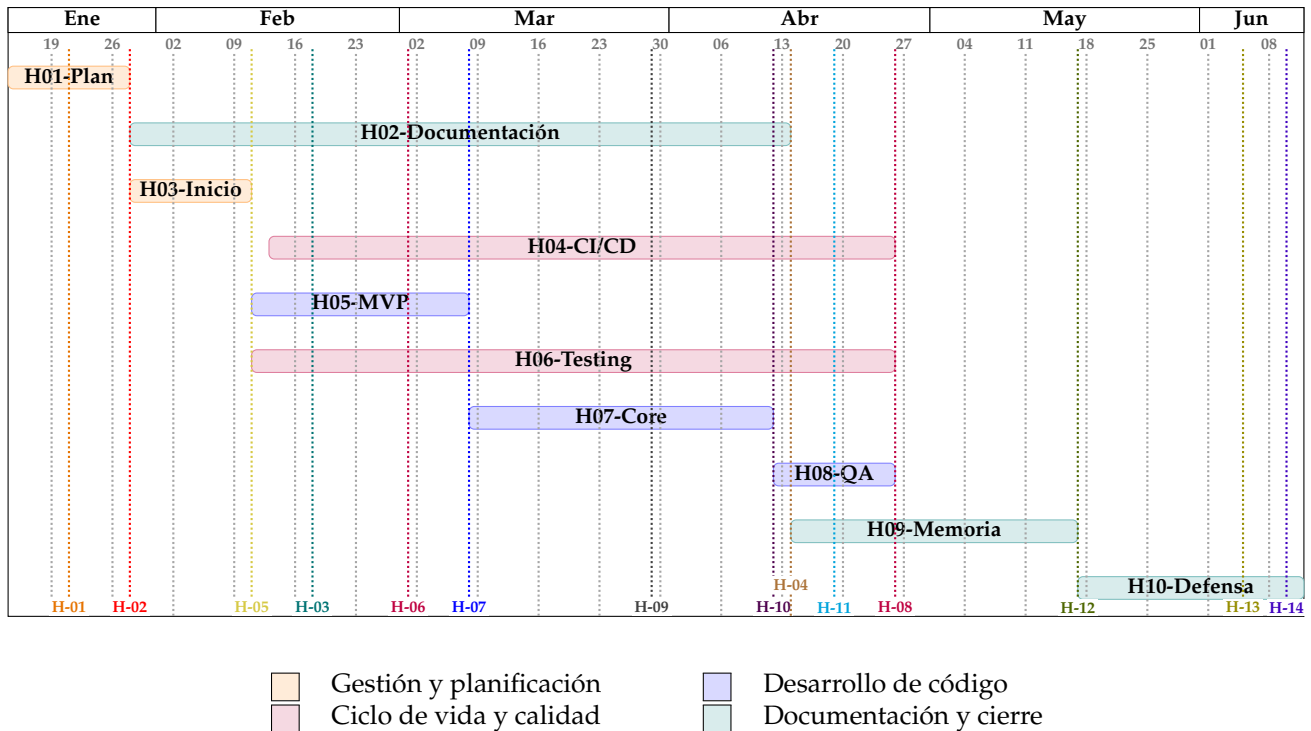


Figura 3.2: Diagrama de Gantt inicial del proyecto.

3.4. Línea base de Costes

El desarrollo de un proyecto como *Quizzie* no solo exige viabilidad técnica, sino también un modelo financiero sostenible que justifique su implantación frente a otras soluciones. Para ello, se analizan a continuación los costes operativos, el impacto económico que supone el escalado de la infraestructura y el Coste Total de Propiedad (TCO).

3.4.1. Análisis de costes

El estudio económico de la plataforma se fundamenta en la clasificación financiera de costes de tecnologías de la información [24], dividiéndose en dos componentes esenciales: los Gastos de Capital (*Capital Expenditure* o CAPEX), que engloban el coste de desarrollo y los recursos fijos iniciales, y los Gastos Operativos (*Operational Expenditure* o OPEX), que representan el coste recurrente de la infraestructura en la nube para la explotación en producción.

Concepto	Coste neto
Costes laborales	11.552,65 €
Equipamiento informático	500,00 €
Licencia comercial anual Google AI Premium (Gemini Pro)	218,07 €
Licencia GitHub Team (6 meses)	21,82 €
Licencia Microsoft Teams Essentials (6 meses)	21,82 €
Reserva de contingencia	12.314,36 €
Total CAPEX	13.545,80 €

Tabla 3.26: Resumen de Gastos de Capital (CAPEX).

Para justificar coste laboral del CAPEX, se adopta un modelo de roles mixtos debido a que el proyecto ha sido ejecutado íntegramente por un único ingeniero. De este modo, cada actividad de la EDT se presupuesta teniendo en cuenta el rol profesional que la llevará a cabo. Para garantizar el rigor y la objetividad de las tarifas aplicadas, los precios base por hora se han extraído del *Informe Final sobre la Consulta Preliminar del Mercado: “Perfiles Profesionales Ámbito Informático”* de la Junta de Andalucía [25]. Siguiendo las directrices de dicho organismo, se utiliza como referencia la métrica de la **media acotada**, un indicador estadístico corregido mediante el test de Tukey que elimina valores atípicos del sector. Sobre la base salarial neta resultante de 8.886,65 €, se ha aplicado un recargo del 30% en concepto de Seguridad Social y costes empresariales de contratación directos del mercado español, consolidando el coste laboral real en **11.552,65 €**. El desglose detallado de las horas por actividad y perfiles profesionales que sustenta este cálculo se presenta en la tabla de **costes netos laborales**.

Recurso (Rol)	Actividad	Precio/hora	Horas	Coste neto
Project Manager	ACT-01	46,09 €	4 h	184,36 €
	ACT-02	46,09 €	4 h	184,36 €
	ACT-29	46,09 €	10 h	460,90 €
	Subtotal		18 h	829,62 €
Analista	ACT-06	38,33 €	2 h	76,66 €
	ACT-08	38,33 €	3 h	114,99 €
	ACT-09	38,33 €	4 h	153,32 €
	ACT-10	38,33 €	4 h	153,32 €
	ACT-11	38,33 €	4 h	153,32 €
	ACT-27	38,33 €	70 h	2.683,10 €
	Subtotal		87 h	3.334,71 €
Diseñador UI	ACT-03	33,43 €	2 h	66,86 €
	ACT-04	33,43 €	5 h	167,15 €
	ACT-05	33,43 €	5 h	167,15 €
	ACT-28	33,43 €	10 h	334,30 €
	Subtotal		22 h	735,46 €
Programador Backend	ACT-12	23,85 €	2 h	47,70 €
	ACT-16	23,85 €	25 h	596,25 €
	ACT-18	23,85 €	15 h	357,75 €
	ACT-21	23,85 €	20 h	477,00 €
	ACT-23	23,85 €	12 h	286,20 €
	ACT-24	23,85 €	5 h	119,25 €
	Subtotal		79 h	1.884,15 €
Programador Frontend	ACT-17	23,85 €	25 h	596,25 €
	ACT-22	23,85 €	17 h	405,45 €
	ACT-25	23,85 €	5 h	119,25 €
	Subtotal		47 h	1.120,95 €
Tester	ACT-19	23,68 €	7 h	165,76 €
	ACT-20	23,68 €	8 h	189,44 €
	Subtotal		15 h	355,20 €
Técnico	ACT-07	19,58 €	1 h	19,58 €
	ACT-13	19,58 €	11 h	215,38 €
	ACT-14	19,58 €	5 h	97,90 €
	ACT-15	19,58 €	12 h	234,96 €
	ACT-26	19,58 €	3 h	58,74 €
	Subtotal		32 h	626,56 €
TOTAL			300 h	8.886,65 €

Tabla 3.27: Costes netos laborales agrupados por perfil profesional.

Concepto	Coste neto
Infraestructura web Frontend (<i>Vercel Pro</i>)	18,18 €/mes
Servidor de aplicaciones y Base de Datos (<i>Render Starter</i>)	12,73 €/mes
Consumo de la API de <i>Gemini Pro</i> para el asistente de IA	15,00 €/mes
Total OPEX	45,91 €/mes

Tabla 3.28: Resumen de Gastos Operativos mensuales (OPEX).

Cabe destacar que las cifras reflejadas en el **resumen de Gastos Operativos mensuales (OPEX)** no constituyen un coste estático, sino que representan una proyección basada en una estimación inicial de 500 usuarios activos mensuales, asumiendo un consumo moderado de recursos y una utilización estándar de la API de inteligencia artificial. En escenarios de escalado masivo, donde el volumen de usuarios concurrentes o la demanda de cómputo aumente significativamente, es previsible que estos costes operativos se incrementen de forma proporcional. Por ello, en la siguiente subsección se detalla y aclara matemáticamente el comportamiento financiero de la plataforma ante diferentes escenarios de carga y volumen de usuarios.

3.4.2. Costes de infraestructura según el número de usuarios

Para prever el impacto financiero de la plataforma en entornos reales, se analiza el comportamiento de los costes operativos (OPEX) bajo tres escenarios de demanda: pesimista, conservador y optimista. Esta clasificación permite evaluar la elasticidad de la infraestructura en la nube (*Render* y *Vercel*) y el consumo variable de la API de *Gemini* según el volumen de uso.

Concepto	Pesimista	Conservador	Optimista
Infraestructura web Frontend (<i>Vercel Pro</i>)	0,00 €	18,18 €	18,18 €
Servidor de aplicaciones y Base de Datos (<i>Render Starter</i>)	0,00 €	12,73 €	27,27 €
Consumo de la API de <i>Gemini Pro</i> para el asistente de IA	2,50 €	15,00 €	60,00 €
Total OPEX	2,50 €/mes	45,91 €/mes	105,45 €/mes

Tabla 3.29: Comparativa de OPEX según escenario de escalado.

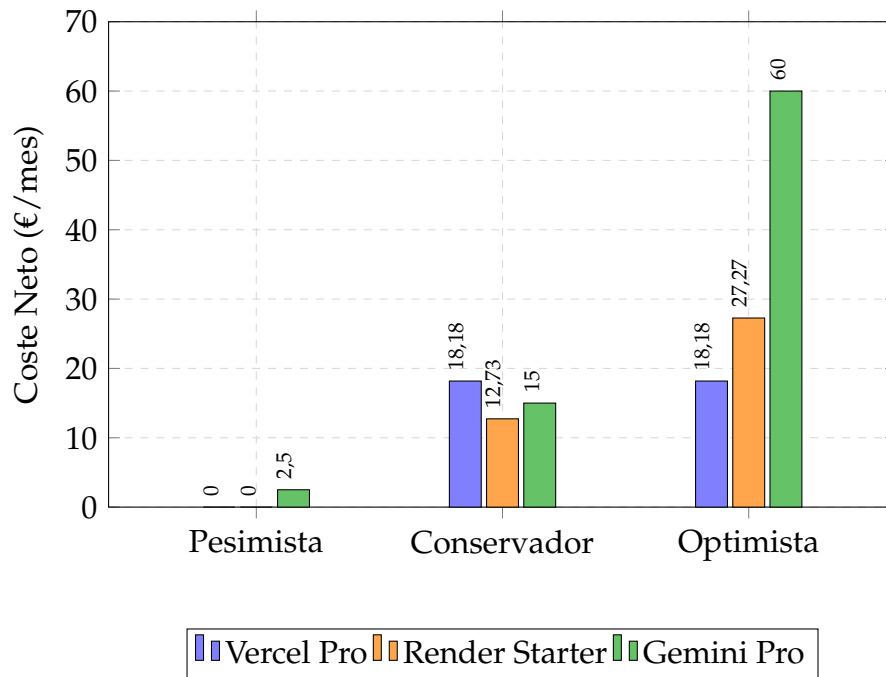


Figura 3.3: Costes mensuales de infraestructura según escenario de escalado.

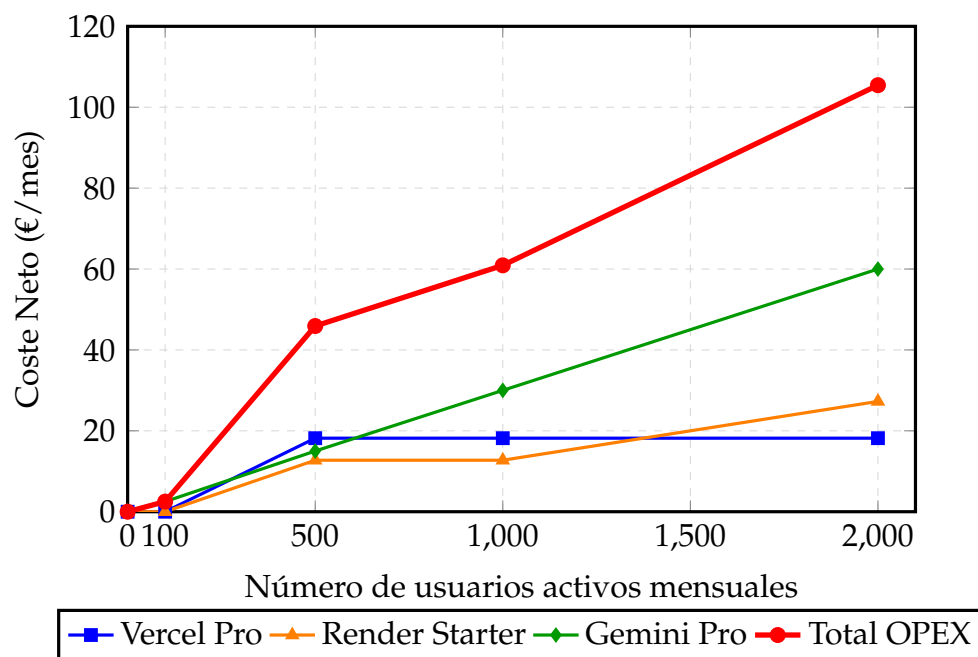


Figura 3.4: Evolución de OPEX en función del volumen de usuarios.

3.4.3. Coste total de propiedad (TCO)

El coste total de propiedad (*Total Cost of Ownership*, TCO) permite evaluar el impacto económico global de la plataforma a medio plazo mediante la metodología de consolidación financiera de costes directos e indirectos planteada por Ellram [26], integrando la inversión inicial de desarrollo (CAPEX) junto con la proyección de los gastos operativos (OPEX) a tres años.

De este modo, asumiendo los gastos de infraestructura del escenario conservador, la proyección resultante se detalla en la siguiente tabla.

Concepto	Año 1	Año 2	Año 3
Inversión inicial (CAPEX)			
Desarrollo de Software (Ingeniería)	5.935,20 €	0,00 €	0,00 €
Gastos operativos (OPEX)			
Infraestructura Frontend (<i>Vercel Pro</i>)	218,16 €	218,16 €	218,16 €
Servidor de aplicaciones y BD (<i>Render Starter</i>)	152,76 €	152,76 €	152,76 €
Consumo de la API de <i>Gemini Pro</i>	180,00 €	180,00 €	180,00 €
Total anual	6.486,12 €	550,92 €	550,92 €
TCO acumulado (3 Años)	7.587,96 €		

Tabla 3.30: TCO proyectado a tres años.

Nota: No se han imputado costes de personal ni horas de mantenimiento dentro del OPEX debido a que el esfuerzo laboral se ha contabilizado íntegramente como parte del CAPEX, contando con las 300 horas reglamentarias del TFG.

3.5. Plan de gestión de cambios y riesgos

Un desarrollo tecnológico está inherentemente sujeto a imprevistos y desviaciones. Para asegurar el éxito y la entrega en plazo de la plataforma, el procedimiento formal de control de cambios y contingencias se estructura según las directrices de la norma IEEE 828 para la gestión de la configuración en ingeniería de software [27], apoyándose de forma sistemática en las tablas de riesgos (**de alcance**, **técnicos** y **temporales**) definidas en la sección de **riesgos iniciales**.

Cualquier propuesta de modificación se gestionará siguiendo este flujo secuencial:

1. **Identificación:** Toda propuesta de modificación se registrará en el *backlog*, especificando si afecta al código de la aplicación, a la redacción de la memoria o a la planificación temporal del proyecto.
2. **Análisis de impacto:** Evaluar el esfuerzo y las horas necesarias para aplicar el cambio, estimando cómo afectará al calendario global de las asignaturas, a la

documentación pendiente o a la arquitectura del sistema.

3. **Resolución:** Decidir de forma justificada si el cambio se aprueba para su ejecución inmediata, se pospone como una mejora futura o se descarta si compromete críticamente los plazos de entrega del TFG.
4. **Implementación y despliegue:** Se ejecuta el cambio en el área correspondiente (programación de código, reajuste de la planificación o corrección de la memoria), realizando las pruebas o revisiones pertinentes antes de dar por actualizada la documentación del proyecto.

4. Estado del arte y fundamentación

El éxito de una plataforma tecnológica no solo radica en su correcto funcionamiento, sino en su capacidad para aportar valor real frente a las soluciones existentes y sustentarse sobre decisiones de ingeniería justificadas. Antes de iniciar el diseño arquitectónico, es clave analizar el ecosistema competitivo y evaluar críticamente las herramientas de desarrollo disponibles en la industria actual.

A lo largo de las siguientes secciones se realiza un análisis del mercado de plataformas de evaluación mediante cuestionarios para identificar sus limitaciones y fijar los factores diferenciadores de *Quizzie*. Asimismo, se detalla la fundamentación tecnológica del proyecto, examinando las alternativas consideradas y justificando la selección del stack definitivo, para concluir con la síntesis estratégica del estudio previo.

4.1. Análisis del mercado

Para fundamentar la necesidad de desarrollar una nueva plataforma de evaluación, es imprescindible analizar el ecosistema de soluciones empleadas actualmente en el ámbito educativo. Aunque el mercado cuenta con soluciones consolidadas, el estudio de sus características revela limitaciones críticas en el control de la presencialidad, la exportación de datos y la asistencia al profesorado. A continuación, se examinan las tres alternativas más representativas:

- **Kahoot!:** Pionera en el ámbito de la gamificación educativa, destaca por su alta capacidad de fidelización y su interfaz visual atractiva. Sin embargo, su enfoque lúdico penaliza la rigurosidad académica: prioriza la velocidad de respuesta sobre la reflexión profunda, apenas ofrece analíticas avanzadas exportables en sus versiones gratuitas y carece de cualquier mecanismo que impida a un alumno participar de forma remota compartiendo el código PIN de la sala.
- **Wooclap:** Orientada a un entorno universitario y corporativo más formal, ofrece una excelente variedad de tipos de preguntas y herramientas de interacción en vivo. Su principal barrera reside en la usabilidad y la agilidad de los flujos de trabajo; el proceso de diseño de cuestionarios extensos resulta monótono y manual para el docente, y no tiene funcionalidades inteligentes de automatización que agilicen la generación de contenido académico.
- **Cuestionarios de Enseñanza Virtual:** Las plataformas institucionales integradas ofrecen una robustez indiscutible a nivel de seguridad, evaluación formal y exportación nativa de calificaciones. No obstante, sacrifican por completo la inmediatez y la experiencia de usuario. Su interfaz rígida, la complejidad técnica para configurar sesiones en tiempo real y la

imposibilidad de mitigar de forma sencilla el fraude por suplantación digital a distancia limitan su efectividad como dinámicas de aula ágiles.

Frente a este escenario, se ha sintetizado un análisis en la [tabla comparativa del mercado](#), evaluando las características esenciales para la docencia presencial moderna. Las carencias detectadas en las alternativas comerciales e institucionales delimitan el espacio de oportunidad para una solución integral.

Característica	Kahoot!	Wooclap	Enseñanza Virtual	Quizzie
Evaluación en tiempo real	Sí	Sí	Limitado	Sí
Acceso rápido (sin registro de alumnos)	Sí	Sí	No	Sí
Verificación de presencialidad física	No	No	No	Sí
Generación de test asistida por IA	No	Limitado	No	Sí
Creación ágil de cuestionarios	Sí	Limitado	No	Sí
Visualización de estadísticas y analíticas	Sí	Sí	Limitado	Sí
Exportación de resultados	De pago	Limitado	Sí	Sí

Tabla 4.1: Tabla comparativa del mercado.

4.2. Factores diferenciadores

Como respuesta a las debilidades del mercado identificadas en la sección anterior, *Quizzie* se diseña no como una alternativa lúdica adicional, sino como un entorno equilibrado que fusiona el dinamismo interactivo con la formalidad académica. Sus elementos innovadores y ventajas competitivas se estructuran en los siguientes pilares:

- **Validación de presencia física mediante QR:** Para evitar el fraude por suplantación o que los alumnos participen a distancia, la plataforma genera un código QR único en el dispositivo del estudiante justo al finalizar el test, el cual incluye su nombre y la nota obtenida. El docente es quien escanea este código desde su propio dispositivo al terminar la clase, confirmando de manera inequívoca la asistencia física del alumno y registrando su calificación en el acto.
- **Creación inteligente asistida por IA:** Para resolver el flujo de trabajo monótono de diseñar cuestionarios desde cero, *Quizzie* integra un motor de Inteligencia Artificial que asiste al docente generando automáticamente baterías de preguntas coherentes a partir del temario o texto introducido, agilizando drásticamente la preparación de la clase.
- **Exportación directa de calificaciones:** La plataforma simplifica la gestión de las calificaciones permitiendo descargar las notas de los alumnos directamente en formatos estándar como CSV o PDF, listos para su revisión o publicación en canales oficiales.

- **Accesibilidad y agilidad en el aula:** Los alumnos acceden de forma inmediata sin necesidad de registros previos ni cuentas obligatorias, mientras que el profesor dispone de un panel simplificado para lanzar y controlar el flujo del cuestionario con un solo clic.

4.3. Fundamentación tecnológica

A continuación, se presenta la arquitectura tecnológica que da soporte a *Quizzie*. Como vista general de la infraestructura, la siguiente tabla muestra la distribución definitiva de las herramientas seleccionadas siguiendo un orden ascendente desde la persistencia hasta el despliegue del cliente.

Base de datos	SQLite
ORM / Validación	SQLModel
Backend	FastAPI (Python)
Lenguaje frontend	TypeScript
Frontend (SPA)	React + Vite
Estilos frontend	CSS <i>vanilla</i>
Alojamiento servidor	Render
Alojamiento cliente	Vercel
Flujo de CI/CD	GitHub Actions

Tabla 4.2: Resumen del stack tecnológico de Quizzie.

4.3.1. Alternativas consideradas y justificación de las decisiones

Para fundamentar la elección de cada componente del sistema, se analizan a continuación las matrices comparativas correspondientes a cada uno de los bloques lógicos de la aplicación.

Persistencia de datos

La gestión del almacenamiento del proyecto requiere evaluar la estructura de los datos frente a la complejidad de la infraestructura en la nube.

Tecnología	Ventajas	Desventajas
Opción elegida		
SQLite	Portabilidad en un solo archivo; cero administración de red y coste económico nulo.	Bloqueo del archivo completo durante la concurrencia de escritura masiva.
Alternativas valoradas		
PostgreSQL	Escalabilidad masiva y gestión avanzada de accesos y concurrencia.	Elevada complejidad de despliegue y administración de accesos en producción.
MySQL	Motor relacional maduro, robusto y con gran soporte de la comunidad.	Costes económicos asociados a instancias en la nube innecesarios en esta fase.
MongoDB	Flexibilidad de esquemas basada en documentos NoSQL y alta velocidad de lectura.	Descartado porque el dominio de datos del proyecto es puramente relacional.

Tabla 4.3: Alternativas tecnológicas para la base de datos.

Mapeo objeto-relacional (ORM)

Se evalúa la forma de interactuar con el motor relacional buscando la mayor limpieza y eficiencia en el código.

Tecnología	Ventajas	Desventajas
Opción elegida		
SQLModel	Cumple el principio DRY: fusiona eficientemente esquemas de API y modelos de BD.	Herramienta más reciente en el ecosistema, con menor volumen de hilos de soporte.
Alternativas valoradas		
SQLAlchemy + Pydantic	Estándar clásico de la industria con máxima madurez y documentación.	Duplicidad de código obligatoria al tener que declarar los modelos por separado.

Tabla 4.4: Alternativas tecnológicas para el ORM y validación.

Backend y servidor de sockets

La lógica de negocio exige un entorno que soporte una alta concurrencia interactiva en tiempo real.

Tecnología	Ventajas	Desventajas
Opción elegida		
FastAPI	Rendimiento asíncrono nativo (<i>async/await</i>) según los principios de aplicaciones web modernas de Fox y Patterson [28], ideal para mantener conexiones WebSocket concurrentes.	Estructura menos guiada en comparación con frameworks monolíticos.
Alternativas valoradas		
Node.js	Estándar tradicional para WebSockets; unifica el lenguaje del proyecto a JavaScript.	Se descartó para priorizar el uso de Python debido a su ecosistema de IA.
Django	Framework muy robusto, maduro y con arquitectura <i>batteries-included</i> basada en patrones empresariales de Fowler [29].	Estructura fundamentalmente síncrona por defecto y peso excesivo para una API REST.
Flask	Microframework extremadamente simple, minimalista y rápido de configurar.	Carece de herramientas nativas de validación y de soporte asíncrono integrado.

Tabla 4.5: Alternativas tecnológicas para el desarrollo del backend.

Lenguaje del cliente web

Se contrasta la seguridad del tipado frente a la velocidad de desarrollo directa en el frontend.

Tecnología	Ventajas	Desventajas
Opción elegida		
TypeScript	Tipado estático estricto; mitiga errores con contratos claros para WebSockets.	Requiere tiempo adicional en el ciclo de desarrollo para configurar interfaces.
Alternativas valoradas		
JavaScript <i>vanilla</i>	Curva de aprendizaje nula y compatibilidad nativa directa en el navegador.	Falta de seguridad de tipos en la mensajería asíncrona en tiempo real.

Tabla 4.6: Alternativas para el lenguaje de programación frontend.

Frontend (interfaz de usuario)

Se analiza la arquitectura de las interfaces y la velocidad en la compilación del entorno de desarrollo.

Tecnología	Ventajas	Desventajas
Opción elegida		
React + Vite	<i>Virtual DOM</i> reactivo siguiendo las pautas de React de Porcello y Banks [30], patrones de interfaz según MacDonald [31] y compilación HMR instantánea.	Curva de aprendizaje inicial superior frente a opciones más básicas.
Alternativas valoradas		
Angular	Framework corporativo muy sólido, estructurado y altamente escalable.	Curva de aprendizaje muy empinada y verbosidad excesiva para el alcance del TFG.
Vue.js	Alternativa equilibrada con un sistema de reactividad sencillo y progresivo.	Ecosistema y penetración en el mercado laboral inferior en comparación con React.
Svelte	Máxima ligereza al eliminar el <i>virtual DOM</i> y compilar a código nativo.	Comunidad reducida y menor disponibilidad de librerías de terceros maduras.
Webpack	Estándar histórico para el empaquetado de aplicaciones web.	Procesos de construcción muy lentos en desarrollo local comparado con Vite.

Tabla 4.7: Alternativas tecnológicas para la interfaz de usuario.

Estilización visual de la interfaz

Se evalúa el equilibrio entre la velocidad de diseño y la sobrecarga estética en la aplicación del cliente.

Tecnología	Ventajas	Desventajas
Opción elegida		
CSS <i>vanilla</i>	Control total de layouts y animaciones sin dependencias ni sobrecarga.	Requiere la escritura manual de todos los estilos desde cero.
Alternativas valoradas		
Tailwind CSS	Maquetación veloz mediante clases de utilidad atómicas directas.	Saturación excesiva del marcado JSX, perjudicando la legibilidad del código.
Bootstrap	Componentes preconstruidos adaptables y fáciles de implementar.	Estética genérica y obsoleta; complejiza la personalización visual premium.

Tabla 4.8: Alternativas tecnológicas para el diseño de estilos visuales.

Infraestructura y despliegue

Se analizan los entornos de alojamiento priorizando la compatibilidad con canales bidireccionales y la viabilidad económica.

Tecnología	Ventajas	Desventajas
Opción elegida		
Vercel + Render	Plan gratuito sin tarjeta, GitOps automatizado y soporte ASGI para WebSockets.	El plan gratuito de Render sufre de retardos iniciales por <i>cold start</i> .
Alternativas valoradas		
AWS	Control absoluto e ilimitado de la infraestructura de red.	Configuración excesivamente compleja y riesgo de costes económicos imprevistos.
Heroku	Entorno PaaS históricamente óptimo para servidores de sockets interactivos.	Definición obsoleta por la eliminación por completo de sus planes gratuitos.

Tabla 4.9: Alternativas para la infraestructura y el despliegue.

4.3.2. Limitaciones tecnológicas

A pesar de la idoneidad del stack seleccionado, se han identificado las siguientes limitaciones técnicas inherentes a las decisiones tomadas:

- **Cold start en Render (plan gratuito):** El contenedor del backend se desactiva tras 15 minutos sin recibir peticiones. Esto genera un retardo inicial de entre 30 y 50 segundos en la primera interacción de un usuario mientras el contenedor vuelve a iniciarse.
- **Concurrencia de escritura en SQLite:** SQLite bloquea la base de datos a nivel de archivo completo durante las operaciones de escritura. Aunque es imperceptible para un grupo estándar de alumnos, limitaría el escalado para miles de usuarios simultáneos registrando respuestas al unísono.
- **Falta de SSR (*server-side rendering*) en el frontend:** Al ser una SPA de React pura, el indexado por motores de búsqueda (SEO) es limitado. Sin embargo, al tratarse de una aplicación académica de acceso cerrado mediante credenciales, esta limitación no compromete su viabilidad.

4.3.3. Herramientas de soporte al desarrollo y control de calidad

La robustez del sistema y el cumplimiento de los estándares de calidad se articulan mediante las siguientes herramientas de soporte:

- **Control de versiones y flujo de trabajo:** Se utiliza **Git** como sistema de control local y **GitHub** como repositorio remoto. La gestión de ramas aplica el modelo de ramificación de Driessen [32] (*Git Flow*), manteniendo una rama estable (*main*), una de integración (*develop*) y ramas de características (*feat/**). Siguiendo las pautas de organización de Westby [33], cada funcionalidad se desarrolla de forma aislada en su propia rama antes de fusionarse directamente en *develop*.
- **Gestión de tareas y proyectos:** Para el seguimiento del cronograma se utiliza **GitHub Projects** estructurado como un tablero *Kanban*. Esto garantiza la trazabilidad del trabajo al asociar los *commits* y ramas de desarrollo con su correspondiente tarea (*issue*) e hito (*milestone*).
- **Gestión de tareas y proyectos:** Para el seguimiento del cronograma se utiliza **GitHub Projects** estructurado como un tablero *Kanban*. Esto garantiza la trazabilidad del desarrollo al vincular cada *pull request* con su correspondiente tarea (*issue*) e hito (*milestone*).
- **Estandarización de commits:** Para garantizar un historial de Git limpio, semántico y legible, se integra **Commitizen**. Esta herramienta obliga al autor a seguir la convención de *Conventional Commits*, categorizando de forma estricta cada cambio (por ejemplo, *feat*, *fix*, *docs* o *refactor*), lo que facilita la auditoría del código y la futura generación automatizada de diarios de cambios (*changelogs*).
- **Entorno de desarrollo integrado (IDE):** El entorno principal de trabajo es **Visual Studio Code**, seleccionado por su ligereza y su ecosistema de extensiones. Entre ellas, se ha configurado **LaTeX Workshop** para centralizar la composición y compilación local de la memoria académica mediante los motores de **MiKTeX** y **Strawberry Perl**.
- **Gestión de entornos y dependencias:** En el entorno de backend se ha adoptado **uv**, una herramienta de gestión de paquetes y entornos virtuales para Python extremadamente rápida escrita en Rust, que sustituye de forma eficiente a herramientas tradicionales como *pip* y *poetry*. Para el entorno frontend se utiliza **npm** como gestor de paquetes estándar de Node.js.
- **Generación y lectura de códigos QR:** Para dar soporte al sistema de validación de presencia física, en el frontend se integra la librería **qrcode.react** para renderizar de manera eficiente y reactiva el código QR dinámico en la pantalla del alumno al finalizar el test. Por la parte del docente, se utiliza **html5-qrcode** para acceder a la cámara del dispositivo móvil y procesar el escaneo en tiempo real con baja latencia.
- **Linter y formateo:** En el backend se utiliza **Ruff**, *linter* y formateador ultrarrápido programado en Rust que optimiza el cumplimiento de PEP 8 y unifica tareas que antes requerían múltiples librerías independientes (como *Flake8*, *Black* e *isort*). En el frontend se emplea **ESLint** con *flat config* para controlar la calidad de TypeScript junto a **Prettier** para garantizar un formateo de código unificado y consistente.

- **Pre-commit hooks:** Con la librería **pre-commit**, se interceptan los *commits* locales para garantizar que solo se sube código limpio que supere de forma automatizada las reglas de formato, tipado y análisis estático antes de salir del entorno local.
- **Pruebas unitarias y de integración:** Se utiliza **pytest** en el backend junto a **pytest-cov** para el cálculo de cobertura, y **Vitest** con **React Testing Library** en el frontend para evaluar de manera aislada el comportamiento de los componentes. Para flujos de usuario extremo a extremo (E2E), se integra **Playwright**, permitiendo simular múltiples navegadores concurrentes.
- **Pruebas de carga:** Se emplea **Locust** para verificar de manera empírica la capacidad del servidor y medir la latencia del protocolo WebSocket al soportar la avalancha de respuestas concurrentes de los alumnos.
- **SonarCloud:** Se ha configurado este auditor externo de análisis estático como parte del flujo de integración y entrega continuas (CI/CD) en GitHub Actions, aplicando los principios de canalizaciones de despliegue automatizadas descritos por Farley [5] para exigir una meta de calidad estricta (*quality gate A*, duplicación < 3 %, cobertura de tests > 80 %).

4.4. Conclusiones del estudio previo

El análisis del estado del arte y el estudio comparativo detallados en este capítulo ponen de manifiesto una brecha crítica en el ámbito de la evaluación gamificada en clase. A pesar de la madurez de las soluciones analizadas, el diagnóstico del mercado revela que ninguna alternativa resuelve con éxito el problema del fraude por suplantación a distancia ni ofrece flujos optimizados que mitiguen la carga administrativa del profesorado. La falta de mecanismos de control de asistencia eficaces en las herramientas actuales y la rigidez de los entornos virtuales tradicionales justifican plenamente la necesidad de desarrollar una solución integral que logre unificar inmediatez, control de presencia real e interoperabilidad.

Como respuesta directa a este escenario de oportunidad, la propuesta de valor de *Quizzie* se consolida a través de los factores diferenciadores definidos. El núcleo innovador del proyecto se materializa en el diseño de un flujo de validación mediante códigos QR, devolviendo al docente el control físico del aula sin penalizar el ritmo de la sesión. Asimismo, para contrarrestar la monotonía organizativa identificada en la competencia, la plataforma complementa su arquitectura con herramientas secundarias destinadas a agilizar la gestión diaria: la generación de cuestionarios asistida por Inteligencia Artificial y la exportación directa de calificaciones en formatos universales (CSV y PDF). Con ello, el proyecto trasciende el enfoque puramente lúdico y se posiciona como una herramienta formal y eficiente para el ecosistema educativo.

Para hacer esto posible, se ha elegido a conciencia un stack tecnológico donde cada pieza responde directamente a los retos de rendimiento del proyecto. La

combinación de una base de datos relacional y ligera en local junto a un servidor backend asíncrono permite gestionar la concurrencia de los cuestionarios en el aula con total fluidez, demostrando que es viable construir un sistema interactivo robusto sin depender de infraestructura costosa en la nube. Esta selección técnica se cierra de forma lógica con el ecosistema de automatización integrado; al coordinar el formateo, las pruebas de rendimiento y la auditoría continua antes de cada despliegue, se garantiza que *Quizzie* no solo funcione bien en teoría, sino que sea un software estable, mantenible y listo para su uso real en clase.

5. Análisis de requisitos

Una vez contrastada la viabilidad de las ideas iniciales y analizadas las oportunidades del mercado, es imperativo definir de manera precisa el comportamiento del sistema antes de comenzar la fase de desarrollo técnico. Explicitar las funciones de negocio en este punto es un paso clave para construir un producto robusto y libre de ambigüedades. Con el fin de transformar de forma efectiva las aspiraciones generales plasmadas en los **objetivos del proyecto**, la lógica de la aplicación se desglosa inicialmente en doce objetivos funcionales concretos que asientan las bases del sistema.

ID	Nombre	Descripción
OBJ-01	Módulo de usuario y autenticación	Proveer un sistema de autenticación seguro para profesores.
OBJ-02	Gestión de cuestionarios	Permitir la creación, edición y configuración de cuestionarios de preguntas.
OBJ-03	Creación de salas	Habilitar sesiones de test únicas identificadas mediante un código PIN.
OBJ-04	Acceso rápido	Permitir el acceso de alumnos a las salas mediante PIN y <i>nickname</i> sin registro.
OBJ-05	Interacción en tiempo real	Sincronizar el estado de la partida y preguntas entre dispositivos.
OBJ-06	Verificación QR	Generar comprobante digital QR para validación presencial por el profesor.
OBJ-07	Persistencia validada	Almacenar en histórico solo los resultados verificados físicamente.
OBJ-08	Estadísticas y análisis	Visualizar estadísticas de rendimiento basadas en datos validados.
OBJ-09	Asistente IA	Integrar IA para generar preguntas automáticamente según temáticas.
OBJ-10	Escaneo de código QR	Habilitar el uso de la cámara para la lectura ágil de códigos QR.
OBJ-11	Gestión de grupos	Organizar alumnos en clases para un seguimiento histórico.
OBJ-12	Exportación de datos	Permitir la descarga de resultados en formatos CSV/Excel.

Tabla 5.1: Objetivos funcionales de *Quizzie*.

A lo largo de las siguientes secciones se identifican en primer lugar los actores que interactúan con la aplicación para, posteriormente, modelar su comportamiento mediante historias de usuario y casos de uso detallados.

Asimismo, se determinan los requisitos de información, las reglas de negocio y los requisitos funcionales asociados a sus respectivos criterios de aceptación. El capítulo concluye con la definición de las restricciones no funcionales y la matriz de trazabilidad que garantiza la cobertura total del desarrollo propuesto.

5.1. Historias de usuario

Una vez definidos los perfiles que interactúan con la plataforma, el comportamiento del sistema se define mediante historias de usuario siguiendo la metodología de elicitación de requisitos descrita por Sommerville [34]. Estas especificaciones permiten capturar las necesidades reales de cada actor siguiendo la estructura clásica de rol [**como**], acción [**quiero**] y beneficio [**para**]. Para facilitar la trazabilidad y el seguimiento de las historias, se han dividido y numerado de forma independiente según el perfil al que corresponden: el alumno (*HU-AL*) y el profesor (*HU-PR*).

ID	Descripción
HU-AL-01	Como alumno, quiero unirme a una sala solo con un PIN y un apodo para participar de forma inmediata sin necesidad de crear una cuenta.
HU-AL-02	Como alumno, quiero recibir la pregunta y las opciones de respuesta en mi móvil de forma sincronizada para competir en igualdad de condiciones con mis compañeros.
HU-AL-03	Como alumno, quiero saber si mi respuesta ha sido correcta justo después de enviarla para reforzar mi aprendizaje en el momento.
HU-AL-04	Como alumno, quiero que se genere un código QR único al finalizar el test para mostrárselo al profesor y que mi nota sea oficializada.
HU-AL-05	Como alumno, quiero ver mi posición en el ranking después de cada bloque de preguntas para motivarme durante la realización del cuestionario.
HU-AL-06	Como alumno, quiero poder reengancharme a la partida si pierdo la conexión a internet para no perder mi progreso y poder terminar el test.
HU-AL-07	Como alumno, quiero recibir puntos extra por responder rápido o mantener una racha de aciertos para sentirme recompensado por mi agilidad y constancia.

Tabla 5.2: Historias de usuario asociadas al rol de Alumno.

ID	Descripción
HU-PR-01	Como profesor, quiero crear y organizar mis propios cuestionarios para disponer de un repositorio de actividades personalizado por asignatura o tema.
HU-PR-02	Como profesor, quiero poder editar mis cuestionarios existentes para corregir errores o crear variantes rápidas para diferentes grupos.
HU-PR-03	Como profesor, quiero crear y administrar una sala con un PIN único para controlar el acceso de los alumnos y decidir cuándo comienza el test.
HU-PR-04	Como profesor, quiero ver en tiempo real qué están respondiendo los alumnos y quiénes se van conectando para llevar un control del progreso de la clase.
HU-PR-05	Como profesor, quiero poder cerrar la sala una vez iniciada la actividad para que ningún alumno se una tarde o de forma externa.
HU-PR-06	Como profesor, quiero escanear el móvil del alumno para confirmar su presencia física y validar su nota en el sistema evitando respuestas no autorizadas.
HU-PR-07	Como profesor, quiero crear grupos o clases para tener organizados a mis alumnos y facilitar el volcado de notas histórico.
HU-PR-08	Como profesor, quiero visualizar estadísticas de acierto por pregunta al finalizar para detectar conceptos que no han quedado claros y reforzarlos.
HU-PR-09	Como profesor, quiero generar borradores de preguntas mediante IA para reducir el tiempo de preparación de mis clases.
HU-PR-10	Como profesor, quiero exportar los resultados verificados a CSV para integrarlos en mis herramientas de gestión docente externas.
HU-PR-11	Como profesor, quiero configurar si se otorgan puntos extra por rapidez o racha para adaptar el nivel de competitividad de la sesión.
HU-PR-12	Como profesor, quiero decidir si el ranking se muestra tras cada pregunta para gestionar el ritmo y la atención de la clase.
HU-PR-13	Como profesor, quiero añadir etiquetas e imágenes a mis cuestionarios para identificar los y personalizarlos visualmente.
HU-PR-14	Como profesor, quiero acceder al historial de salas pasadas para revisar resultados y analizar el progreso de mis alumnos en el tiempo.

Tabla 5.3: Historias de usuario asociadas al rol de Profesor.

5.2. Reglas de negocio y restricciones

El correcto funcionamiento de la plataforma *Quizzie* está sujeto a un conjunto de directrices operativas y limitaciones técnicas que garantizan tanto la validez funcional como la seguridad del sistema.

5.2.1. Reglas de negocio

Las reglas de negocio definen las políticas, derechos y leyes lógicas que gobiernan el flujo de trabajo de la aplicación:

- **Ciclo de vida de las salas:** Una sala de evaluación podrá encontrarse en espera, en juego, en fase de verificación o cerrada. Los alumnos carecerán de privilegios para alterar el estado operativo de la sesión.
- **Acceso a salas activas:** Un alumno solo podrá participar en una sala activa si introduce el código PIN correcto y un *nickname* que no esté siendo utilizado por otro participante dentro de la misma sesión.
- **Validación presencial de calificaciones:** Ninguna calificación obtenida en una sesión en vivo se consolidará en el histórico permanente ni será válida para su exportación si no ha sido verificada físicamente en el aula mediante el escaneo del código QR del alumno por parte del dispositivo del profesor.
- **Caducidad y unicidad del token QR:** El código QR generado por el dispositivo del alumno al finalizar el test contendrá un token criptográfico único asociado a su sesión. Este token será de un solo uso; una vez escaneado y marcado como "Verificado", el sistema bloqueará cualquier intento posterior de lectura para evitar su duplicación.
- **Propiedad y gestión de cuestionarios:** Los cuestionarios creados, editados o eliminados lógicamente en la plataforma estarán asociados únicamente al profesor que los diseñó. Un docente no podrá visualizar, modificar ni utilizar para una sala los cuestionarios propiedad de otro usuario del sistema.
- **Generación asistida por IA:** El sistema proporcionará la generación automatizada de preguntas mediante Inteligencia Artificial, actuando como una herramienta de ayuda para el profesor. Todo contenido generado se considerará una propuesta preliminar, siendo responsabilidad exclusiva del profesor su revisión, edición y aprobación final.
- **Almacenamiento del historial de salas:** Una sala de evaluación se trasladará de forma definitiva al historial de sesiones pasadas únicamente tras concluir con éxito la fase de verificación presencial. En este registro permanente solo se consolidarán las calificaciones e identidades de aquellos alumnos que completaron el flujo de validación mediante código QR.
- **Análisis de rendimiento:** El sistema generará métricas analíticas orientadas a facilitar la detección de tendencias de aprendizaje en el aula, identificando

automáticamente los conceptos con mayor índice de error y los patrones de acierto global del alumnado.

- **Exportación de resultados:** El profesorado dispondrá del derecho de portabilidad de las calificaciones oficiales del sistema, permitiendo la generación y descarga local de boletines de calificaciones en formatos estandarizados como PDF o CSV.
- **Seguimiento por grupos:** La plataforma permitirá la estructuración de los alumnos en grupos o clases con el fin de centralizar y evaluar la evolución histórica de un mismo conjunto de estudiantes a lo largo de diferentes sesiones académicas y cuestionarios.

5.2.2. Restricciones

Para que las políticas de negocio descritas anteriormente puedan ejecutarse de manera efectiva, el sistema debe acatar una serie de restricciones técnicas y de diseño, aplicando el principio de arquitectura con autoridad en el servidor expuesto por Gregory [35]:

- **Validación de *uvus*:** El sistema restringirá el formato del *uvus* de los alumnos para que coincida estrictamente con el patrón definido por la Universidad de Sevilla. El servidor validará mediante expresiones regulares que la cadena cumpla con los formatos oficiales, rechazando cualquier formato de nombre inválido.
- **Dimensionamiento de cuestionarios:** En la creación de contenido se impondrá un límite estructural rígido por motivos de maquetación de la interfaz móvil y rendimiento de la base de datos, restringiendo cada cuestionario a un máximo de 30 preguntas y fijando un límite máximo de 8 opciones de respuesta por cada enunciado.
- **Persistencia ante desconexiones:** Ante una pérdida accidental de conectividad a internet por parte de un alumno en plena partida, el backend retendrá su estado y puntuación. Esto permitirá su reconexión mientras la sala siga activa sin perder las respuestas previas a la desconexión.
- **Protocolos de comunicación síncrona:** Para garantizar la sincronización en tiempo real y asegurar que los eventos de la partida se procesen con la latencia mínima requerida, la comunicación bidireccional entre los dispositivos de la sala (profesor/alumnos) estará restringida exclusivamente al uso del protocolo seguro *WebSockets* (WSS).
- **Cifrado de credenciales:** El sistema operativo del backend prohibirá el almacenamiento de contraseñas en texto plano dentro de la base de datos. Será obligatorio aplicar un algoritmo de *hashing* seguro y unidireccional antes de cualquier operación de persistencia de credenciales de profesores.
- **Seguridad del código QR:** El *token* incrustado en el código QR del alumno deberá generarse mediante criptografía segura y no falsificable desde el

servidor.

5.3. Requisitos de información

Los requisitos de información determinan los conjuntos de datos que el sistema debe almacenar de forma persistente para garantizar el correcto funcionamiento de la aplicación y dar soporte a las reglas de negocio e historias de usuario definidas. En esta fase de análisis no se busca detallar la estructura técnica de las tablas, las restricciones de integridad ni sus atributos específicos. Dichos aspectos se detallarán en el capítulo 6. *Diseño y arquitectura*.

ID	Nombre	Descripción
RI-01	Profesores y cuentas	Credenciales de acceso de los profesores (correo electrónico y el <i>hash</i> de la contraseña) y su nombre visible.
RI-02	Cuestionarios y preguntas	Datos relativos a los tests diseñadas por los profesores. Incluye los campos básicos del cuestionario (título, descripción) y la composición de sus preguntas y opciones asociadas.
RI-03	Configuración y control de salas	Código PIN único de acceso, el estado operativo de la sala (espera, en juego, en fase de verificación o cerrada), el temporizador del servidor para la sincronización mediante <i>WebSockets</i> .
RI-04	Registro temporal de participación	Datos de alumnos conectados identificados mediante su <i>uvus</i> , opciones seleccionadas y puntuaciones obtenidas.
RI-05	Comprobantes de presencialidad y tokens QR	Validación de notas y mitigación del fraude digital. Debe almacenar el token criptográfico seguro y firmado digitalmente que se asocia a cada alumno al terminar un test, su estado de verificación actual.
RI-06	Histórico de calificaciones y respuestas	Salas que han completado con éxito la fase de verificación presencial. Debe registrar los mismos datos de cada alumno que el registro temporal, pero con el estado de verificación completado y persistiendo dichos datos.
RI-07	Gestión de grupos	Etiquetas de grupo o clase, profesor responsable y lista de alumnos asociados.

Tabla 5.4: Requisitos de información.

5.4. Requisitos funcionales

Los requisitos funcionales determinan las acciones específicas, comportamientos y servicios que la plataforma *Quizzie* debe ejecutar para satisfacer las necesidades de los usuarios. Este catálogo traduce las historias de usuario y las políticas lógicas del sistema en directrices operativas de desarrollo. A continuación, se detallan los requisitos funcionales del sistema:

ID	Nombre	Descripción	Prioridad
RF-01	Gestión de registro	Registro de nuevos profesores con validación de correo electrónico y <i>hash</i> de contraseña.	Alta
RF-02	Inicio de sesión	Autenticación de docentes mediante correo electrónico y contraseña cifrada con emisión de tokens JWT.	Alta
RF-03	Baja de usuarios	Eliminación de la cuenta de profesor y todos sus datos asociados de forma permanente del sistema.	Media
RF-04	Recuperación contraseña	Solicitud y tramitación de restablecimiento de credenciales de acceso vía correo electrónico.	Baja
RF-05	Edición perfil	Modificación de los datos de usuario del profesor, incluyendo el nombre visible y la contraseña.	Media
RF-06	Cierre de sesión	Invalidación del token de autenticación activo en el servidor y redirección automática al formulario de acceso.	Alta
RF-07	Crear cuestionario	Creación de nuevas pruebas de evaluación con un límite máximo estructurado de hasta 30 preguntas.	Alta
RF-08	Listar cuestionarios	Visualización del catálogo de pruebas diseñadas por el profesor, ordenadas cronológicamente por fecha.	Alta
RF-09	Mostrar ranking	Presentación de la tabla de clasificación de alumnos después de cada pregunta y al finalizar la sesión.	Media
RF-10	Aleatoriedad	Configuración opcional para alterar de forma automática el orden de las preguntas y sus respectivas respuestas.	Baja
RF-11	Configurar tiempo	Definición personalizada del tiempo límite de respuesta para cada enunciado durante la creación de la sala.	Media
RF-12	Importación	Carga masiva de preguntas y opciones desde archivos externos estructurados exclusivamente en formato <code>.txt</code> o <code>.csv</code> .	Media
RF-13	Edición de cuestionarios	Modificación integral del título, descripción, enunciados y opciones de respuestas de un test existente.	Media
RF-14	Eliminación de datos	Borrado lógico en el sistema de cuestionarios o preguntas individuales sin destruir el histórico pasado.	Media

Tabla 5.5: Requisitos funcionales (1/3)

ID	Nombre	Descripción	Prioridad
RF-15	Crear sala	Instanciación en tiempo real de una sesión de evaluación protegida mediante un código PIN único de acceso.	Alta
RF-16	Validar PIN	Verificación inmediata en el servidor de que el código PIN introducido corresponde a una sala en estado de espera.	Alta
RF-17	Validar <i>uvus</i>	Control de acceso que valida el formato alfanumérico institucional de la Universidad de Sevilla y mitiga duplicados.	Alta
RF-18	Sala de espera	Gestión y mantenimiento del flujo de alumnos conectados en tiempo real mediante <i>WebSockets</i> antes del inicio.	Alta
RF-19	Cerrar sala	Finalización manual o automática de la sesión, despliegue de resultados finales y desconexión de los participantes.	Media
RF-20	Comenzar sala	Activación de la cuenta atrás sincronizada en el servidor y apertura de la primera pregunta del test.	Alta
RF-21	Distribución síncrona	Envío simultáneo de la secuencia de enunciados y opciones desde el backend a todos los clientes conectados.	Alta
RF-22	Temporizador servidor	Control centralizado y ejecución estricta del tiempo de respuesta remota en el hilo del backend.	Alta
RF-23	Recepción respuestas	Captura, marca de tiempo y procesamiento de la opción de respuesta seleccionada por el alumno.	Alta
RF-24	Feedback inmediato	Notificación instantánea en el dispositivo del alumno sobre el acierto o fallo sin consolidar la nota permanente.	Alta
RF-25	Registro provisional	Almacenamiento transitorio del resultado del participante bajo el estado de "No Verificado" emisión de un token único.	Alta
RF-26	Generación QR	Renderizado de un código QR dinámico en la pantalla del alumno que encapsula su identidad y su token criptográfico.	Alta
RF-27	Activación cámara	Solicitud de permisos e inicialización del flujo de vídeo de la cámara web en el dispositivo del profesor.	Alta
RF-28	Lectura de QR	Captura óptica, decodificación y extracción de la estructura de datos del alumno desde la interfaz docente.	Alta
RF-29	Validación Token	Comparación criptográfica en el servidor del token escaneado frente al registro transitorio de la sesión activa.	Alta
RF-30	Control de estado	Bloqueo lógico automático de solicitudes de validación de notas si el registro ya consta como "Verificado".	Alta

Tabla 5.6: Requisitos funcionales (2/3)

ID	Nombre	Descripción	Prioridad
RF-31	Check verificación	Actualización de la sesión a estado verificado y consolidación oficial de la nota en el histórico definitivo.	Alta
RF-32	Listado en vivo	Actualización en tiempo real del panel docente reflejando la lista de alumnos validados presencialmente.	Media
RF-33	Estadísticas de sala	Cálculo de los porcentajes de acierto y fallo por enunciado para la detección automatizada de conceptos complejos.	Media
RF-34	Generación por IA	Procesamiento automatizado de textos introducidos por el docente para la sugerencia de borradores de preguntas mediante IA.	Media
RF-35	Asistente de ayuda	Resolución de dudas conceptuales y de uso del sistema mediante una interfaz de lenguaje natural integrada.	Baja
RF-36	Navegación por IA	Sistema de soporte inteligente encargado de detectar intenciones de uso y guiar al profesor en el entorno web.	Baja
RF-37	Revisión post IA	Interfaz de edición específica para la inspección manual, corrección y aprobación de las preguntas sugeridas por la IA.	Media
RF-38	Crear clase	Agrupación e indexación de conjuntos estables de alumnos bajo etiquetas de grupo o clase académica.	Baja
RF-39	Exportación resultados	Generación y descarga local de las calificaciones oficiales y verificadas en formatos estandarizados PDF o CSV.	Alta
RF-40	Gestión de conexiones	Manejo de admisiones tardías, retención de estados por caída de red y flujos de reconexión adaptativa.	Media
RF-41	Filtros de búsqueda	Herramientas de filtrado avanzado en el catálogo de cuestionarios (activos, inactivos, recientes o etiquetas).	Media
RF-42	Bonificación por tiempo	Algoritmo de cálculo de puntuación adicional ponderada según la rapidez de respuesta del participante.	Baja
RF-43	Sistema de rachas	Concesión de multiplicadores de puntuación por el encadenamiento consecutivo de respuestas correctas.	Baja
RF-44	Modo de puntuación	Selección de las reglas de asignación de puntos (estándar o adaptativa) en los parámetros iniciales de la sala.	Baja
RF-45	Visibilidad de ranking	Opción para conmutar la presencia de la pantalla intermedia de clasificación global entre preguntas.	Baja
RF-46	Formulario avanzado	Inclusión de imágenes de soporte y gestión de etiquetas avanzadas (tags) en la edición del cuestionario, con hasta 8 opciones.	Baja
RF-47	Historial de salas	Acceso permanente al registro histórico de sesiones cerradas que hayan completado la fase de verificación presencial.	Alta

Tabla 5.7: Requisitos funcionales (3/3)

5.4.1. Criterios de aceptación y definición de hecho

Para garantizar que los requisitos funcionales especificados alcancen los estándares de calidad, seguridad y robustez técnica exigidos, el proceso de desarrollo se rige por una *Definition of Done* (DoD) o Definición de Hecho. Esta definición actúa como una *checklist* obligatoria y transversal que cualquier funcionalidad debe satisfacer antes de ser dada por finalizada y consolidada en el sistema.

De manera general, un requisito funcional se considerará formalmente aceptado cuando cumpla con las siguientes dimensiones de verificación:

■ Comportamiento funcional:

- El flujo principal de la funcionalidad se ejecuta correctamente en la interfaz del usuario sin producir bloqueos, errores ni excepciones.
- El sistema gestiona los flujos alternativos y de error (entradas inválidas, caídas de red o datos duplicados) devolviendo un feedback informativo e inmediato al usuario.
- Se respetan los límites físicos, lógicos y de formato estructurados en las restricciones del sistema.

■ Calidad técnica y seguridad:

- El código fuente cumple con las directrices de seguridad relativas a la persistencia de datos y el cifrado de credenciales.
- Las comunicaciones síncronas o en tiempo real emplean de forma estricta los canales bidireccionales y protocolos establecidos en la arquitectura del sistema.
- Los datos asociados se mapean de forma exacta en las estructuras definidas en los requisitos de información.

■ Verificación y pruebas:

- La funcionalidad tiene asociado, al menos, un caso de prueba unitario, de integración o de sistema dentro del plan de verificación.
- El comportamiento real del software coincide con el esperado tras superar con éxito las pruebas de caja negra, de carga o de seguridad.

■ Documentación y gestión del ciclo de vida:

- Se ha redactado la documentación técnica correspondiente en los módulos afectados del sistema, incluyendo comentarios en el código según las buenas prácticas de desarrollo.
- El *issue* asignado en la plataforma de control de versiones (GitHub) se ha actualizado con la solución implementada, asociando los *commits* pertinentes y procediendo a su cierre formal tras la aprobación del despliegue.

Estos criterios globales de aceptación aseguran que el paso del análisis de requisitos a la fase de pruebas finales sea completamente trazable, transparente y verificable, sirviendo de base para la posterior validación del sistema en la sección [8.3 Casos de prueba y validación](#).

5.5. Casos de uso

Los diagramas y especificaciones de casos de uso sirven para modelar el comportamiento del sistema desde el punto de vista de los actores involucrados, transformando los requisitos funcionales abstractos en interacciones lógicas concretas. Con el propósito de mantener la claridad de la memoria y evitar redundancias estructurales, las funcionalidades de la plataforma *Quizzie* se estructuran en paquetes funcionales según el área operativa donde actúan los usuarios principales (*Profesor* y *Alumno*).

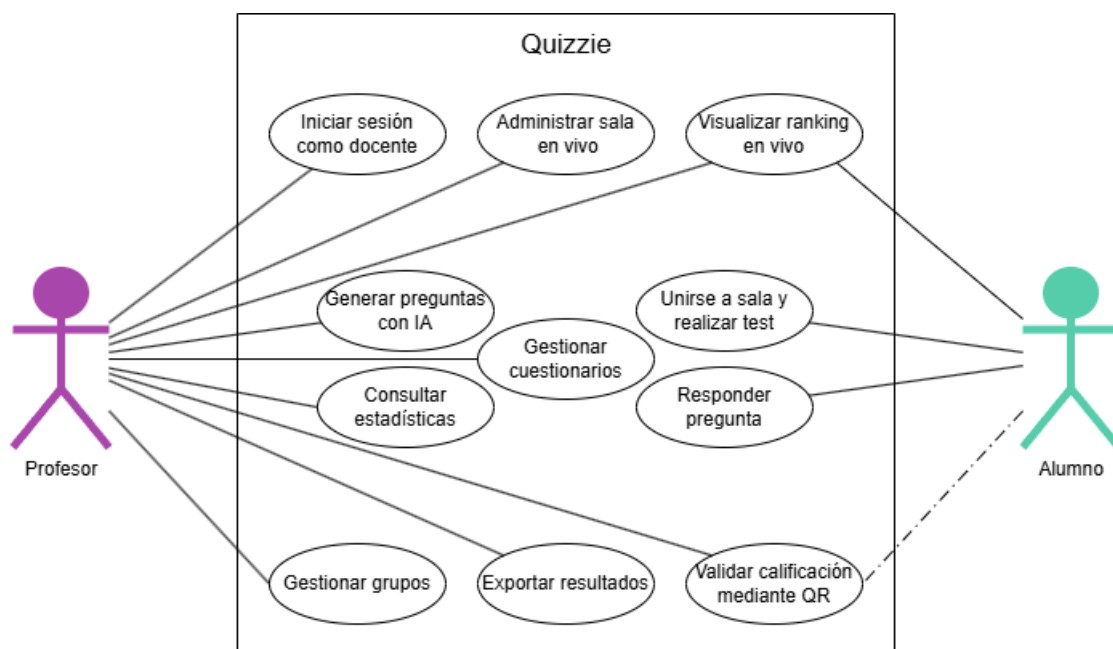


Figura 5.1: Diagrama general de casos de uso del sistema.

El conjunto de los casos de uso que componen el sistema es el siguiente:

- **CU-01: Iniciar sesión como docente.** Permite al profesor acceder a su panel privado, registrarse o cerrar sesión de forma segura.
- **CU-02: Gestionar cuestionarios.** El profesor puede crear, editar, listar y eliminar sus bancos de preguntas.
- **CU-03: Generar preguntas con IA.** Extensión del CU-02 donde el profesor solicita a la IA la creación de borradores de preguntas basados en un tema.

- **CU-04: Administrar sala en vivo.** El profesor instancia un cuestionario, genera un PIN de acceso y controla el flujo de la partida (inicio, paso de preguntas y cierre).
- **CU-05: Unirse a sala y realizar test.** El alumno accede mediante PIN, responde a las preguntas sincronizadas y recibe feedback inmediato sobre su rendimiento.
- **CU-06: Visualizar ranking en vivo.** Los alumnos y el profesor visualizan la progresión de los líderes de la sala tras cada pregunta.
- **CU-07: Validar calificación mediante QR.** El profesor utiliza su dispositivo para escanear el código QR del alumno, verificando su identidad y presencia física para oficializar la calificación.
- **CU-08: Consultar estadísticas.** El profesor analiza los puntos críticos y el rendimiento general de la clase tras finalizar la sesión.
- **CU-09: Exportar resultados.** El profesor descarga los datos de los alumnos verificados en formato CSV para su uso en plataformas externas.
- **CU-10: Responder pregunta.** El alumno puede responder durante el tiempo determinado a las preguntas del test obteniendo la puntuación pertinente.
- **CU-11: Gestionar grupos.** El profesor organiza el alumnado creando etiquetas de grupo para asociar y evaluar la evolución histórica de un mismo conjunto de estudiantes.

5.5.1. Especificación detallada de Casos de Uso

A continuación, se detallan las plantillas descriptivas de cada caso de uso del sistema, especificando sus actores, precondiciones, secuencias de pasos lógicos y la gestión de escenarios alternativos de error:

Caso de Uso	CU-01: Iniciar sesión como docente
Actor principal	Profesor.
Precondición	El profesor debe estar registrado en la plataforma.
Flujo principal	1. El profesor introduce su correo electrónico y contraseña en el formulario de login. 2. El sistema cifra la contraseña introducida y la compara con el hash almacenado. 3. El sistema genera un token de sesión (JWT) y redirige al profesor a su panel principal.
Flujo alternativo	Si las credenciales son incorrectas, el sistema muestra un mensaje de error y permite reintentar el acceso.

Tabla 5.8: Especificación del caso de uso CU-01.

Caso de Uso	CU-02: Gestionar cuestionarios
Actor principal	Profesor.
Precondición	El profesor ha iniciado sesión correctamente.
Flujo principal	1. El profesor accede a su biblioteca de contenidos. 2. Selecciona la opción de crear un nuevo cuestionario o editar uno existente. 3. El profesor define el título, descripción y añade preguntas con sus respectivas opciones. 4. El profesor marca la opción correcta para cada pregunta y asigna una puntuación. 5. El sistema guarda los cambios en la base de datos permanente.
Postcondición	El cuestionario queda disponible para ser utilizado en una sala.

Tabla 5.9: Especificación del caso de uso CU-02.

Caso de Uso	CU-03: Generar preguntas con IA
Actores	Principal: Profesor. Secundario: Servicio de IA (API).
Precondición	El profesor se encuentra dentro del editor de cuestionarios.
Flujo principal	1. El profesor activa el asistente de IA e introduce un tema o pega un texto base. 2. El sistema envía la información a la API de IA externa. 3. El sistema recibe y presenta al profesor un listado de preguntas y respuestas sugeridas. 4. El profesor selecciona las preguntas que desea importar y las edita si es necesario. 5. El sistema integra las preguntas seleccionadas en el cuestionario actual.

Tabla 5.10: Especificación del caso de uso CU-03.

Caso de Uso	CU-04: Administrar sala en vivo
Actor principal	Profesor.
Precondición	El profesor está autenticado y ha seleccionado un cuestionario de su biblioteca.
Flujo principal	1. El profesor solicita la creación de una nueva sala de juego. 2. El sistema genera un PIN único de 6 dígitos y abre el socket de comunicación. 3. El profesor visualiza en tiempo real la entrada de alumnos en la sala de espera. 4. El profesor inicia el test, enviando la señal de comienzo a todos los alumnos. 5. El profesor controla el paso de las preguntas o permite el avance automático. 6. El profesor finaliza la sesión manualmente o al terminar las preguntas.

Tabla 5.11: Especificación del caso de uso CU-04.

Caso de Uso	CU-05: Unirse a sala y realizar test
Actor principal	Alumno.
Precondición	Existe una sala activa y el alumno posee el código PIN.
Flujo principal	1. El alumno introduce el PIN y un Nickname. 2. El sistema valida el acceso y posiciona al alumno en la sala de espera. 3. Al comenzar el test, el alumno recibe las preguntas de forma sincronizada con el resto de la clase. 4. El alumno selecciona la opción deseada dentro del tiempo límite. 5. El sistema registra la respuesta y el tiempo empleado. 6. Al finalizar el cuestionario, el alumno recibe un código QR único de validación.
Flujo alternativo	Si el alumno sufre una desconexión, puede reingresar con el mismo PIN y Nickname; el sistema restaurará su sesión y lo situará en la pregunta activa.

Tabla 5.12: Especificación del caso de uso CU-05.

Caso de Uso	CU-06: Visualizar ranking en vivo
Actores	Principal: Profesor / Alumno.
Precondición	Se ha completado al menos una pregunta en una sala activa.
Flujo principal	1. El sistema calcula la puntuación de cada alumno basándose en aciertos y velocidad de respuesta. 2. El sistema actualiza el ranking global de la sala. 3. El profesor visualiza el "Top 5.^{en} la pantalla principal para motivar a la clase. 4. El alumno visualiza su posición relativa y sus puntos en su propio dispositivo.

Tabla 5.13: Especificación del caso de uso CU-06.

Caso de Uso	CU-07: Validar calificación mediante QR
Actores	Principal: Profesor. Secundario: Alumno.
Precondición	La sala ha finalizado y el alumno muestra el QR generado por el sistema.
Flujo principal	1. El profesor selecciona la opción de "Validar Alumno.^{en} su panel de sala. 2. El profesor escanea el código QR del dispositivo del alumno. 3. El sistema extrae el Token cifrado del QR y lo valida contra el registro provisional en el backend. 4. Tras la validación positiva, el sistema cambia el estado del alumno a "Verificado". 5. La nota se hace persistente y se vincula oficialmente al registro del alumno.

Tabla 5.14: Especificación del caso de uso CU-07.

Caso de Uso	CU-08: Consultar estadísticas
Actor principal	Profesor.
Precondición	Existen salas finalizadas con respuestas registradas.
Flujo principal	1. El profesor accede al histórico de salas desde su panel. 2. Selecciona una sesión específica para analizar. 3. El sistema genera gráficas comparativas sobre el porcentaje de aciertos por pregunta. 4. El profesor identifica las áreas de conocimiento donde los alumnos han tenido más dificultades.

Tabla 5.15: Especificación del caso de uso CU-08.

Caso de Uso	CU-09: Exportar resultados
Actor principal	Profesor.
Precondición	La sala ha finalizado y existen alumnos con estado "Verificado".
Flujo principal	1. El profesor solicita la exportación de resultados desde el informe de la sala. 2. El sistema filtra los registros para incluir únicamente a los alumnos validados presencialmente. 3. El sistema genera un archivo en formato CSV o Excel. 4. El archivo se descarga automáticamente en el dispositivo del profesor.

Tabla 5.16: Especificación del caso de uso CU-09.

Caso de Uso	CU-10: Responder pregunta
Actor principal	Alumno.
Precondición	El alumno está dentro de una sala activa y el profesor ha lanzado una pregunta.
Flujo principal	1. El sistema muestra el enunciado y las opciones de respuesta en el dispositivo del alumno. 2. El sistema inicia el cronómetro de cuenta atrás configurado para esa pregunta. 3. El alumno selecciona una de las opciones antes de que el tiempo se agote. 4. El sistema captura la opción elegida y registra el milisegundo exacto de la respuesta (timestamp). 5. Al agotarse el tiempo o responder todos los alumnos, el sistema cierra la recepción de datos. 6. El sistema compara la respuesta con la opción correcta definida en el cuestionario. 7. El sistema calcula los puntos obtenidos basándose en el acierto y la bonificación por rapidez.
Flujo alternativo	Si el alumno no selecciona ninguna opción antes de que el cronómetro llegue a cero, el sistema registra la pregunta como "No contestada" con 0 puntos.
Postcondición	El resultado de la pregunta se almacena temporalmente para actualizar el ranking en vivo.

Tabla 5.17: Especificación del caso de uso CU-10.

Caso de Uso	CU-11: Gestionar grupos
Actor principal	Profesor.
Precondición	El profesor ha iniciado sesión correctamente en el sistema.
Flujo principal	1. El profesor accede a la sección de gestión de grupos desde el panel de control. 2. Selecciona la opción de crear una nueva clase e introduce un nombre descriptivo o etiqueta de grupo. 3. El profesor asocia la lista de identificadores de alumnos válidos correspondientes a ese conjunto académico. 4. El sistema almacena la estructura del grupo vinculándola directamente al perfil del docente responsable. 5. El profesor consulta los reportes específicos para comprobar el progreso y la evolución histórica de las calificaciones de dicha clase.

Tabla 5.18: Especificación del caso de uso CU-11.

5.6. Requisitos no funcionales

Los requisitos no funcionales determinan los atributos de calidad, restricciones técnicas y criterios de rendimiento que debe satisfacer la plataforma *Quizzie*. Su categorización se articula atendiendo a las dimensiones del modelo de calidad del producto de software definido en la norma ISO/IEC 25010 [36]:

Dimensión	ID	Definición del requisito
Rendimiento	RNF-01	Sincronización: Los eventos en tiempo real deben procesarse de forma óptima en el backend para garantizar la coherencia síncrona en el aula.
	RNF-02	Concurrencia: Soporte mínimo de 100 alumnos interactuando de forma simultánea dentro de una misma sala sin degradación de los servicios básicos.
Seguridad	RNF-03	Integridad: El token de validación presencial embebido en el QR del alumno debe ser criptográficamente seguro, firmado por el servidor y no falsificable.
	RNF-04	Privacidad: Almacenamiento e inserción obligatoria de credenciales de acceso de profesores mediante algoritmos de <i>hashing</i> seguro y unidireccional.
	RNF-05	Cifrado: Uso estricto de canales de red protegidos bajo protocolos de comunicación seguros HTTPS y <i>WebSockets</i> seguros (WSS).
Usabilidad	RNF-06	Adaptabilidad: Interfaz de usuario adaptativa (<i>Responsive Design</i>) optimizada para una navegación fluida tanto en dispositivos móviles como en ordenadores.
	RNF-07	Eficiencia QR: Optimización del flujo óptico para que el tiempo de lectura, decodificación y validación de la firma del código QR por la cámara sea inmediato.
Fiabilidad	RNF-08	Reconexión: Capacidad de retener y recuperar el estado lógico y la puntuación transitoria de un participante ante caídas o pérdidas accidentales de conectividad.
Disponibilidad	RNF-09	Persistencia: Mecanismo de salvaguarda y respaldo de los resultados provisionales en el backend ante interrupciones críticas o fallos imprevistos del servidor.
Mantenibilidad	RNF-10	Modularidad: Separación clara de responsabilidades entre el servidor de base de datos, el backend y el cliente frontend para facilitar futuras actualizaciones.

Tabla 5.19: Requisitos no funcionales clasificados por dimensión.

5.7. Trazabilidad de requisitos

Para garantizar la integridad del desarrollo y demostrar el cumplimiento de las metas del proyecto, este apartado presenta la trazabilidad bidireccional del sistema. Este proceso permite certificar que cada decisión responde directamente a un fin práctico de la aplicación, evitando la existencia de elementos redundantes o injustificados. Las tres matrices que se muestran a continuación relacionan objetivos funcionales con requisitos de información, requisitos no funcionales y requisitos funcionales, respectivamente.

	OBJ-01	OBJ-02	OBJ-03	OBJ-04	OBJ-05	OBJ-06	OBJ-07	OBJ-08	OBJ-09	OBJ-10	OBJ-11	OBJ-12
RI-01	X											
RI-02		X							X			
RI-03			X	X	X							
RI-04					X	X	X	X				
RI-05							X	X				X
RI-06											X	X

Tabla 5.20: Matriz de trazabilidad RI-OBJ.

	OBJ-01	OBJ-02	OBJ-03	OBJ-04	OBJ-05	OBJ-06	OBJ-07	OBJ-08	OBJ-09	OBJ-10	OBJ-11	OBJ-12
RNF-01					X							
RNF-02					X							
RNF-03						X	X					
RNF-04	X											
RNF-05	X				X							
RNF-06	X	X		X	X							
RNF-07										X		
RNF-08					X							
RNF-09			X		X		X					
RNF-10	X	X	X	X	X	X	X	X	X	X	X	X

Tabla 5.21: Matriz de trazabilidad RNF-OBJ.

	OBJ-01	OBJ-02	OBJ-03	OBJ-04	OBJ-05	OBJ-06	OBJ-07	OBJ-08	OBJ-09	OBJ-10	OBJ-11	OBJ-12
RF-01	X											
RF-02	X											
RF-03	X											
RF-04	X											
RF-05	X											
RF-06	X											
RF-07		X										
RF-08		X										
RF-09								X				
RF-10		X										
RF-11			X									
RF-12												X
RF-13		X										
RF-14		X										
RF-15			X									
RF-16				X								
RF-17				X								
RF-18			X									
RF-19			X									
RF-20					X							
RF-21					X							
RF-22			X									
RF-23					X							
RF-24					X							
RF-25							X					
RF-26						X						
RF-27										X		
RF-28										X		
RF-29						X						
RF-30							X					
RF-31							X					
RF-32								X				
RF-33								X				
RF-34									X			
RF-35									X			
RF-36									X			
RF-37									X			
RF-38											X	
RF-39												X
RF-40					X							
RF-41		X										
RF-42					X							
RF-43					X							
RF-44			X									
RF-45			X									
RF-46		X										
RF-47							X					

Tabla 5.22: Matriz de trazabilidad RF-OBJ.

Por último, con el fin de unificar todo el ciclo de vida del desarrollo, se presenta la siguiente matriz de trazabilidad unificada. Este mapa conecta de manera jerárquica los módulos del sistema, los objetivos funcionales que persiguen, los casos de uso técnicos, y cómo estos se desglosan en las historias de usuario, compuestas por sus respectivos requisitos funcionales.

Módulo	OBJ	CU	HU	RF
Contenido	OBJ-02	CU-02	HU-PR-01	RF-07, RF-08, RF-10, RF-12
			HU-PR-02	RF-13, RF-14, RF-41
			HU-PR-13	RF-46
	OBJ-09	CU-03	HU-PR-09	RF-34, RF-35, RF-36, RF-37
Salas	OBJ-03	CU-04	HU-PR-03	RF-11, RF-15, RF-18, RF-22
			HU-PR-05	RF-19
			HU-PR-11	RF-42, RF-43, RF-44
			HU-PR-12	RF-45
	OBJ-04	CU-05	HU-AL-01	RF-16, RF-17, RF-40
	OBJ-05	CU-05	HU-AL-01	RF-16, RF-17, RF-40
		CU-10	HU-AL-02	RF-20, RF-21
	OBJ-06	CU-07	HU-AL-03	RF-23, RF-24, RF-42, RF-43
			HU-PR-06	RF-27, RF-28, RF-29, RF-30, RF-31
	OBJ-07	CU-07	HU-AL-04	RF-25, RF-26
			HU-PR-06	RF-27, RF-28, RF-29, RF-30, RF-31
	OBJ-08	CU-06	HU-PR-04	RF-32
			HU-PR-12	RF-09
		CU-08	HU-PR-08	RF-33
			HU-PR-14	RF-47
	OBJ-10	CU-07	HU-PR-06	RF-27, RF-28, RF-29, RF-30, RF-31
			HU-AL-04	RF-25, RF-26
	OBJ-12	CU-09	HU-PR-10	RF-39
Usuarios	OBJ-01	CU-01	HU-PR-01	RF-01, RF-02, RF-03, RF-04, RF-05, RF-06
	OBJ-11	CU-11	HU-PR-07	RF-38

Tabla 5.23: Matriz de trazabilidad unificada del sistema.

6. Diseño y arquitectura

En este capítulo se describe la estructura técnica y conceptual sobre la que se asienta el sistema, detallando las decisiones arquitectónicas que garantizan su correcto funcionamiento, mantenibilidad y rendimiento. El diseño propuesto responde tanto a los requisitos funcionales del proyecto como a sus restricciones operativas: la ejecución de cuestionarios en tiempo real, la baja latencia en la transmisión de eventos a múltiples participantes simultáneos y una rigurosa separación de responsabilidades entre el cliente y el servidor.

6.1. Patrones arquitectónicos elegidos

Antes de seleccionar el *stack* y la estructura del código, se establecieron una serie de principios de diseño orientados a garantizar la integridad de las calificaciones, la seguridad del sistema y su capacidad de escalado:

- **Validación centralizada y control de roles (*anti-cheat*):** El cliente no toma decisiones sobre la corrección de respuestas, cálculo de puntuaciones ni temporización oficial. Existe una estricta división de responsabilidades en la que las acciones administrativas (como el control del avance de preguntas o el cierre de salas) quedan reservadas exclusivamente al profesor. El alumno se limita únicamente a unirse a la sala y enviar sus respuestas.
- **Desacoplamiento de la presentación y los datos:** El servidor se limita a enviar los datos y eventos de la partida, dejando que la aplicación del usuario se ocupe exclusivamente de cómo mostrar la información en pantalla, gestionar las animaciones y adaptarse a cada dispositivo.
- **Autenticación sin estado (*stateless*):** La validación de identidades mediante tokens integrados en las cabeceras de cada solicitud evita el almacenamiento de estado de sesión en la memoria del servidor, facilitando la distribución de la carga.
- **Gestión de la concurrencia y escalabilidad:** El sistema debe ser capaz de mantener decenas de conexiones bidireccionales simultáneas en un aula sin bloquear el procesamiento del servidor, distribuyendo el rendimiento de forma eficiente entre peticiones HTTP habituales y eventos en tiempo real.

Para dar respuesta a estos principios, se ha optado por una arquitectura híbrida que combina tres patrones complementarios: el modelo **cliente-servidor desacoplado**, formulado por Bass *et al.* [37] para la separación de responsabilidades; la **arquitectura en capas** (*layered architecture*), formalizada por Martin [4] para el aislamiento de las reglas de negocio; y la **arquitectura basada en eventos** (*event-driven architecture*), fundamentada en los patrones de comunicación asíncrona de Hohpe y Woolf [38].

6.1.1. Arquitectura cliente-servidor desacoplada

La aplicación adopta un esquema cliente-servidor desacoplado con una rigurosa delimitación de responsabilidades entre el cliente y el servidor:

- **Cliente (*frontend* - SPA):** Desarrollado como una (*SPA (single page application)*), se encarga de la representación visual, la experiencia de usuario y el control de temporizadores locales de apoyo. Opera sin acceso directo a la persistencia ni a las reglas de negocio.
- **Servidor (*backend* - API REST + WebSockets):** Actúa como la única fuente de verdad (*single source of truth*), gestionando la autenticación, la seguridad criptográfica, el ciclo de vida de los cuestionarios, la persistencia en base de datos y la orquestación síncrona de las salas.
- **Canales de comunicación:**
 - *HTTP/HTTPS (API REST):* Empleado para operaciones síncronas sin estado, tales como el inicio de sesión, la gestión de cuestionarios y la consulta de informes históricos.
 - *WebSockets:* Empleado para mantener un canal de comunicación persistente, bidireccional y de baja latencia durante la ejecución en vivo de los cuestionarios.

6.1.2. Arquitectura en capas

Tanto en el servidor como en el cliente, el sistema se estructura internamente en niveles independientes para aislar responsabilidades (*separation of concerns*).

En el *backend*, la lógica se organiza en cuatro capas conceptuales:

1. **Capa de presentación y controladores:** Expone los puntos de entrada HTTP (endpoints REST) y los canales *WebSocket*. Valida las peticiones de entrada, gestiona las respuestas de la API y coordina la autenticación.
2. **Capa de lógica de negocio y servicios:** Implementa las reglas del dominio: suma de puntuaciones por respuesta, transiciones de estado de las salas, opciones de cuestionario en vivo y lógica de verificación.
3. **Capa de acceso a datos y modelado (ORM):** Mapea las entidades de negocio del dominio con el modelo relacional, garantizando las restricciones de integridad y las relaciones entre entidades.
4. **Capa de persistencia:** Administra la conexión con el motor de base de datos relacional y gestiona las transacciones de lectura y escritura.

En el *frontend*, la arquitectura reproduce este aislamiento separando la **capa de vistas** (componentes de interfaz), la **capa de estado y contexto** (sesión de usuario y datos de partida), la **capa de servicios de red** (módulos REST y sockets) y la **capa de contratos de datos** (definición de tipos e interfaces).

6.1.3. Arquitectura basada en eventos

Para las sesiones en tiempo real, el sistema adopta una arquitectura basada en eventos que mantiene sincronizados a todos los participantes conectados a una sala.

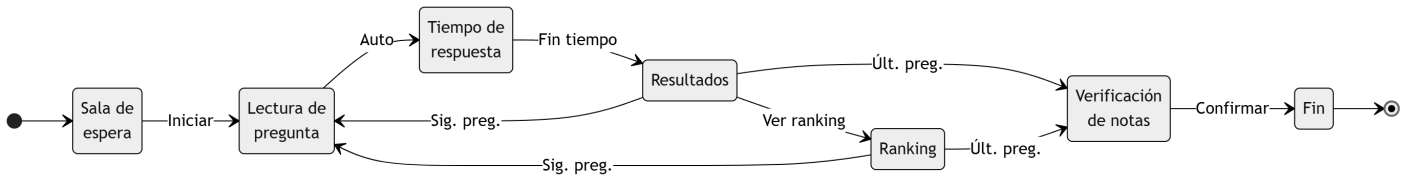


Figura 6.1: Estados de la sala de juego en tiempo real.

El diseño de este comportamiento se apoya en los siguientes componentes conceptuales:

- **Gestor centralizado de conexiones:** Mantiene el registro de las sesiones activas en memoria, administrando el ciclo de vida de los sockets y aplicando el patrón de difusión (*broadcasting*) para emitir instrucciones a todos los participantes de una sala de forma simultánea.
- **Bucle de sincronización temporal:** Proceso asíncrono en segundo plano que supervisa el transcurso del tiempo de respuesta en las salas activas, notifica el tiempo restante periódicamente y desencadena automáticamente la transición de fase al expirar el tiempo.
- **Ciclo de eventos y transmisión:** Los cambios de estado de la sala (fase de lectura, apertura de respuestas, muestra de resultados, actualización de temporizadores) se propagan como eventos estructurados JSON, permitiendo que la interfaz del cliente reaccione de forma inmediata.

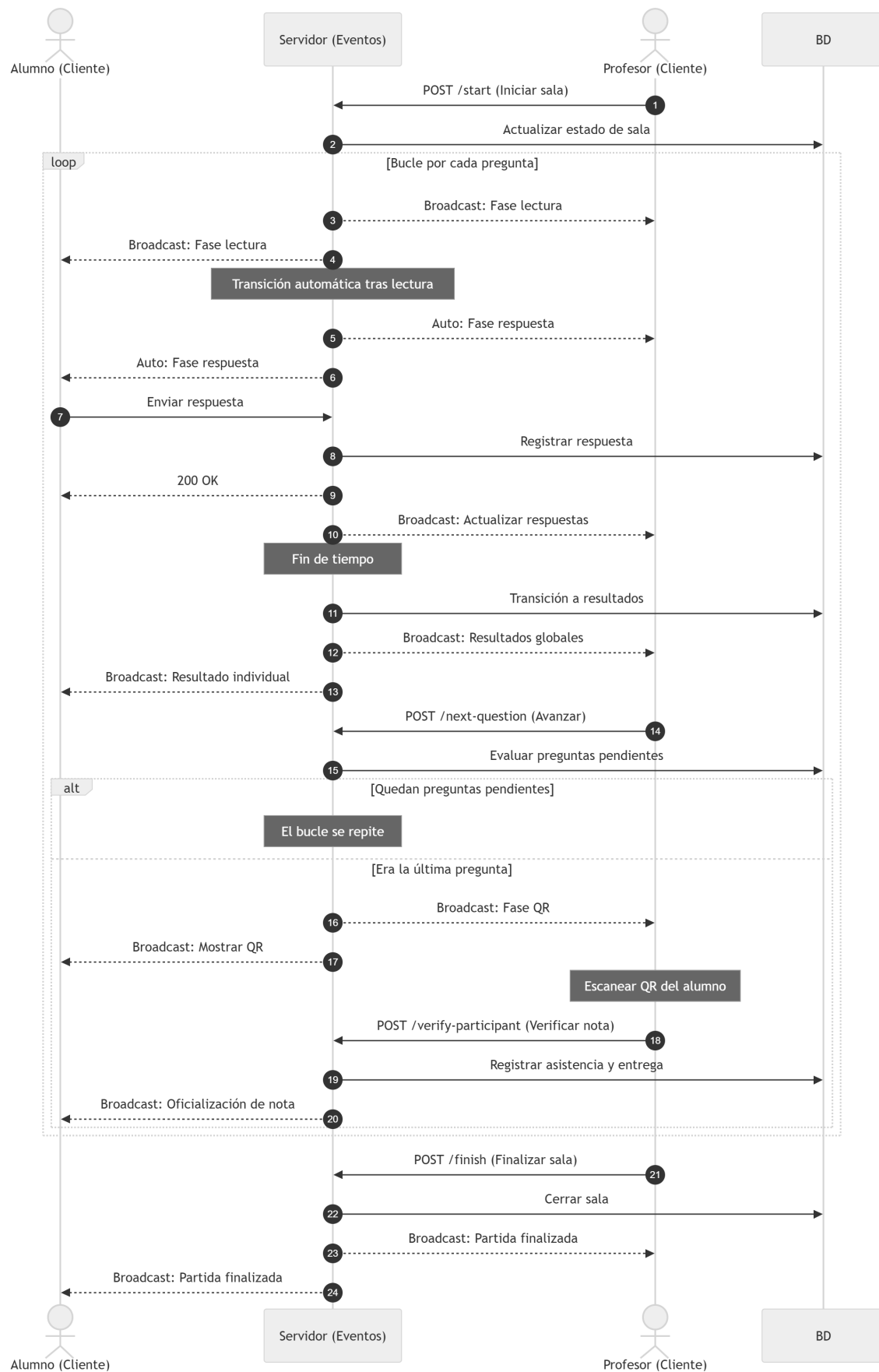


Figura 6.2: Diagrama de secuencia de la transmisión de eventos en una sala.

6.2. Arquitectura global del sistema

La combinación de los tres patrones previamente descritos se consolida en una estructura global unificada. En la [comparativa de la arquitectura](#) se presenta, a la izquierda, el marco conceptual abstracto de la arquitectura global y, a la derecha, su correspondencia directa con los componentes de software, módulos y tecnologías reales que ejecutan el sistema.

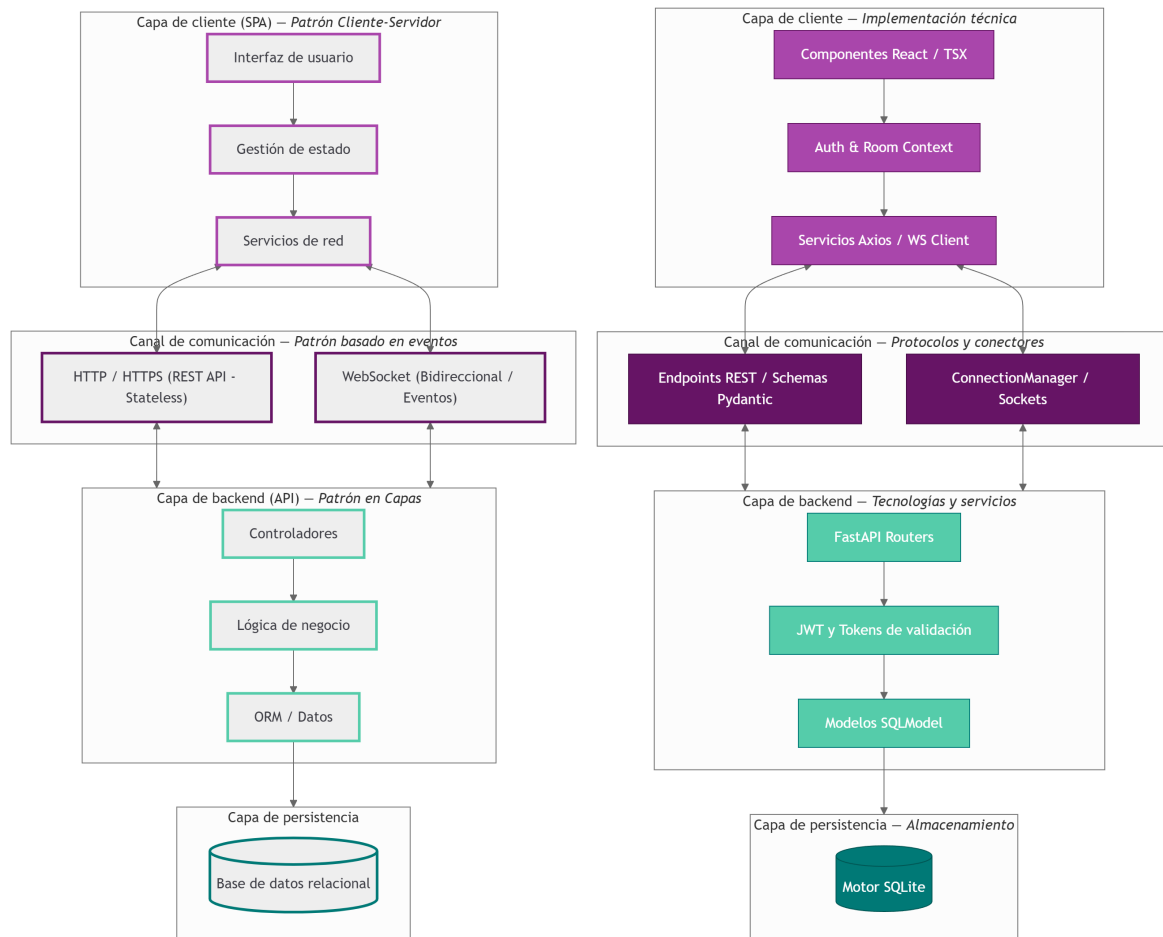


Figura 6.3: Comparativa entre la arquitectura global y su implementación concreta.

Como se observa en la comparativa, cada capa conceptual se corresponde de forma directa con sus módulos de software equivalentes: la interfaz y el estado en el cliente React, los controladores y gestores de eventos en FastAPI, y el mapeo de datos mediante SQLAlchemy sobre el motor SQLite en la persistencia.

6.3. Justificación y decisiones de arquitectura

Una vez definidos el esquema conceptual y la estructura de componentes del sistema, resulta imprescindible formalizar las decisiones tecnológicas adoptadas y verificar su alineación con las necesidades del proyecto. En este apartado se aborda dicho proceso desde dos perspectivas complementarias: la documentación formal de las decisiones de diseño clave y la verificación explícita de su impacto sobre los atributos de calidad del proyecto.

6.3.1. Registros de decisiones de arquitectura (ADR)

Para documentar y formalizar las decisiones de diseño estructural más críticas del proyecto, se ha adoptado el patrón de registros de decisiones de arquitectura (*Architecture Decision Records*, ADR) propuesto por Nygard [39]. A continuación se detallan las decisiones arquitectónicas fundamentales que vertebran el diseño de Quizzie:

ADR	ADR-01: Adopción de FastAPI y WebSockets para la comunicación en tiempo real
Estado	Aprobado.
Contexto	La plataforma requiere gestionar la comunicación bidireccional y síncrona con hasta 100 alumnos por sala, ofreciendo baja latencia en la transmisión de preguntas y recepción de respuestas.
Alternativas	API REST mediante <i>polling</i> tradicional y <i>Server-Sent Events (SSE)</i> .
Decisión	Utilizar FastAPI como motor asíncrono con protocolo <i>WebSockets</i> bidireccional nativo.
Consecuencias	Positivas: Se logra una latencia inferior a 50 ms en la propagación de eventos y un consumo mínimo de RAM por conexión activa. Negativas: Exige gestionar manualmente la reconexión y retención de estado de sockets en el cliente.

Tabla 6.1: Registro de decisiones de arquitectura - ADR-01.

ADR	ADR-02: Arquitectura basada en eventos para el manejo de salas
Estado	Aprobado.
Contexto	El ciclo de vida de una sala exige informar inmediatamente a todos los clientes en tiempo real, ya que todos deben estar en la misma fase en todo momento.
Alternativas	Bucle de consulta síncrono <i>polling</i> desde el cliente o <i>sockets</i> puros sin tipado de eventos.
Decisión	Diseñar un gestor centralizado de eventos (ConnectionManager) que difunda mensajes estructurados JSON mediante un patrón de publicación/suscripción local.
Consecuencias	Positivas: Desacoplamiento total entre la lógica del juego y la capa de transporte; facilidad para testear eventos aislados.
	Negativas: Mayor complejidad en la serialización y deserialización de esquemas Pydantic.

Tabla 6.2: Registro de decisiones de arquitectura - ADR-02.

ADR	ADR-03: Selección del stack de persistencia SQLite + SQLAlchemy
Estado	Aprobado.
Contexto	Se requiere un motor relacional liviano, de coste cero y portabilidad total, minimizando la carga operativa en el servidor sin duplicar esquemas de validación de datos.
Alternativas	PostgreSQL administrado en la nube o SQLAlchemy + Pydantic desacoplados.
Decisión	Adoptar SQLite como motor relacional en contenedor combinado con SQLAlchemy como ORM unificado.
Consecuencias	Positivas: Cumplimiento estricto del principio <i>DRY</i> (<i>Don't Repeat Yourself</i>); despliegue sin dependencias complejas de infraestructura.
	Negativas: Bloqueo a nivel de archivo en escrituras masivas concurrentes.

Tabla 6.3: Registro de decisiones de arquitectura - ADR-03.

ADR	ADR-04: Autenticación híbrida JWT + Código QR presencial
Estado	Aprobado.
Contexto	Es necesario conciliar el acceso seguro del profesor (persistente) con la incorporación rápida del alumno (efímera y sin registro), garantizando que la nota solo se consolide si el estudiante está presente en el aula.
Alternativas	Cuentas obligatorias de usuario para alumnos o comprobación manual en lista por parte del docente.
Decisión	Implementar autenticación síncrona JWT para docentes y un flujo de tokens efímeros firmados criptográficamente presentados en formato código QR por el alumno al finalizar la prueba.
Consecuencias	Positivas: Eliminación del fraude por participación a distancia; fricción nula para el estudiante. Negativas: Requiere el uso de la cámara del dispositivo docente durante la fase de cierre de la sesión.

Tabla 6.4: Registro de decisiones de arquitectura - ADR-04.

6.3.2. Trazabilidad entre RNF y tácticas arquitectónicas

Para garantizar que el diseño propuesto satisface los atributos de calidad definidos en la especificación de requisitos, la siguiente tabla mapea cada requisito no funcional (RNF) con la táctica de arquitectura aplicada, el componente responsable de su ejecución y el método de verificación utilizado en el proyecto:

Requisito	Táctica arquitectónica	Responsable	Verificación
RNF-01	Canales bidireccionales y difusión asíncrona (<i>broadcasting</i>).	Controladores (routers/)	Test de latencia (< 50 ms)
RNF-02	Bucle de eventos no bloqueante e E/S asíncrona.	Motor FastAPI y Uvicorn	Carga simultánea (100 usuarios)
RNF-03	Tokens criptográficos firmados con HMAC de un solo uso.	routers/ y auth.py	Test de unicidad de firmas
RNF-04	Derivación y <i>hashing</i> adaptativo de credenciales.	auth.py	Inspección de almacenamiento
RNF-05	Forzado de canal seguro extremo a extremo (HTTPS / WSS).	Proxy inverso (TLS)	Auditoría de certificados TLS
RNF-06	Diseños fluidos reactivos con puntos de ruptura adaptativos.	Componentes React UI y layouts/	Verificación en multidispositivo
RNF-07	Escaneo optimizado del flujo de cámara en hilo secundario.	Módulo de partida (room-play/)	Medición de tiempo (< 1 s)
RNF-08	Persistencia de sesión local y reconexión mediante token/PIN.	Estado global (context/)	Simulación de caída de red
RNF-09	Transacciones relacionales ACID con sincronización en disco.	Motor SQLite y SQLAlchemy	Prueba de fallo de proceso
RNF-10	Separación estricta de responsabilidades en capas independientes.	Arquitectura global	Inspección de dependencias

Tabla 6.5: Matriz de trazabilidad entre RNF y tácticas arquitectónicas.

6.4. Estructura modular del código

Para trasladar los patrones arquitectónicos descritos a la fase de diseño, el sistema se organiza conceptualmente en torno a una estructura monorepo dividida en dos grandes módulos independientes (backend/ y frontend/). Esta organización delimita claramente las responsabilidades y facilita el desarrollo desacoplado de ambos componentes.

A continuación se presenta el esquema preliminar de la jerarquía de directorios y componentes principales del proyecto:

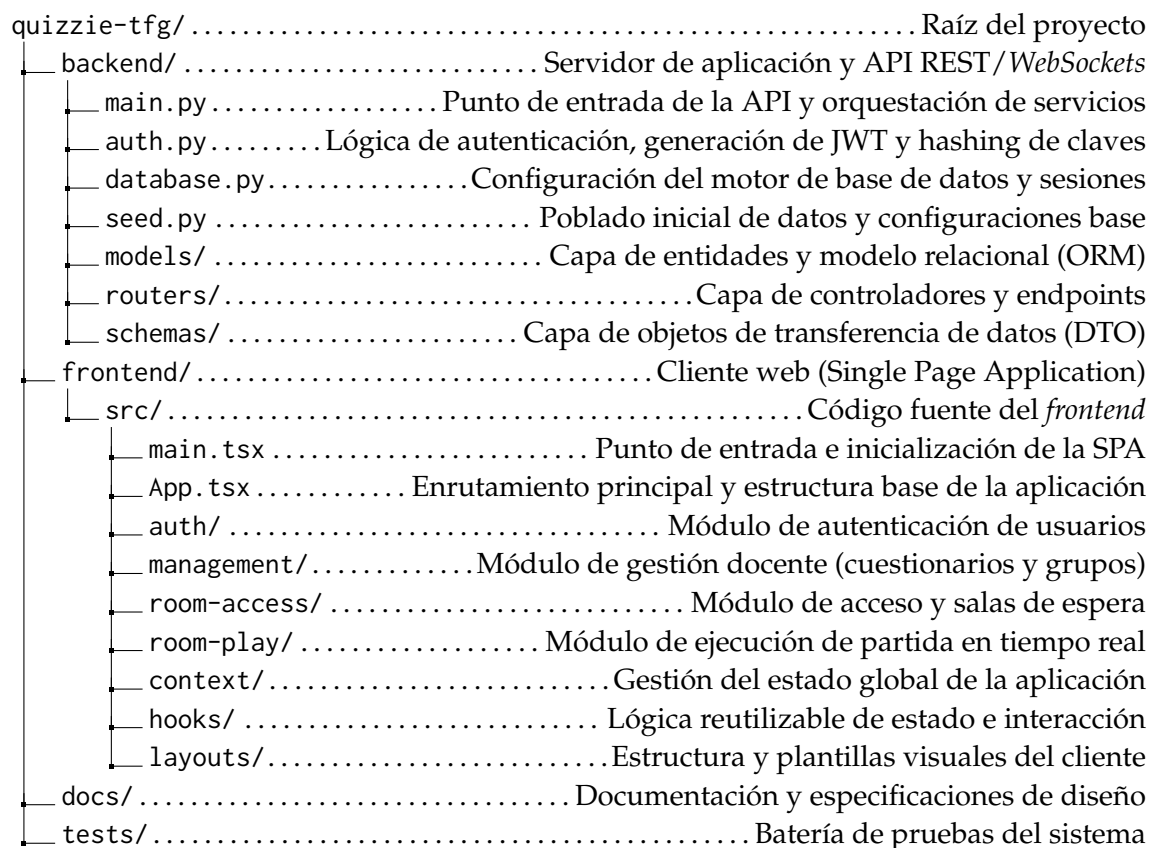


Figura 6.4: Estructura preliminar de módulos y componentes del proyecto.

6.4.1. Organización del backend

En el servidor, la estructura abstrae la lógica en tres archivos clave de inicialización y tres paquetes fundamentales que responden a la arquitectura en capas:

- **Servicios base de arranque:** Los módulos `main.py`, `database.py` y `seed.py` se encargan del punto de entrada del servidor, la gestión del motor de conexión a la base de datos y la siembra inicial de información necesaria para la ejecución.
- **Capa de controladores (routers/):** Define los puntos de entrada para las peticiones HTTP REST y gestiona las conexiones bidireccionales mediante *WebSocket*.
- **Capa de modelos (models/):** Define la representación de las entidades del dominio y sus relaciones en la base de datos relacional.
- **Capa de validación y DTOs (schemas/):** Define el manejo de datos para la entrada y salida de la API, garantizando el filtrado y la validación de la información.

6.4.2. Organización del frontend

En el cliente, el directorio `src/` agrupa los componentes organizados por dominios funcionales y capas de soporte:

- **Puntos de entrada y enrutamiento:** Los archivos `main.tsx` y `App.tsx` inicializan la aplicación cliente y configuran las rutas de navegación.
- **Módulos de dominio funcional:** Las carpetas `auth/`, `management/`, `room-access/` y `room-play/` encapsulan de forma independiente las distintas pantallas y flujos de trabajo de la aplicación.
- **Módulos de soporte e infraestructura:** Las carpetas `context/`, `hooks/` y `layouts/` proporcionan los servicios comunes de gestión del estado global, ganchos de lógica reutilizable y la estructura visual compartida.

6.5. Modelo de datos

El diseño de la capa de persistencia se articula en torno a la definición de las entidades del dominio, sus relaciones y las restricciones de integridad necesarias para soportar la ejecución del sistema. En el [diagrama de clases](#) se ilustra la estructura general y la interacción entre las distintas entidades previstas para la aplicación.

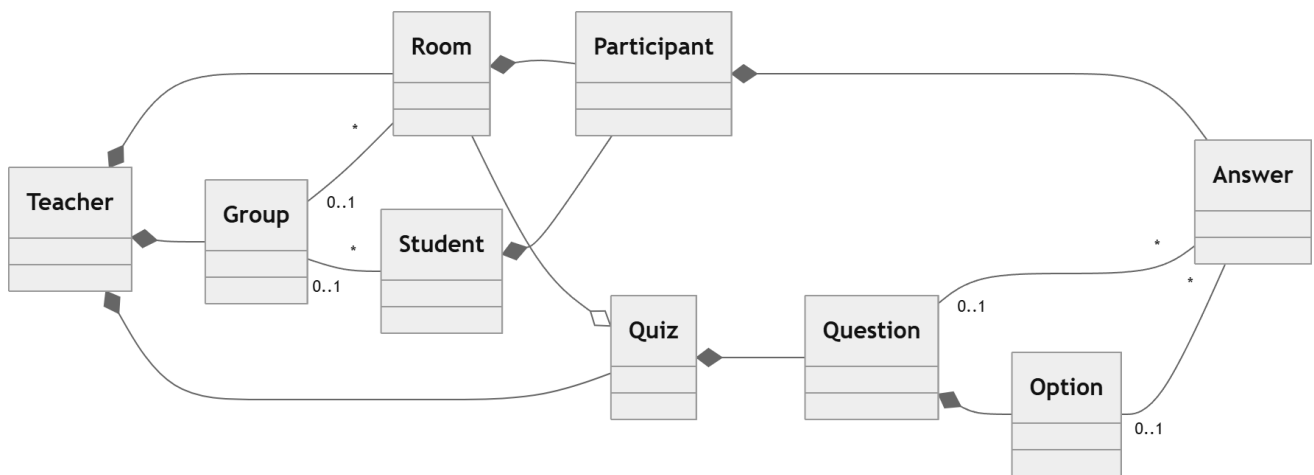


Figura 6.5: Diagrama de clases conceptual y relaciones globales del dominio.

Las entidades del sistema se organizan conceptualmente en tres módulos funcionales: **usuarios**, **contenido** y **sesiones**. A continuación se analiza cada uno de estos bloques.

6.5.1. Módulo de gestión de usuarios

Este módulo administra la identidad de los actores del sistema y la articulación del aula:

- **Teacher:** Modela al docente y sus credenciales de acceso, actuando como el propietario principal de los recursos creados (cuestionarios, salas y grupos).
- **Student:** Representa el registro persistente del alumno en la base de datos mediante su identificador o credencial de usuario (*nickname/UVUS*), el cual se vincula a las salas mediante la entidad intermedia de participación.
- **Group:** Articula a los estudiantes en conjuntos académicos para facilitar la asignación y seguimiento de partidas dirigidas.

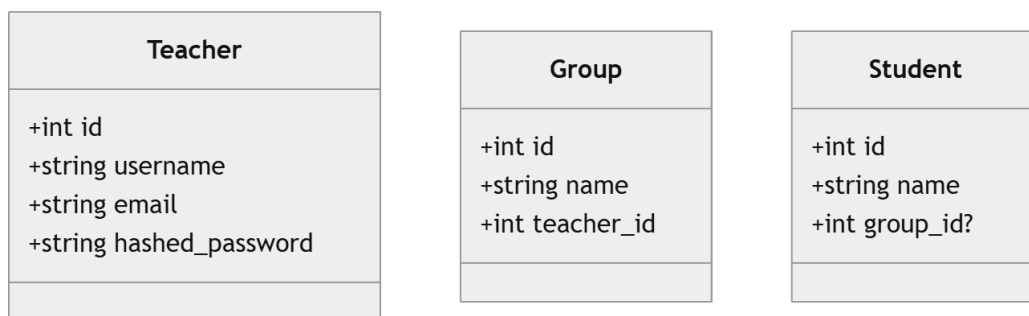


Figura 6.6: Entidades y atributos del módulo de usuarios.

6.5.2. Módulo de contenido

Comprende las entidades que estructuran el material de evaluación generado por el profesorado, vinculadas por relaciones de composición estricta:

- **Quiz:** Entidad raíz de la evaluación. Agrupa la información contextual del cuestionario e incluye metadatos visuales y de categorización.
- **Question:** Define cada ítem evaluable perteneciente a un cuestionario, determinando su enunciado y el peso en puntuación.
- **Option:** Contiene las alternativas de respuesta propuestas por ítem, marcando mediante un booleano cuál es la opción correcta para la validación automática.

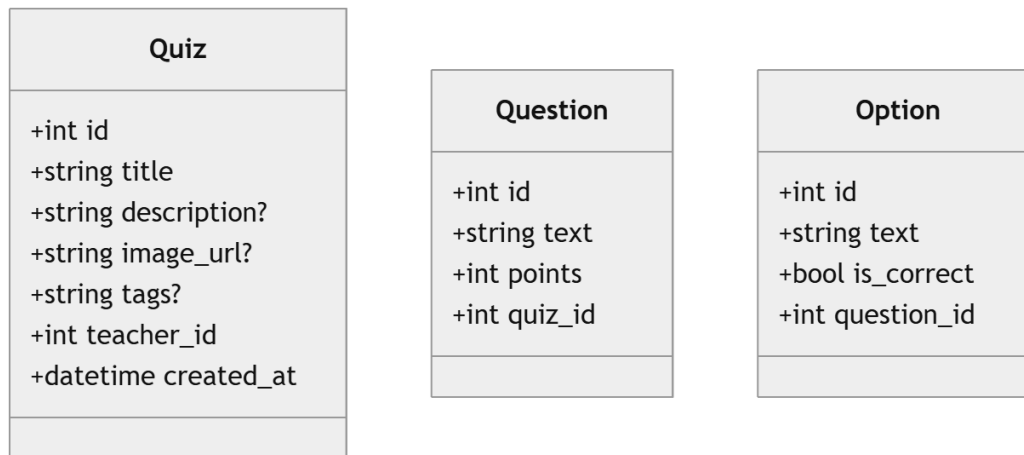


Figura 6.7: Entidades y atributos del módulo de contenido.

6.5.3. Módulo de sesiones

Engloba la lógica persistente encargada de soportar las partidas síncronas en tiempo real y el registro de la actividad de los estudiantes:

- **Room:** Gestiona la instancia activa de un cuestionario en vivo, controlando el acceso mediante un código de acceso y su disponibilidad.
- **Participant:** Actúa como entidad intermedia para resolver la relación de muchos a muchos entre Student y Room, registrando la incorporación del alumno a la partida en una marca de tiempo específica.
- **Answer:** Almacena los eventos de respuesta enviados por los participantes, registrando la opción elegida, los puntos calculados y el instante exacto de recepción.

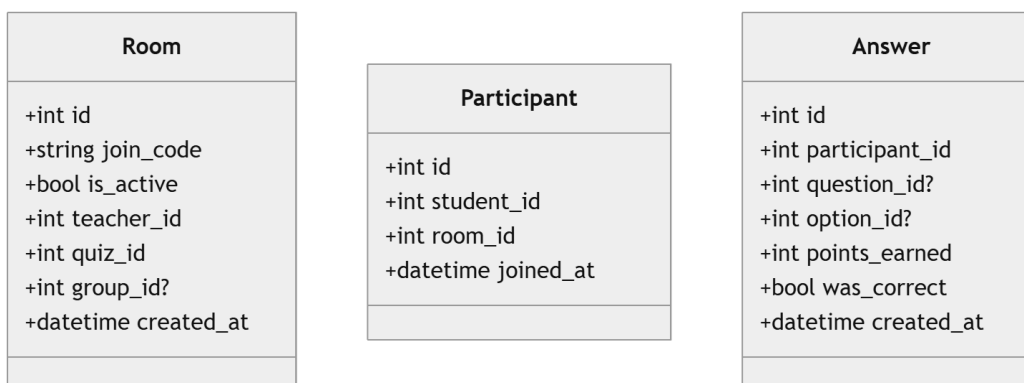


Figura 6.8: Entidades y atributos del módulo de sesiones.

6.6. Diseño de interfaz de usuario

El diseño de las interfaces de usuario (UI) y la experiencia de usuario (UX) constituye una fase crítica dentro del desarrollo de la plataforma, ya que condiciona la adopción, la facilidad de uso y la fluidez interactiva durante las sesiones de *tests*.

Para satisfacer las necesidades del sistema, el diseño ha considerado de manera diferenciada dos perfiles de usuario principales: el **profesorado**, que requiere herramientas de gestión, control de salas y análisis de resultados con baja latencia visual; y el **alumnado**, para el cual prima una interfaz limpia, altamente intuitiva y libre de distracciones, orientada al envío rápido de respuestas en entornos móviles o de escritorio.

6.6.1. Principios de diseño y guía de estilo

Las interfaces han sido diseñadas siguiendo principios consolidados de usabilidad y diseño centrado en el usuario (*User-Centered Design*):

- **Consistencia e identidad visual:** Se ha mantenido una estructura coherente en el maquetado de formularios, botones de acción y distribución de contenedores. Para facilitar la identificación inmediata del rol durante el análisis de las pantallas del sistema, los prototipos incorporan una codificación por color en su barra lateral:
 - **Borde magenta:** Asociado a la interfaz del perfil del profesor.
 - **Borde cian:** Asociado a la interfaz del perfil del estudiante.
- **Simplicidad y claridad cognitiva:** Durante las partidas en vivo, se reduce al mínimo el texto explicativo y los elementos estéticos para enfocar la atención del estudiante exclusivamente en las preguntas, opciones y temporizadores.
- **Diseño adaptable y accesibilidad:** La interfaz del estudiante está optimizada para una interacción táctil ágil, empleando áreas de pulsación amplias y garantizando un diseño plenamente adaptativo (*responsive*).

6.6.2. Flujos funcionales y prototipos

La arquitectura de navegación y la interacción visual de la plataforma se han organizado en cuatro secuencias funcionales principales. Estos escenarios se han maquetado en prototipos de alta fidelidad (*mockups*) y se encuentran detallados en el **Anexo A: Prototipos de interfaz de usuario**.

Los cuatro flujos cubren el ciclo de vida completo de una sesión interactiva, estructurándose según el momento operativo de la actividad:

1. **Registro y autenticación de usuarios:** Módulo administrativo de control de acceso y alta del perfil docente mediante credenciales institucionales.

2. **Creación de cuestionarios y gestión de salas:** Entorno de preparación previa donde el profesorado gestiona los contenidos, configura los parámetros de la sesión y habilita la sala de espera.
3. **Incorporación del estudiante a la sala:** Proceso simplificado de acceso en tiempo real para el alumnado mediante código PIN e identificación por UVUS o apodo, diseñado para minimizar la fricción de entrada.
4. **Ejecución de la partida en vivo:** Núcleo dinámico del sistema que coordina la experiencia síncrona entre ambos perfiles durante el desarrollo del cuestionario, abarcando la sincronización de tiempos, el envío de respuestas, la retroalimentación inmediata y la presentación de clasificaciones.

El diseño integrado de estos cuatro módulos responde a una lógica de interacción continua. Mientras que los dos primeros proporcionan la infraestructura de gestión previa a la clase, los dos últimos coordinan la sesión interactiva en directo.

6.7. Seguridad

La seguridad de la plataforma se ha diseñado atendiendo a la naturaleza diferenciada de sus dos perfiles de usuario: los profesores (usuarios registrados que requieren acceso persistente mediante credenciales) y los alumnos (usuarios efímeros que acceden a las partidas en vivo mediante un PIN y un alias, sin necesidad de registro previo).

6.7.1. Autenticación síncrona mediante JWT

Para la gestión de accesos del perfil docente se ha optado por un esquema sin estado (*stateless*) basado en *JSON Web Tokens* (JWT). Este enfoque evita almacenar el estado de las sesiones en el servidor para la API REST, permitiendo un diseño desacoplado.

El token se genera tras verificar las credenciales del usuario (cuyas contraseñas se almacenan de forma segura mediante algoritmos de *hashing* y derivación de claves) y se adjunta en el encabezado estándar `Authorization: Bearer <token>` para validar las solicitudes posteriores. A continuación se ilustra la secuencia de emisión y validación del token.

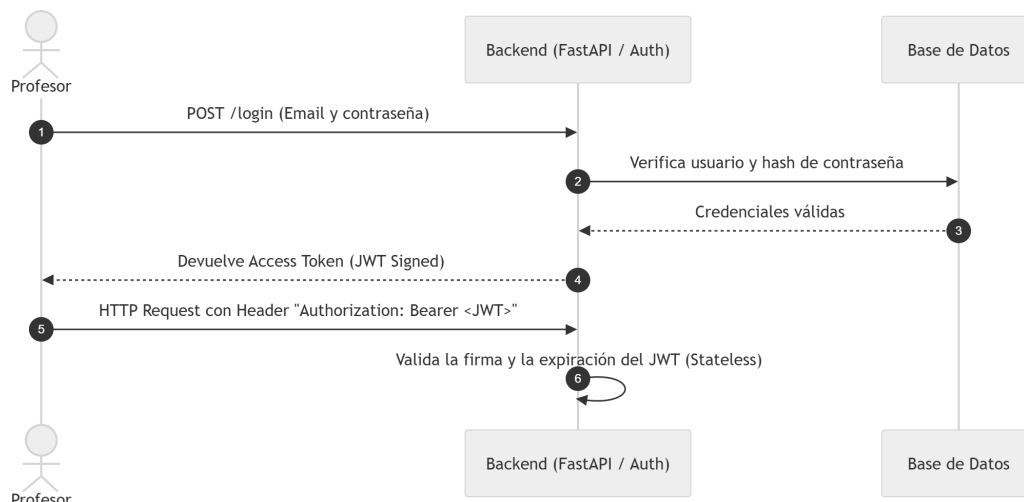


Figura 6.9: Flujo de autenticación del profesor mediante JWT.

6.7.2. Gestión de roles y control de acceso (RBAC)

El control de acceso se fundamenta en un modelo RBAC (*Role-Based Access Control*), definiendo dos niveles de autorización claramente delimitados:

- **Rol de profesor (Autenticado):** Protegido mediante la validación del JWT, otorga privilegios para la gestión integral de cuestionarios, la creación de salas en vivo, el control del flujo del juego y la consulta de informes.
- **Rol de participante (Efímero / No autenticado):** Asignado temporalmente a la sala activa sin requerir registro previo, restringe los permisos exclusivamente al envío de respuestas a las preguntas habilitadas en ese instante.

Bajo una arquitectura con autoridad en el servidor (*Server-Authoritative Architecture*), toda la lógica de validación, la temporización de la partida y la corrección de respuestas residen exclusivamente en el *backend*. El servidor actúa así como la única fuente de verdad (*Single Source of Truth, SSOT*) del sistema, garantizando la integridad del proceso de evaluación y previniendo la manipulación de datos desde el cliente (*anti-cheat*).

6.7.3. Tokens de verificación para participantes

Para consolidar las calificaciones de forma presencial y asegurar que el resultado pertenece al estudiante correspondiente, la plataforma incluye un mecanismo de verificación mediante códigos QR. El docente escanea en el aula el código presentado por el alumno, el cual contiene su alias y su token de verificación temporal (*verification_token*).

Como se resume en el siguiente diagrama, al realizar el escaneo la aplicación solicita al *backend* la verificación del participante. El servidor valida la correspondencia del token con la sala activa, confirma la actualización del estado

del participante inscribiendo la nota en la base de datos y difunde la actualización a la sala mediante la conexión en tiempo real por *WebSocket*.

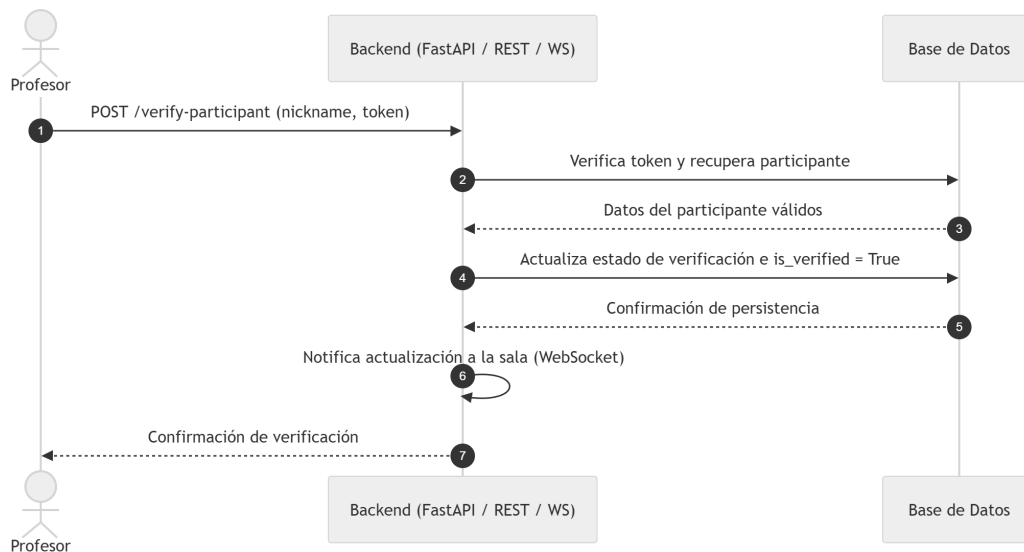


Figura 6.10: Diagrama del proceso de verificación del participante.

6.8. Documentación de la API

Para garantizar un desarrollo desacoplado entre el *frontend* y el *backend*, así como facilitar la mantenibilidad y evolución del sistema, la *API* de Quizzie se ha diseñado bajo el estándar OpenAPI.

6.8.1. Integración con OpenAPI y Swagger

La generación automatizada de la documentación se fundamenta en el uso de modelos de datos estrictos mediante Pydantic. Al definir las estructuras de solicitud (*request bodies*), parámetros de consulta (*query parameters*) y respuestas esperadas mediante esquemas fuertemente tipados, el servidor compila y expone dinámicamente la especificación en formato JSON (*openapi.json*).

Esta integración proporciona dos interfaces gráficas de documentación accesibles desde el propio navegador:

- **Swagger UI (/docs):** Entorno interactivo para probar y explorar los endpoints en tiempo real.
- **ReDoc (/redoc):** Vista estructurada orientada a especificación técnica de producción.

Como se ilustra en la siguiente figura, cualquier validación de tipo o código de estado HTTP se refleja en la documentación.

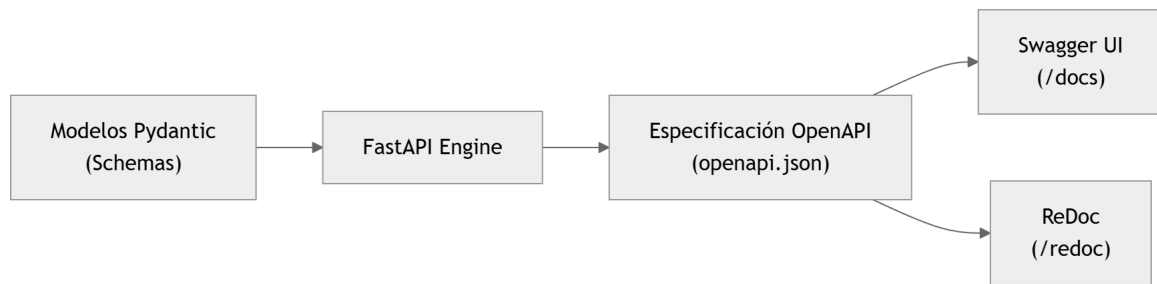


Figura 6.11: Flujo de generación de documentación OpenAPI desde los esquemas Pydantic.

6.8.2. Especificación de rutas fundamentales

La API traslada de forma directa los módulos conceptuales definidos en la [sección de modelado de datos](#). Esta separación de responsabilidades se materializa en el código fuente dividiendo los enrutadores (routers/) y los modelos de datos (models/) en tres bloques principales: users, content y stage.

A continuación se resumen las rutas proyectadas más representativas para cada uno de estos módulos:

Usuarios (routers/users.py)

- POST /users/login: Autentica las credenciales del docente y emite el token de acceso para autorizar sus peticiones posteriores.
- POST /users/register: Registra a un nuevo profesor en la plataforma e inicia el proceso de verificación por correo electrónico.

Contenido (routers/content.py)

- POST /content/quizzes: Permite al profesor crear un cuestionario completo con su conjunto de preguntas y opciones de respuesta.

Sesiones (routers/stage.py) Combina peticiones REST para la gestión del estado de la sala y un canal WebSocket para la comunicación en tiempo real:

- POST /stage/rooms: Crea e inicializa una nueva sala de juego asociada a un cuestionario y aplica su configuración inicial.
- POST /stage/participants: Permite el acceso efímero de los estudiantes a una sala activa mediante un PIN y un alias.
- POST /stage/answers: Almacena la respuesta enviada por un alumno a la pregunta activa.

- POST /stage/rooms/{room_id}/verify-participant: Permite al docente verificar presencialmente el resultado de un estudiante en el aula, según se describe en la [sección de verificación de seguridad](#).
- WS /stage/rooms/{room_id}/ws: Mantiene la conexión en tiempo real para sincronizar el avance del juego, los temporizadores y la clasificación entre todos los participantes.

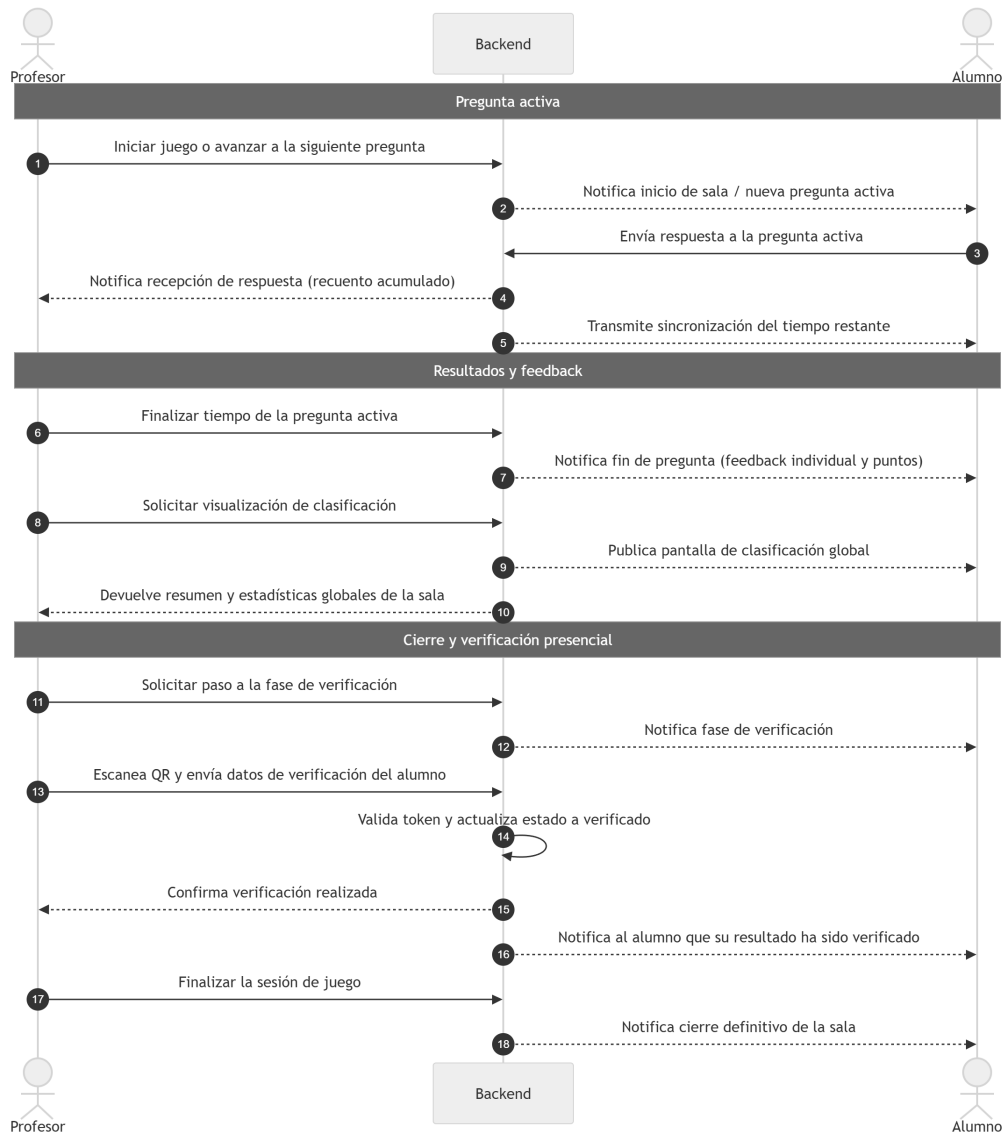


Figura 6.12: Secuencia del intercambio de eventos durante la sesión en tiempo real.

7. Metodología técnica y de desarrollo

Una vez establecida la arquitectura y el diseño conceptual del sistema, resulta imprescindible definir un marco operativo riguroso para la fase de construcción e integración del código. En el desarrollo de software, la ausencia de políticas explícitas en el control de versiones incrementa paulatinamente la deuda técnica y dificulta la detección temprana de errores.

Este capítulo detalla la metodología técnica implantada en el proyecto *Quizzie*. En él se abordan las políticas de trabajo en el repositorio, la automatización del ciclo de Integración y Despliegue Continuos (CI/CD), la gestión segura de perfiles de configuración y el análisis estático de la calidad del código.

7.1. Ecosistema base de desarrollo local

Concretando la selección tecnológica descrita en la subsección [soporte de desarrollo](#), el trabajo local se sostiene sobre tres pilares operativos principales. La integración de este ecosistema busca asegurar la reproducibilidad homogénea del proyecto, garantizar el aislamiento de dependencias y simplificar la ejecución de herramientas de desarrollo:

- Gestión de entorno *backend* con **uv**: En lugar de recurrir a entornos virtuales creados manualmente (venv) o gestores tradicionales, el ciclo de vida del *backend* se articula íntegramente mediante el gestor de dependencias *uv*. Esta herramienta centraliza el aislamiento de la versión de Python, la resolución del árbol de dependencias expresado en `pyproject.toml` y la ejecución de comandos de soporte (linters, formateadores y motores de prueba) en un entorno reproducible mediante el prefijo `uv run`.
- Gestión de entorno *frontend* con **npm** y **Node.js**: Para la parte cliente, el ecosistema de trabajo se apoya en **npm** sobre el entorno de ejecución de **Node.js**. A través del fichero `package.json`, se estandarizan las tareas de desarrollo, la compilación y la verificación de tipos con TypeScript.
- Sistema de control de versiones distribuido con **Git**: Registra la evolución del código fuente, sirviendo de nexo entre el trabajo en la máquina local y las políticas de integración remota que se detallan a continuación.

Esta combinación permite que cualquier desarrollador pueda clonar el repositorio y levantar el entorno completo de trabajo de forma idéntica con un conjunto mínimo y estandarizado de comandos.

Los pasos detallados para la preparación inicial del entorno y el registro de dependencias se recogen en el [manual de instalación](#).

7.2. Políticas del repositorio y flujo de trabajo

Para asegurar un desarrollo estructurado, auditable y mantenible, se ha definido una normativa estricta sobre el repositorio centralizado en GitHub[40]. Esta normativa abarca desde el modelado de tareas hasta la consolidación de cambios en los entornos estables, siguiendo un flujo secuencial estandarizado compuesto por seis etapas consecutivas:

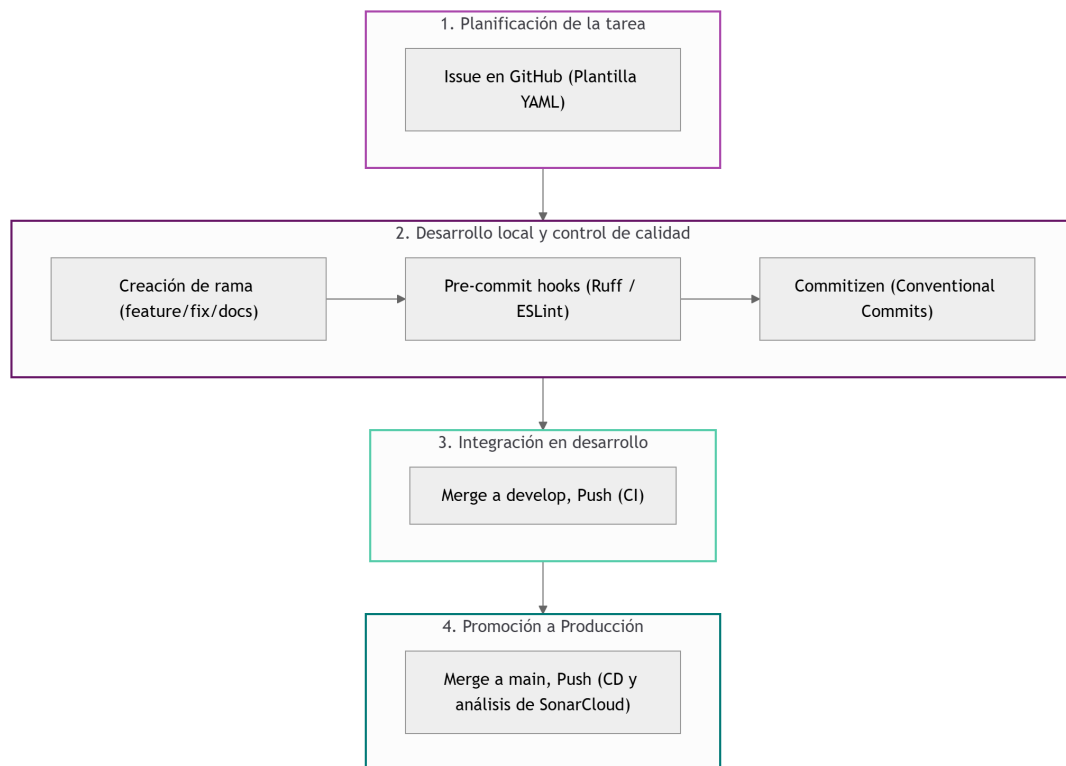


Figura 7.1: Ciclo de trabajo secuencial implementado en Quizzie.

7.2.1. Gestión de tareas mediante plantillas de *GitHub Issues*

Todo trabajo de desarrollo, refactorización, documentación o pruebas debe originarse mediante una incidencia (*Issue*) en GitHub. Esto garantiza la trazabilidad bidireccional entre los requisitos funcionales (RF), los objetivos del proyecto y las modificaciones del código.

Para homogeneizar el registro de *Issues*, se han implementado plantillas declarativas en formato *YAML* ubicadas en el directorio `.github/ISSUE_TEMPLATE` del repositorio:

- `feature_request.yml`: Diseñada para la implementación de nuevos Requisitos Funcionales. Exige vincular explícitamente el identificador del requisito (ej. RF-07), el objetivo asociado (OBJ-02), la descripción técnica de la solución y los criterios de aceptación.
- `docs.yml`: Orientada a la redacción de documentación técnica o capítulos de la memoria en $\text{\LaTeX}$, indicando el documento afectado y los criterios de revisión.
- `devops.yml`: Destinada a tareas de automatización, ajuste de canalizaciones de GitHub Actions, integración con SonarCloud o configuración de infraestructura.
- `test.yml`: Específica para la creación o ampliación de baterías de pruebas (unitarias, integración, de componentes, E2E, o pruebas de carga con Locust).

El uso de esta estructura estandarizada asegura que cualquier incidencia registrada contenga los datos necesarios para que el desarrollo posterior se adecúe a los **criterios de aceptación**.

7.2.2. Estrategia de ramas

El proyecto adopta una variante adaptada del modelo *Git Flow* orientada a entornos de desarrollo ágil. A diferencia del modelo tradicional que emplea solicitudes de extracción (*Pull Requests*) complejas para un único desarrollador, *Quizzie* utiliza ramas de trabajo efímeras e integración mediante fusiones directas (*merge*) locales.

Rama	Propósito	Origen
main	Código estable en producción. Solo recibe código probado y validado.	—
develop	Rama principal de integración y desarrollo continuo.	main
feature/*	Implementación de requisitos funcionales (RF).	develop
fix/*	Corrección de errores y mejoras rápidas.	develop
docs/*	Actualización de documentación o memoria del TFG.	develop
test/*	Incorporación o mejora de baterías de pruebas.	develop
refactor/*	Reestructuración interna de código sin cambio de comportamiento.	develop
build/*	Ajustes en CI/CD, dependencias o infraestructura.	develop

Tabla 7.1: Clasificación de ramas, nomenclatura y rama de origen.

El flujo de aislamiento y convergencia entre las ramas temporales, la rama de integración develop y la rama de producción main se esquematiza en el siguiente grafo:

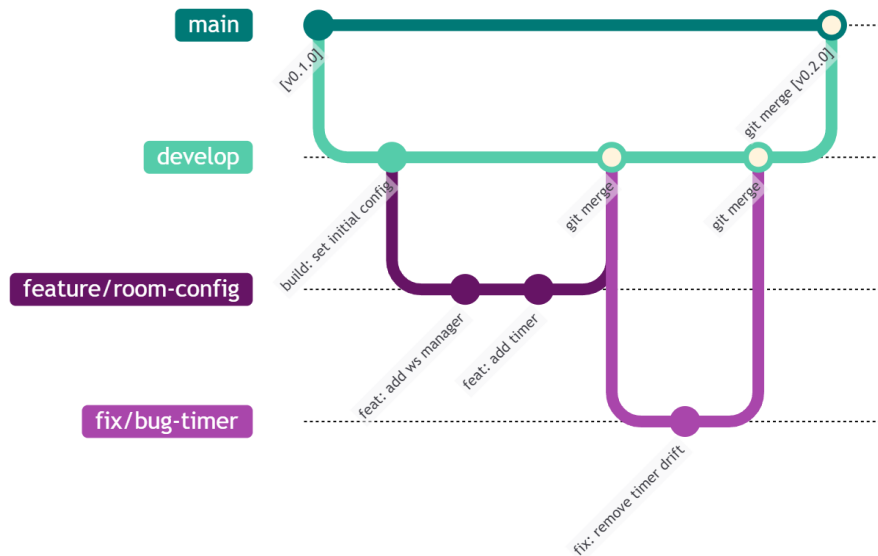


Figura 7.2: Diagrama del flujo de ramas en el repositorio.

7.2.3. Verificación local mediante *pre-commit*

Antes de que cualquier *commit* (modificación en el código) sea registrada definitivamente en el historial de Git, es fundamental asegurar que cumple con los estándares de calidad y formato fijados en el proyecto. Para automatizar este control, en la máquina local del desarrollador se utiliza la herramienta **pre-commit**, la cual evalúa el código de forma previa a la confirmación.

La configuración centralizada de estas verificaciones se define en el archivo `.pre-commit-config.yaml` ubicado en la raíz del repositorio. En dicho fichero se declaran los conectores (*hooks*) y herramientas que se deben ejecutar de forma transparente sobre los archivos modificados:

1. **Backend (Python):** Se integra la herramienta **Ruff** para realizar un análisis estático ultra rápido, corregir automáticamente fallos de formato, eliminar importaciones sin usar y garantizar el cumplimiento del estándar PEP 8 [41].
2. **Frontend (React / TypeScript):** Se ejecutan **ESLint** y **Prettier** para inspeccionar la sintaxis de los componentes JSX/TSX y unificar el estilo visual del código.
3. **Seguridad y limpieza:** Se añaden verificaciones generales para detectar y bloquear la subida accidental de archivos de secretos (`.env`), claves privadas o archivos con formato incorrecto.

Para forzar manualmente la inspección de la totalidad del código fuente sin necesidad de intentar un *commit*, se utiliza la instrucción:

```
uv run pre-commit run --all-files
```

Si alguna de las comprobaciones detecta un fallo o una discrepancia de formato, *pre-commit* aborta la confirmación automáticamente, exigiendo corregir el problema antes de continuar.

7.2.4. Estandarización de commits

Para mantener un historial de control de versiones limpio, semántico y legible, se ha adoptado la especificación **Conventional Commits**. Esta convención asigna un significado explícito a cada confirmación, lo que permite auditar la evolución del código y automatizar la gestión de versiones mediante Versionado Semántico (*SemVer*) [42].

La construcción de los mensajes en la máquina local se canaliza a través de la herramienta **Commitizen**. Al ejecutar la instrucción interactiva correspondiente, se guía al desarrollador en la redacción del mensaje respetando la estructura obligatoria:

`<tipo>(<alcance>) : <descripción>`

Tipo	Descripción	Impacto SemVer
feat	Incorporación de una nueva funcionalidad o requisito.	MINOR (v0.X.0)
fix	Corrección de un fallo o error en el código.	PATCH (v0.0.X)
docs	Cambios exclusivamente en la documentación o memoria.	Ninguno
style	Ajustes de formato o estilo que no alteran la lógica.	Ninguno
refactor	Reestructuración de código sin cambio de comportamiento.	Ninguno
test	Añadido o corrección de baterías de pruebas.	Ninguno
build	Modificaciones en la configuración de build o dependencias.	Ninguno
ci	Ajustes en los flujos de integración continua (CI/CD).	Ninguno

Tabla 7.2: Tipos de *commits* según Conventional Commits.

Asimismo, se exige establecer la trazabilidad bidireccional entre el código registrado y las incidencias de GitHub. Para ello, en el pie del mensaje de confirmación se deben incluir las referencias correspondientes:

- Refs #83: Vincula la confirmación con la incidencia #83 en la interfaz de GitHub para facilitar su seguimiento.
- Closes #83 / Fixes #83: Asocia el cambio y provoca el cierre automático de la incidencia al fusionar la rama en la rama principal (main).

Para automatizar este proceso, se ha configurado previamente una plantilla en el proyecto que guía al desarrollador paso a paso. Al ejecutar la siguiente instrucción en la terminal, se despliega el asistente de Commitizen y se genera el *commit* de forma directa en el repositorio local una vez rellenados los campos:

```
uv run cz commit
```

7.2.5. Consolidación y despliegue a producción

Una vez completado el desarrollo y comprobada la estabilidad de las funcionalidades en la rama de integración (*develop*), se procede a la transferencia del código hacia la rama principal (*main*). Este procedimiento traslada de forma controlada únicamente versiones consolidadas que han superado previamente todas las verificaciones locales y de integración continua. Dicho flujo de trabajo se encuentra oficialmente respaldado por las reglas de protección de ramas en GitHub (*Branch Protection Rules*), las cuales impiden subidas directas (*direct push*) y fuerzan el cumplimiento de las políticas de validación establecidas.

El procedimiento para llevar a cabo la fusión del código se compone de seis pasos clave. A continuación, se ilustran los pasos a seguir para integrar el código en la rama *main*, junto a las acciones y comandos de consola correspondientes.

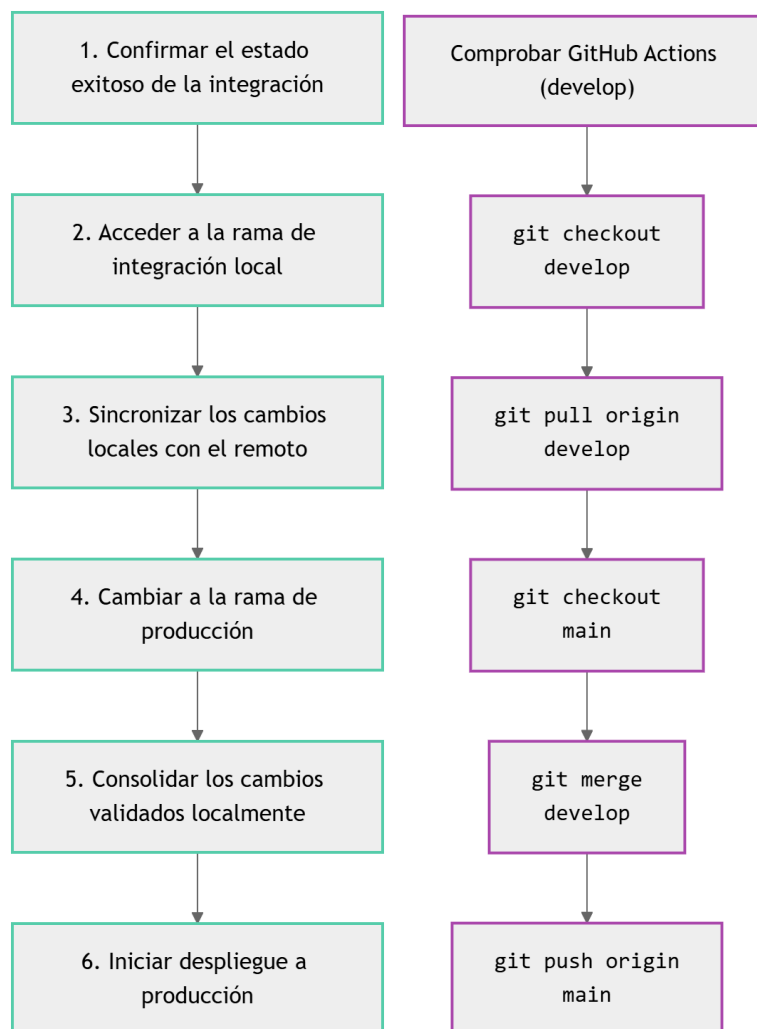


Figura 7.3: Procedimiento para integrar código en la rama *main*.

Para garantizar la estabilidad del entorno de producción, la rama main se reserva de forma exclusiva para el código listo para su despliegue. Las versiones del sistema (*Releases*) se formalizan al culminar las *issues* asociadas a cada una de las etapas de desarrollo, materializando formalmente los entregables de producto software definidos en la EDT. En dicho punto, se asigna la correspondiente etiqueta de versión semántica (*SemVer*) y se documentan las modificaciones directamente en la sección de *Releases* de GitHub a partir de las incidencias resueltas.

7.3. Métricas de calidad y análisis estático de código

Con el fin de asegurar la mantenibilidad y robustez del código consolidado en la rama principal (main), se utiliza **SonarCloud** como auditor externo para el análisis estático. Esta herramienta evalúa de forma continuada la salud del proyecto, monitorizando la presencia de errores lógicos, brechas de seguridad, complejidad cognitiva y el porcentaje global de cobertura de pruebas tras cada integración en main, definiendo los criterios de calidad que posteriormente se automatizan en el flujo de CI/CD.

7.3.1. Justificación y métricas objetivo (*Quality Gate*)

Con el objetivo de evitar la acumulación de deuda técnica y asegurar la mantenibilidad a largo plazo, se ha configurado un umbral de calidad (*Quality Gate*) de obligado cumplimiento para cualquier nueva funcionalidad integrada en main.

La siguiente tabla resume los objetivos mínimos exigidos por el sistema y la justificación técnica de cada uno:

Métrica	Objetivo	Justificación técnica
Cobertura	> 80 %	Garantizar que la mayoría de los flujos críticos del sistema están probados.
Duplicación	< 3 %	Fomentar el principio DRY y facilitar el mantenimiento futuro evitando la repetición de código.
Fiabilidad	Calificación A	Probar la ausencia total de errores lógicos conocidos en el código nuevo.
Seguridad	Calificación A	Asegurar la inexistencia de vulnerabilidades y puntos de acceso no protegidos.
Mantenibilidad	Calificación A	Mantener una deuda técnica mínima y un nivel bajo de complejidad cognitiva.

Tabla 7.3: Métricas objetivo y umbrales del *Quality Gate* en SonarCloud.

7.3.2. Configuración y exclusiones técnicas

Para que los indicadores del panel de control sean un reflejo fiel de la calidad del producto final, se han aplicado las siguientes configuraciones de calibración en la plataforma:

- **Alcance de la cobertura de pruebas:** La métrica de cobertura de código integrada en SonarCloud se enfoca de forma exclusiva en el *backend*. Esto se debe a que el reporte de ejecución de **Pytest** (`coverage.xml`) es el único que se exporta y vincula al análisis estático, mientras que las pruebas de componentes del *frontend* se gestionan y auditan de manera independiente en su respectivo entorno de integración continua
- **Exclusión de scripts de carga (`seed.py`):** Se ha excluido el archivo `backend/seed.py` del cálculo de cobertura y duplicación. Al tratarse de un script para la siembra masiva de datos con estructuras repetitivas, su inclusión penalizaba artificialmente la media de cobertura y distorsionaba la métrica de código duplicado.
- **Integración del reporte de cobertura:** Se ha configurado la propiedad `sonar.python.coverage.reportPaths=coverage.xml` para garantizar una lectura e interpretación precisa del informe XML generado por **Pytest** (`pytest-cov`).
- **Estandarización del entorno de análisis:** La ejecución del *job* de CI para SonarQube se realiza sobre la imagen virtual `ubuntu-latest` en GitHub Actions, asegurando la consistencia en el mapeo de rutas de archivos entre el reporte de cobertura y el código fuente.

7.4. Integración y Despliegue Continuo (CI/CD)

Para garantizar la calidad del software, la detección temprana de errores, la seguridad de las dependencias y la entrega continua del servicio en *Quizzie*, se ha diseñado e implementado una arquitectura de Integración y Despliegue Continuos (CI/CD) fundamentada en GitHub Actions. Este flujo automatizado actúa como un control de calidad estricto antes de actualizar cualquier versión del código hacia los entornos de integración y producción.

7.4.1. Ciclo de vida y arquitectura

El flujo de trabajo automatizado se estructura en tres *workflows* independientes en GitHub Actions, los cuales responden a eventos específicos en el repositorio y gestionan las distintas fases de validación técnica:

1. **Integración Continua (CI):** Se activa automáticamente ante eventos de *push* en la rama *develop*. Ejecuta la batería de análisis estático, seguridad, pruebas unitarias y de integración, de componentes, *End-to-End* y evaluación de rendimiento.
2. **Análisis de calidad (QA):** Se dispara tras consolidar cambios en la rama *main*. Evalúa métricas avanzadas de mantenibilidad, duplicación de código y deuda técnica en la plataforma SonarCloud.
3. **Despliegue Continuo (CD):** Se ejecuta de forma paralela al análisis de calidad tras una fusión exitosa en *main*, activando el despliegue automático en la infraestructura de producción.

La siguiente figura ilustra el funcionamiento general de los *workflows*, destacando las dependencias entre *jobs* y la separación de responsabilidades según la rama afectada.

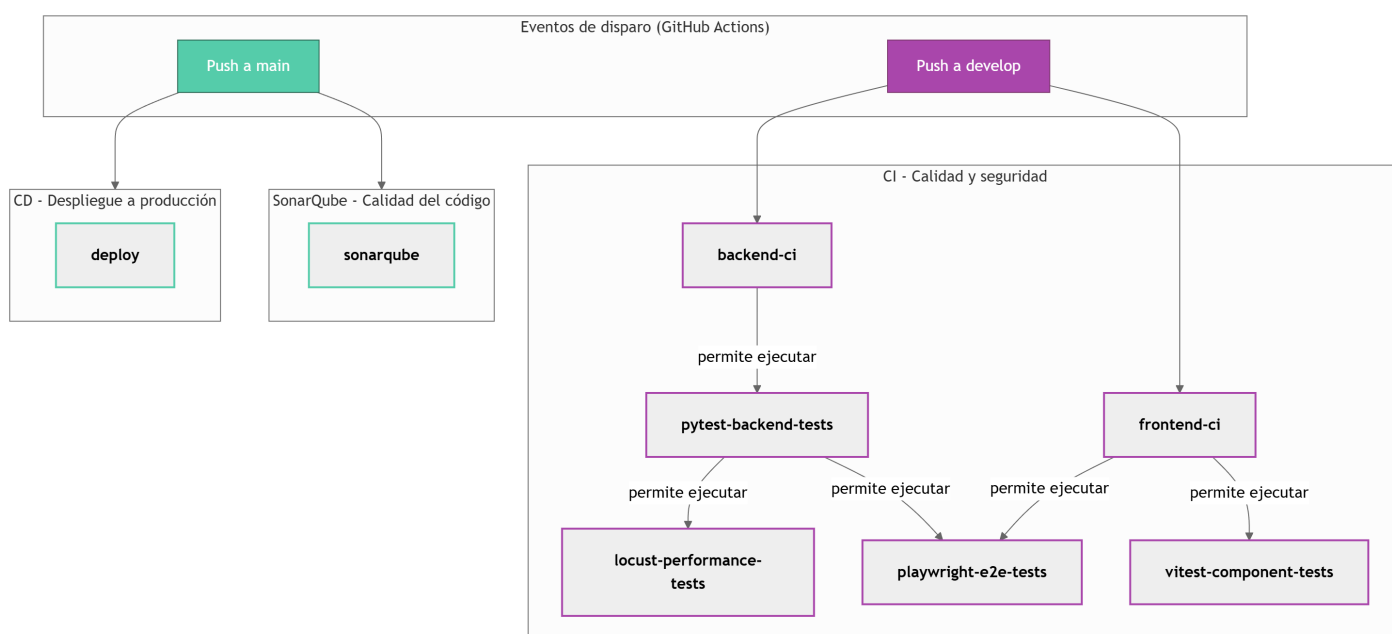


Figura 7.4: Estructura del *pipeline* de CI/CD

7.4.2. Flujo de Integración Continua (CI)

El proceso de validación de calidad y seguridad definido en `quality_and_security.yml` se compone de seis *jobs* interconectados distribuidos en distintas etapas secuenciales:

Análisis estático y verificación

- **backend-ci:** Inicializa el entorno Python 3.12 con `uv`, analiza el formato y estilo del código mediante **Ruff** y realiza la auditoría de seguridad de dependencias con `pip-audit`.

- **frontend-ci**: Prepara el entorno Node.js 20, ejecuta la verificación sintáctica mediante **ESLint** y valida el tipado de TypeScript junto con la compilación del proyecto.

Pruebas de código y componentes

- **pytest-backend-tests**: Requiere la superación previa de **backend-ci**. Ejecuta la suite de pruebas unitarias y de integración del *backend* mediante **Pytest** y exporta el informe de cobertura XML (**backend-coverage**) como artefacto.
- **vitest-component-tests**: Se ejecuta tras la consecución de **frontend-ci**. Realiza las pruebas sobre la interfaz de usuario utilizando **Vitest** y almacena su reporte de cobertura.

Pruebas de sistema y rendimiento

- **playwright-e2e-tests**: Depende del éxito conjunto de **pytest-backend-tests** y **frontend-ci**. Despliega el servidor backend de pruebas y ejecuta la batería *End-to-End* con **Playwright** simulando la navegación de usuario en navegadores *headless*.
- **locust-performance-tests**: Se inicia tras la superación de **pytest-backend-tests**. Inicia el servidor de prueba y ejecuta la simulación de carga concurrente con **Locust**, validando mediante `evaluate_thresholds.py` que las latencias y tasas de error se mantengan dentro de los límites establecidos en la *estrategia de testing*.

7.4.3. Análisis de calidad con SonarQube

Al consolidar cambios en la rama (*main*), se activa el *workflow* `sonar.yml`. Este proceso reejecuta la medición de cobertura del *backend* y envía los resultados a la plataforma SonarCloud mediante la acción oficial `SonarSource/sonarqube-scan-action`, evaluando los umbrales de calidad (*Quality Gate*) y la deuda técnica acumulada.

7.4.4. Estrategia de Despliegue Continuo (CD)

Tras producirse un *push* en la rama *main*, el despliegue del entorno de producción se efectúa automáticamente de forma segregada:

- **Backend (Render)**: Se gestiona mediante el *workflow* `deploy.yml`, el cual notifica al servicio de Render tras cada *push* en *main* para activar la actualización del servidor de aplicaciones.

- **Frontend (Vercel):** Se apoya en la integración nativa de Vercel con el repositorio de GitHub, la cual detecta los nuevos *commits* en la rama main e inicia automáticamente la compilación y publicación de la interfaz.

7.4.5. Resumen de la estructura de CI/CD

La siguiente figura resume la estructura del flujo de CI/CD, integrando los *jobs*, scripts y herramientas utilizadas en cada fase:

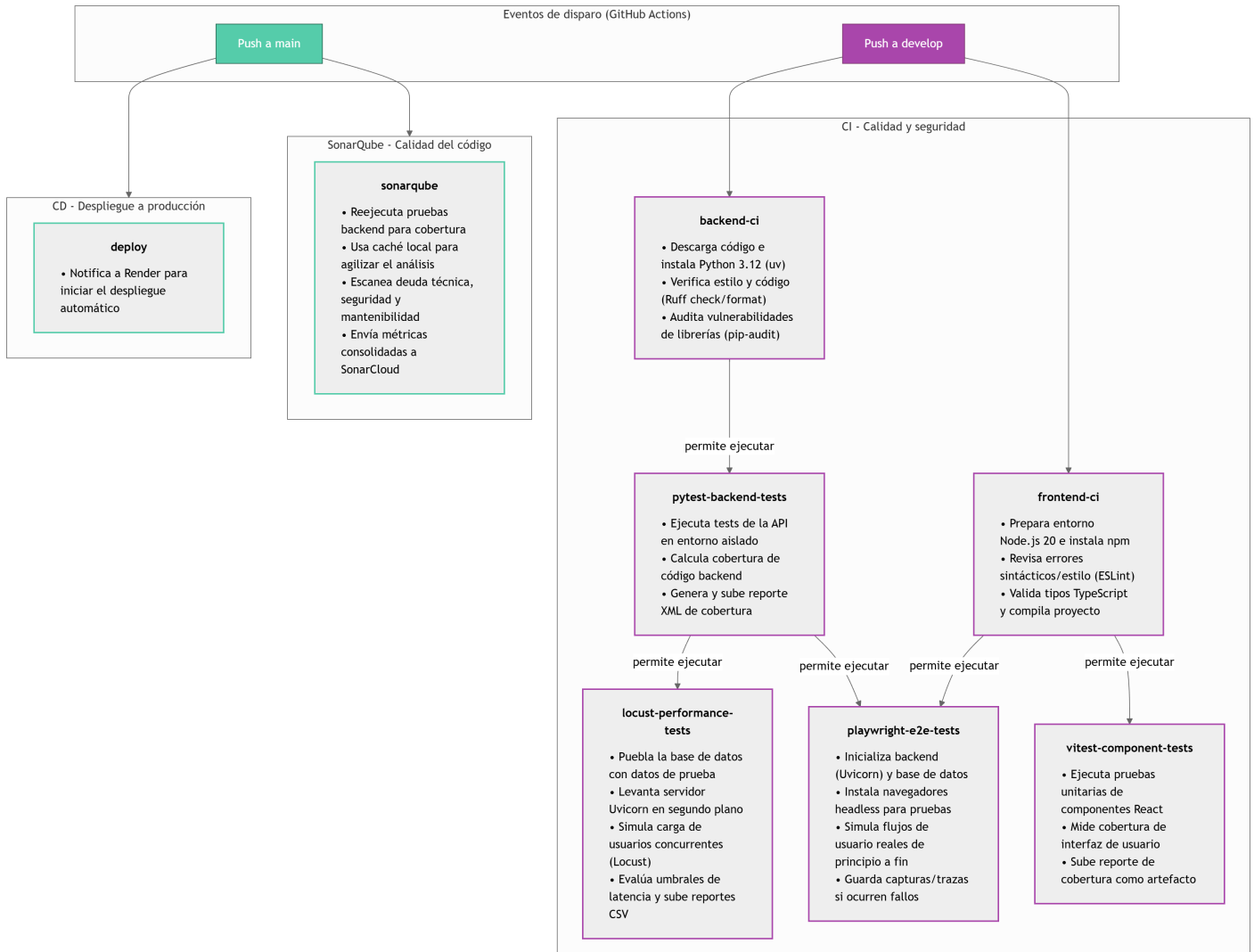


Figura 7.5: Resumen detallado de la estructura de CI/CD.

7.5. Gestión de configuración y entorno

Para garantizar la seguridad, el cumplimiento de buenas prácticas y la portabilidad entre el desarrollo local y los entornos de producción, *Quizzie* desvincula completamente la configuración del código fuente. Los datos sensibles, credenciales de correo y claves de cifrado se gestionan de forma centralizada mediante variables de entorno seguras.

7.5.1. Categorización funcional de los secretos

Las variables de entorno del sistema se estructuran en cinco categorías funcionales según su cometido dentro de la arquitectura:

- **Autenticación y seguridad JWT:** Comprende la clave secreta de firma **SECRET_KEY**, el algoritmo de cifrado **ALGORITHM** y el tiempo de expiración de las sesiones **ACCESS_TOKEN_EXPIRE_MINUTES**. Garantizan la integridad y validez de los tokens de usuario.
- **Servicio de inteligencia artificial (Gemini API):** Agrupa las credenciales necesarias **GEMINI_API_KEY** para la autenticación y consumo de los modelos de lenguaje de Google Gemini dedicados a la generación automatizada de preguntas y cuestionarios.
- **Servicio de correo transaccional (SMTP):** Incluye los parámetros de conexión al servidor SMTP **SMTP_HOST**, **SMTP_PORT**, las credenciales de autenticación **SMTP_USERNAME**, **SMTP_PASSWORD** y la dirección del remitente **SMTP_FROM**.
- **Conectividad y cliente React:** Contiene la URL base del backend **VITE_API_URL**, requerida para orientar las peticiones HTTP del frontend tanto en desarrollo local como en producción.
- **Integración continua y calidad (CI/CD):** Agrupa los tokens de autenticación para servicios externos del pipeline, como el token de análisis en SonarCloud **SONAR_TOKEN** y la URL del disparador de despliegue en Render **RENDER_DEPLOY_HOOK_URL**.

7.5.2. Entorno de desarrollo local

Para facilitar la configuración durante el desarrollo sin comprometer la seguridad del repositorio, se proporciona la plantilla `.env.example`. Este archivo define la estructura y variables obligatorias del sistema definidas anteriormente.

A partir de dicha plantilla, el desarrollador genera su archivo local `.env`, ignorado por Git mediante `.gitignore` para evitar filtraciones. Este proceso se aborda en el [manual de instalación](#).

7.5.3. Distribución de secretos por plataforma

En los entornos remotos se aplica de forma estricta el **principio de mínimo privilegio**, tal y como define el *National Institute of Standards and Technology* (NIST) en su informe especial SP 800-53 [43]. Bajo este criterio, se garantiza que cada plataforma o servicio contenga de forma aislada únicamente las variables estrictamente necesarias para cumplir su función técnica, reduciendo la superficie de ataque y minimizando el impacto ante eventuales vulnerabilidades.

La siguiente tabla resume la matriz de presencia de variables de entorno y secretos según la plataforma de despliegue:

Variable / Secreto	GitHub	Render	Vercel
SECRET_KEY	Sí	Sí	Sí
ALGORITHM	Sí	Sí	Sí
ACCESS_TOKEN_EXPIRE_MINUTES	–	Sí	Sí
VITE_API_URL	–	–	Sí
GEMINI_API_KEY	Sí	Sí	–
RENDER_DEPLOY_HOOK_URL	Sí	–	–
SONAR_TOKEN	Sí	–	–
PYTHON_VERSION / PYTHONPATH	–	Sí	–
SMTP_HOST / SMTP_PORT	–	Sí	–
SMTP_USERNAME / SMTP_PASSWORD	–	Sí	–
SMTP_FROM	–	Sí	–

Tabla 7.4: Variables de entorno y secretos por plataforma.

Justificación por entorno

- **GitHub (Actions):** Registra únicamente las credenciales requeridas para ejecutar las suites de prueba durante los *workflows*, la autenticación del escáner en SonarCloud y el disparador del despliegue en producción.
- **Render (Backend):** Concentra la configuración del servidor de aplicaciones, alojando la infraestructura de correo SMTP, la clave de acceso a la API de Gemini, los parámetros de runtime de Python y la gestión del ciclo de vida de los tokens JWT.
- **Vercel (Frontend):** Contiene exclusivamente el parámetro público VITE_API_URL para el enrutamiento de peticiones HTTP desde la SPA React hacia la API, además de los parámetros de control de sesión en el cliente.

8. Implementación y pruebas

Una vez establecida la base arquitectónica y el diseño del sistema, corresponde abordar la materialización técnica de la plataforma, transformando los requisitos y decisiones de diseño arquitectónico previamente definidos en componentes funcionales de software. Para ello, en primer lugar se detallan los módulos críticos de la implementación tanto en el *backend* como en el *frontend*, haciendo especial énfasis en la sincronización en tiempo real y la seguridad. Posteriormente, se expone la estrategia integral de pruebas, los criterios de calidad adoptados y los resultados de cobertura obtenidos para garantizar la robustez del sistema.

8.1. Componentes clave e implementación del sistema

En esta sección se exponen los aspectos más representativos del núcleo de *Quizzie*, analizando las decisiones de diseño y patrones de software adoptados para dar respuesta a los requisitos funcionales más complejos: la comunicación bidireccional, la gestión del ciclo de vida de las salas y la seguridad del estado en el *frontend*.

8.1.1. Comunicación en tiempo real mediante WebSockets

La interactividad en vivo entre el profesor y los alumnos requiere una arquitectura de baja latencia capaz de difundir eventos de forma simultánea a múltiples clientes mediante el protocolo WebSocket [9].

Gestor de conexiones en el backend (`ConnectionManager`)

En la ruta `backend/routers/stage.py`, el gestor de conexiones implementa una adaptación concurrente del patrón de diseño *Observer* [44] combinada con una arquitectura de publicación-suscripción (*Pub-Sub*) [45], permitiendo registrar, desacoplar y administrar dinámicamente el ciclo de vida de las instancias activas de WebSocket asociadas a cada sala mediante su identificador único (`room_id`).

```

class ConnectionManager:
    def __init__(self):
        self.active_connections: Dict[int, List[WebSocket]] = {}

    async def connect(self, websocket: WebSocket, room_id: int):
        await websocket.accept()
        if room_id not in self.active_connections:
            self.active_connections[room_id] = []
        self.active_connections[room_id].append(websocket)

    def disconnect(self, websocket: WebSocket, room_id: int):
        if room_id in self.active_connections:
            if websocket in self.active_connections[room_id]:
                self.active_connections[room_id].remove(websocket)

    async def broadcast_to_room(self, room_id: int, message: dict):
        if room_id in self.active_connections:
            for connection in self.active_connections[room_id]:
                try:
                    await connection.send_json(message)
                except Exception:
                    self.active_connections[room_id].remove(connection)

manager = ConnectionManager()

```

Extracto de código 8.1: Gestor de conexiones en el *backend*

Esta estructura permite emitir mensajes masivos con formato JSON mediante el método `broadcast_to_room`. Asimismo, incluye un mecanismo de tolerancia a fallos en bloque `try-except` que elimina automáticamente las conexiones diferidas o corruptas sin interrumpir la emisión hacia el resto de los participantes.

Suscripción y recepción de eventos en el cliente React

En el *frontend*, el componente establece la conexión WebSocket con el servidor enviando el parámetro del rol (teacher o student) como parámetro de consulta.

```
useEffect(() => {
  const idToUse = Number(urlRoomId || roomId);
  if (!idToUse || Number.isNaN(idToUse)) return;

  const ws = new WebSocket(
    `${WS_BASE_URL}/stage/rooms/${idToUse}/ws?role=${isHost ? "teacher" : "student"}`
  );

  ws.onopen = () => setIsConnected(true);
  ws.onclose = () => setIsConnected(false);
  ws.onmessage = (event: MessageEvent) => {
    const { type, data } = JSON.parse(event.data);
    switch (type) {
      case "next_question":
      case "room_start":
        updateRoomStatus(data);
        break;
      case "show_results":
        setStatistics(data.statistics);
        setCorrectOptionId(data.correct_option_id);
        break;
      case "show_leaderboard":
        setLeaderboardData(data.leaderboard);
        break;
      case "timer_update":
        if (data.time_left !== undefined) setTimeLeft(data.time_left);
        if (data.is_paused !== undefined) setIsPaused(data.is_paused);
        break;
    }
  };

  return () => ws.close();
}, [roomId, urlRoomId, isHost]);
```

Extracto de código 8.2: Conexión y enrutamiento de eventos WebSocket en React

El *handler* `onmessage` actúa como un enrutador de eventos de la interfaz de usuario, mutando el estado local de React en función del tipo de evento recibido. Asimismo, la función de retorno del *hook* `useEffect` (*cleanup function*) garantiza el cierre formal del socket al desmontar el componente, previniendo fugas de memoria.

8.1.2. Gestión de sala, estado, tiempo y aleatoriedad

El correcto funcionamiento de las sesiones depende de mantener la sala sincronizada y lista para todos los usuarios. Para lograrlo, el servidor asume el control centralizado de los eventos y las decisiones críticas, gestionando el avance del estado de la sala y evitando depender del estado o el reloj individual de cada cliente.

Cálculo determinista del tiempo restante en UTC

En `backend/routers/stage.py`, la medición del tiempo no depende del reloj del cliente, sino de la diferencia estricta de marcas temporales en UTC.

```
def get_calculated_time_left(room: Room) -> int:
    if room.is_paused or not room.timer_started_at:
        return room.remaining_time_at_pause

    now = get_utc_now()
    started_at = room.timer_started_at
    if started_at.tzinfo is None:
        started_at = started_at.replace(tzinfo=tz.utc)

    elapsed = (now - started_at).total_seconds()
    calculated_time = room.remaining_time_at_pause - int(elapsed)
    return max(0, min(room.answer_time, calculated_time))
```

Extracto de código 8.3: Cálculo determinista del tiempo restante en el servidor

Al restar el instante actual (`get_utc_now()`) del momento de inicio (`timer.started_at`), el sistema inmuniza la cuenta atrás frente a retrasos de red o pausas de ejecución en el cliente.

Pausa automática por desconexión del profesor

Para mitigar imprevistos técnicos durante una sesión presencial, el servidor detecta si la conexión WebSocket del docente cae inesperadamente.

```
async def _handle_teacher_disconnect(room_id: int, engine):
    with Session(engine) as session:
        room = session.get(Room, room_id)
        if room and room.status == RoomStatus.LIVE:
            room.remaining_time_at_pause = get_calculated_time_left(room)
            room.is_paused = True
            session.add(room)
            session.commit()
            await manager.broadcast_to_room(
                room_id,
                {
                    "type": "timer_update",
                    "data": {"time_left": room.remaining_time_at_pause, "is_paused": True},
                },
            )
```

Extracto de código 8.4: Gestión de desconexión imprevista del profesor

La función congela la partida almacenando el tiempo restante exacto en `remaining_time_at_pause` mediante `get_calculated_time_left` y notifica inmediatamente a todos los estudiantes, pausando la interfaz hasta la reconexión del profesor.

Barajado determinista de opciones

Para evitar que los estudiantes copien observando la opción seleccionada en la pantalla de un compañero, el *backend* desordena las opciones mediante una semilla *hash*.

```
def deterministic_shuffle(items: List[dict], room_id: int, question_id: int) -> List[dict]:
    def sort_key(item):
        seed_str = f"{room_id}_{question_id}_{item['id']}"
        return hashlib.sha256(seed_str.encode("utf-8")).hexdigest()

    return sorted(items, key=sort_key)
```

Extracto de código 8.5: Algoritmo de barajado determinista anti-copia

El algoritmo genera un *hash* *SHA-256* combinando la sala, la pregunta y la opción. Esto altera la disposición de las respuestas para cada partida pero mantiene un orden idéntico si un alumno reconecta o recarga su cliente.

Transición de preguntas y fase de verificación

El endpoint PATCH `/rooms/:room_id/next-question` coordina el avance del cuestionario o la transición a la fase final.

```
@router.patch("/rooms/{room_id}/next-question")
async def next_question(room_id: int, db: Annotated[Session, Depends(get_session)]):
    room = db.get(Room, room_id)
    q_ids = [int(x) for x in room.question_order.split(",") if x]
    next_index = room.current_question_index + 1

    if next_index <= len(q_ids):
        next_q = db.get(Question, q_ids[next_index - 1])
        room.current_question_index = next_index
        room.phase = RoomPhase.ANSWERING
        room.remaining_time_at_pause = room.answer_time
        room.timer_started_at = get_utc_now()
        room.is_paused = False
        db.commit()

        await manager.broadcast_to_room(room_id, {
            "type": "next_question",
            "data": {
                "question_id": next_q.id,
                "text": next_q.text,
                "options": [{"id": opt.id, "text": opt.text} for opt in next_q.options],
                "time_left": room.answer_time,
                "current_question_index": next_index,
                "total_questions": len(q_ids)
            }
        })
        return {"status": "success"}
    else:
        room.status = RoomStatus.VERIFYING
        participants = db.exec(select(Participant).where(Participant.room_id == room_id)).all()
        for p in participants:
            p.verification_token = secrets.token_hex(8)
            db.add(p)
        db.commit()

        await manager.broadcast_to_room(room_id, {"type": "room_verifying", "data": {"status": "VERIFYING"}})
        return {"status": "VERIFYING"}
```

Extracto de código 8.6: Avance de pregunta y paso a siguiente fase

Si restan preguntas por responder, el controlador actualiza el índice y emite el evento correspondiente. Al agotar la batería de preguntas, la sala cambia al estado VERIFYING y genera tokens criptográficos efímeros mediante `secrets.token_hex(8)`.

Verificación presencial mediante código QR

Una vez concluida la partida, el resultado acumulado debe ser validado por el docente escaneando un código QR en el dispositivo del alumno.

```
@router.post("/rooms/{room_id}/verify-participant")
async def verify_participant(
    room_id: int, req: VerificationRequest, db: Annotated[Session, Depends(get_session)]
):
    statement = (
        select(Participant)
        .join(Student)
        .where(Student.name == req.nickname, Participant.room_id == room_id)
    )
    participant = db.exec(statement).first()

    if not participant or participant.verification_token != req.token:
        raise HTTPException(status_code=400, detail="Token de verificación inválido")
    if participant.is_verified:
        raise HTTPException(status_code=409, detail="El resultado ya ha sido verificado.")

    actual_score = sum(a.points_earned for a in participant.answers)
    participant.score = actual_score
    participant.is_verified = True
    participant.verified_at = get_utc_now()
    db.add(participant)
    db.commit()

    await manager.broadcast_to_room(
        room_id,
        {"type": "participant_verified", "data": {"nickname": req.nickname, "participant_id": participant.id}},
    )
    return {"status": "success", "nickname": req.nickname}
```

Extracto de código 8.7: Validación presencial del participante mediante token QR

El controlador valida el token efímero (req.token), recalcula la puntuación final sumando los puntos auditados, marca al participante como verificado y notifica la confirmación en tiempo real.

8.1.3. Seguridad, contexto y autenticación

Este bloque aborda el tratamiento del estado del jugador, el saneamiento de datos en la SPA y la protección de endpoints sensibles mediante tokens JWT.

Persistencia y saneamiento de estado en la SPA

En RoomContext se encapsula la gestión de la sesión del jugador mediante la *API Context* de React.

```
export function RoomProvider({ children }: Readonly<{ children: ReactNode }>) {
  const [rawRoomCode, setRawRoomCode] = useState(sessionStorage.getItem("roomCode") || "");
  const [rawRoomId, setRawRoomId] = useState<number | null>(() => {
    const savedId = sessionStorage.getItem("roomId");
    return savedId ? Number(savedId) : null;
  });
  const [rawUserNickname, setRawUserNickname] = useState<string | undefined>(
    sessionStorage.getItem("userNickname") || undefined
  );
  const [roomData, setRoomData] = useState<RoomDataType>(null);

  const setUserNickname = useCallback((name: string | undefined) => {
    setRawUserNickname(name);
    if (name !== undefined) {
      const cleanNickname = name.replace(/[<>'&]/g, "");
      if (/^[^<>'&]*$/.test(cleanNickname)) {
        sessionStorage.setItem("userNickname", cleanNickname);
      }
    } else {
      sessionStorage.removeItem("userNickname");
    }
  }, []);

  const contextValue = useMemo(
    () => ({
      roomCode: rawRoomCode,
      roomId: rawRoomId,
      userNickname: rawUserNickname,
      setUserNickname,
      roomData,
      setRoomData,
    }),
    [rawRoomCode, rawRoomId, rawUserNickname, setUserNickname, roomData]
  );

  return <RoomContext.Provider value={contextValue}>{children}</RoomContext.Provider>;
}
```

Extracto de código 8.8: Gestión del contexto de la sala

El componente recupera de forma síncrona los identificadores desde `sessionStorage` para garantizar la persistencia del estado frente a recargas accidentales del navegador. Asimismo, se aplica una función de saneamiento que purga caracteres sensibles (<, >, ', ", &) antes de su persistencia y renderizado, mitigando vectores de ataque por inyección de scripts en sitios cruzados o XSS (*Cross-Site Scripting*) [46, 47].

Autenticación y protección de endpoints con JWT

En backend/auth.py, se implementa un mecanismo de inyección de dependencias para autenticar las peticiones de los docentes mediante cabeceras HTTP.

```
security = HTTPBearer()

def get_current_teacher_id(
    credentials: Annotated[HTTPAuthorizationCredentials, Depends(security)],
) -> int:
    token = credentials.credentials
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        raw_sub = payload.get("sub")
        if raw_sub is None:
            raise HTTPException(status_code=401, detail="Token sin campo 'sub'")
        return int(raw_sub)
    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=401, detail="El token ha expirado")
    except jwt.PyJWTError:
        raise HTTPException(status_code=401, detail="Token inválido")
```

Extracto de código 8.9: Inyección de dependencia para la validación de tokens JWT

Esta función intercepta el *Bearer token* emitido en las peticiones HTTP, descifra la firma criptográfica del estándar JSON Web Token [48] utilizando la clave secreta del servidor (SECRET_KEY) y verifica su validez temporal antes de permitir el acceso a los controladores protegidos.

8.2. Estrategia de testing

Unitarias, integración, E2E, etc

Me gustaría mencionar también su importancia y los objetivos de cobertura que tengo, quizás lo mejor sea en este capítulo mencionar los umbrales objetivos y en el capítulo 9 de resultados hablar de su cumplimiento

8.3. Casos de prueba y validación

Descripción de los casos de prueba utilizados para validar los criterios de aceptación y alcanzar la cobertura objetivo

8.4. Pruebas de carga y rendimiento

Descripción de las pruebas de carga y rendimiento realizadas

Importante porque los cuestionarios son en directo y con muchas personas simultáneamente

9. Resultados y evaluación

9.1. Grado de cumplimiento de la Línea Base del Alcance

Evaluación del cumplimiento de los objetivos y requisitos establecidos en la Línea Base del Alcance.

9.2. Tiempo real frente a Línea Base del Cronograma

Comparación del tiempo real dedicado a cada fase del proyecto con el tiempo estimado inicialmente

9.3. Gastos incurridos frente a Línea Base de Costes

Diferencia entre los gastos previstos y los reales

9.4. Desviaciones y gestión de cambios realizados

Cómo se gestionaron las desviaciones y qué modificaciones de la planificación provocaron

9.5. Resultados de las pruebas y validación

Resultados obtenidos en las pruebas unitarias, de integración, E2E, carga y rendimiento, y su comparación con los objetivos establecidos

9.6. Retrospectiva técnica

Reflexión sobre la implementación de las prácticas de CI/CD, testing y métricas de calidad, y decir si he cumplido los objetivos como las métricas de cobertura o de issues de sonar

10. Conclusiones

10.1. Grado de cumplimiento de los objetivos

Evaluación sobre los objetivos planteados en el acta de constitución.

10.2. Lecciones aprendidas y valoración personal

Reflexión sobre el trabajo realizado.

10.3. Trabajos futuros y líneas de evolución

Conclusiones sobre el trabajo realizado y líneas de evolución futuras.

A. Prototipos de interfaz de usuario

En este anexo se recopilan los prototipos de interfaz de usuario (*mockups*) correspondientes a las principales pantallas del sistema, organizados por flujos funcionales de navegación.

Para facilitar su interpretación visual y distinguir los roles de usuario en las capturas, se ha empleado la siguiente codificación de colores en el borde de las interfaces:

- **Borde magenta:** Identifica las pantallas pertenecientes a la vista del perfil docente (profesor).
- **Borde cian:** Identifica las pantallas pertenecientes a la vista del perfil estudiante (alumno).

A continuación, se presentan las distintas pantallas agrupadas según la secuencia lógica de cada flujo.

Flujo de registro y autenticación

Engloba las pantallas destinadas a la gestión de acceso de los profesores a la plataforma. Comprende el proceso de alta de nuevos usuarios, la verificación de credenciales a través del correo institucional con código de confirmación y el formulario de inicio de sesión estándar.



Crear Cuenta

NOMBRE COMPLETO

Ej. Juan Pérez

EMAIL

usuario@ejemplo.com

CONTRASEÑA

.....

CONFIRMAR CONTRASEÑA

.....

Crear Cuenta →

¿Ya tienes una cuenta? [Iniciar Sesión](#)

Figura A.1: Crear cuenta.



Verifica tu cuenta

Hemos enviado un código de 6 dígitos a tu correo electrónico. Ingresalo para continuar.

Verificar Registro

¿No recibiste el código? [Reenviar](#)

Figura A.2: Verificar cuenta.

Iniciar Sesión

Bienvenido de nuevo, Scholar.

Email o Usuario

Contraseña



[¿Olvidaste tu contraseña?](#)

Entrar al Aula →

¿No tienes una cuenta? [Registrarse](#)

Figura A.3: Iniciar sesión.

Flujo de creación de cuestionarios y gestión de salas

Comprende el conjunto de interfaces utilizadas por profesor para la preparación de las clases. Incluye creación de cuestionarios (preguntas, opciones y tiempos límite), la consulta del catálogo de cuestionarios creados, la pantalla de configuración de parámetros de la sala y el estado inicial de la sala (en espera).

Título del Cuestionario
Ej: Conocimientos Generales

Crear Cuestionario

Descripción
Describe brevemente este cuestionario...

Pregunta 1 10 Puntos

Escribe tu pregunta aquí...

A Opción 1

B Opción 2

C Opción 3

D Opción 4

Figura A.4: Crear cuestionario.

Mis Cuestionarios

Gestiona y juega tus cuestionarios creados.

Todos Recientes Borradores

Desarrollo

Fundamentos de Programacion

Conceptos básicos de lógica, algoritmos y estructuras de control para principiantes.

15 Preguntas Hace 2 dias

Play

Cultura

Geografia Mundial

Pon a prueba tus conocimientos sobre capitales, banderas y accidentes geográficos del mundo.

20 Preguntas Ayer

Play

Ciencia

Química Orgánica II

Repaso avanzado sobre grupos funcionales, nomenclatura y mecanismos de reacción.

12 Preguntas 1 semana

Play

+

Crear un nuevo cuestionario

Comparte tus conocimientos con el mundo creando un set de preguntas personalizado.

Figura A.5: Listado de cuestionarios.

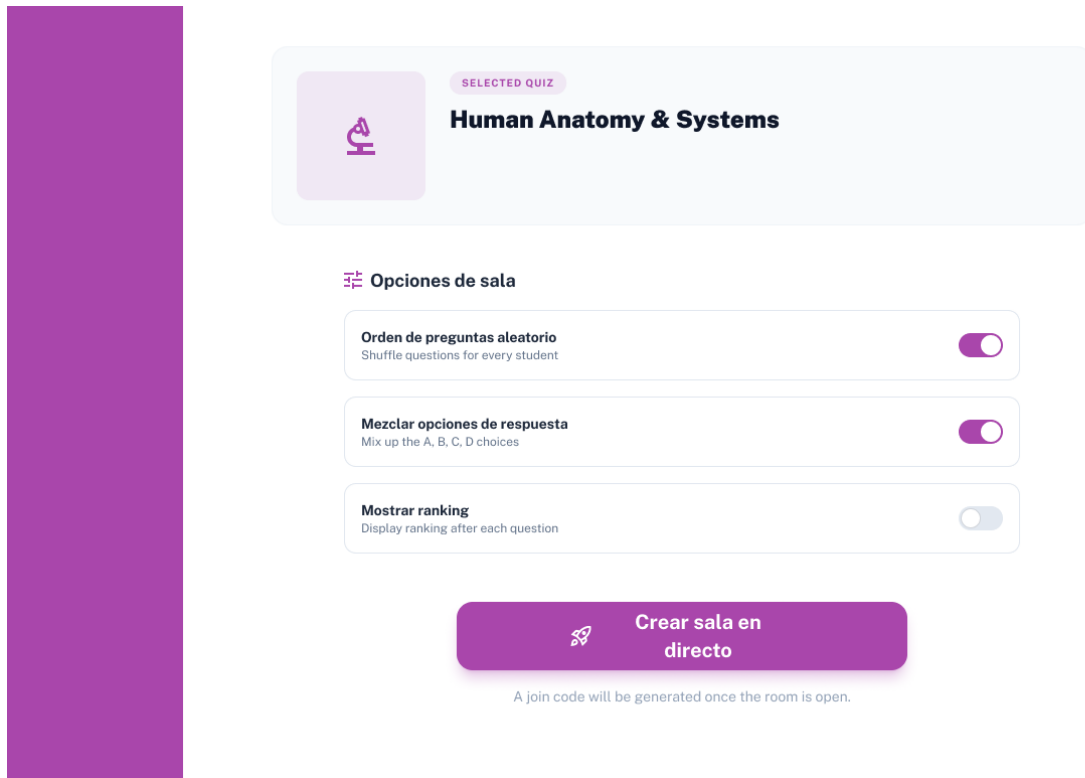


Figura A.6: Configuración de la sala.

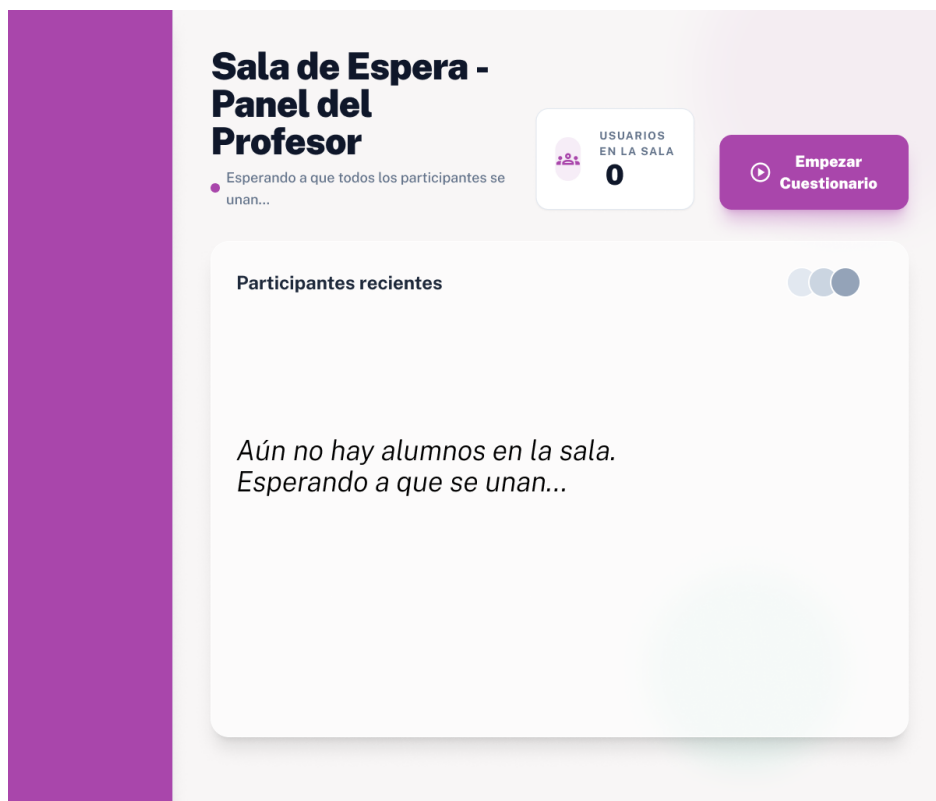


Figura A.7: Sala de espera de la sala recién creada.

Flujo de incorporación del estudiante a la sala

Muestra la secuencia que debe seguir un alumno para unirse a una sesión. Inicia con la introducción del código de sala, y continúa con la asignación de su identificador (UVUS o nickname), con su posterior aviso en caso de que el identificador no esté registrado. Por último, muestra la sala de espera hasta el inicio de la partida.



Figura A.8: Ingresar código de sala.

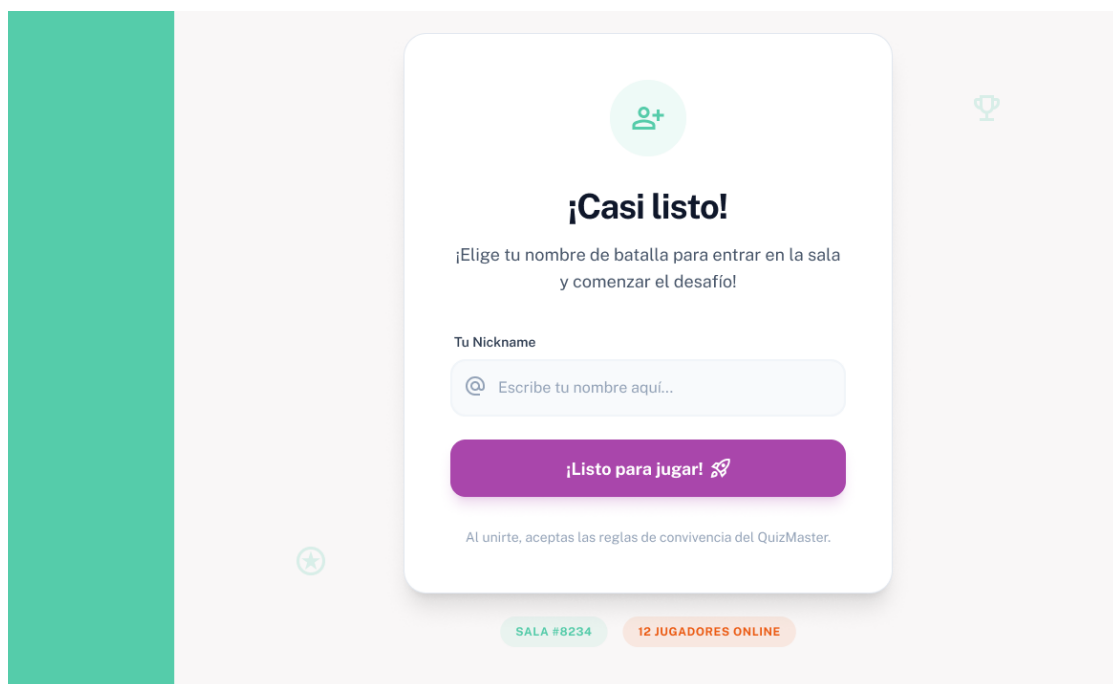


Figura A.9: Ingresar UVUS o nickname.

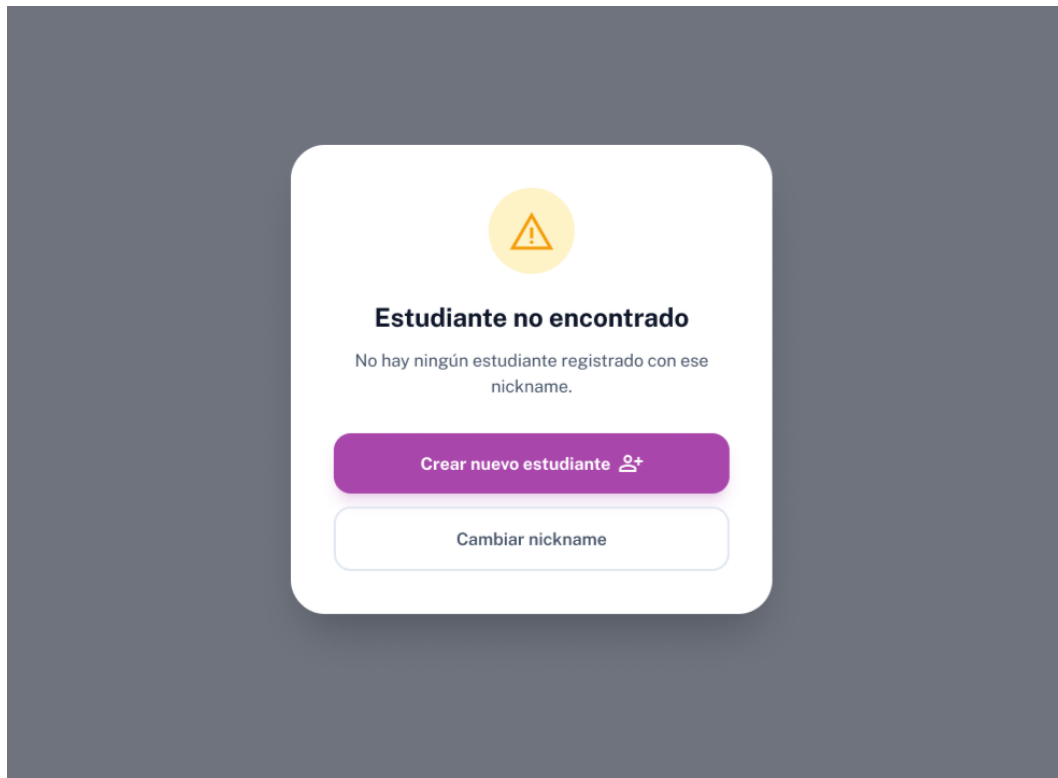


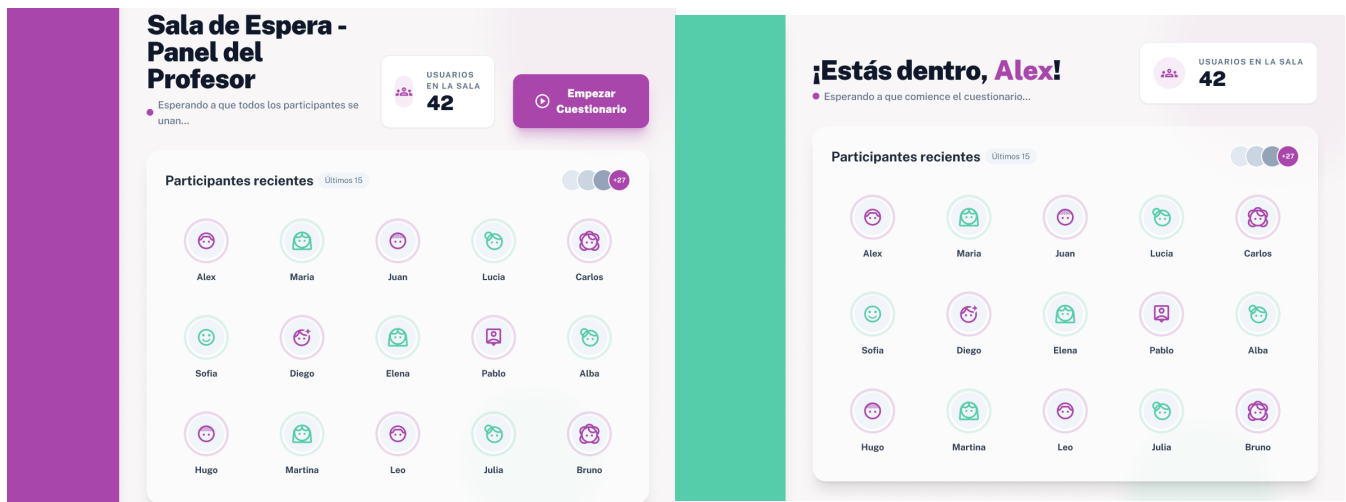
Figura A.10: Aviso para registrar nuevo UVUS o nickname.



Figura A.11: Sala de espera del alumno recién ingresado.

Flujo de ejecución de la partida en vivo

Describe la interacción en tiempo real durante el desarrollo de una sala. Para comparar la experiencia del profesor y del alumno, se presentan en paralelo las interfaces que muestran divergencias funcionales o de diseño. El flujo abarca las fases de sala de espera activa, lectura de pregunta, envío de respuestas, retroalimentación inmediata, tabla de clasificación y verificación final.



Vista Profesor

Vista Alumno

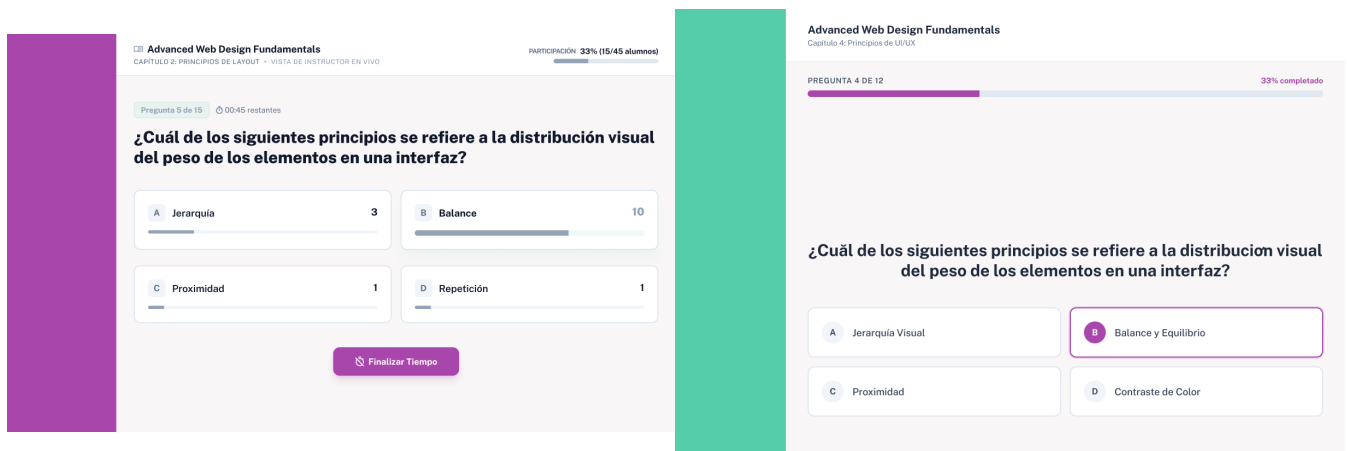
Figura A.12: Sala de espera.



Figura A.13: Cuenta atrás.



Figura A.14: Tiempo de lectura de pregunta.



(a) Vista Profesor

(b) Vista Alumno

Figura A.15: Pregunta en curso y envío de respuesta.



Figura A.16: Resultados de la pregunta.



Figura A.17: Podio de puntuaciones.

Resultados Finales del Cuestionario

Resumen detallado del desempeño del grupo en la sesión de hoy: [Introducción a la Biología Celular](#).

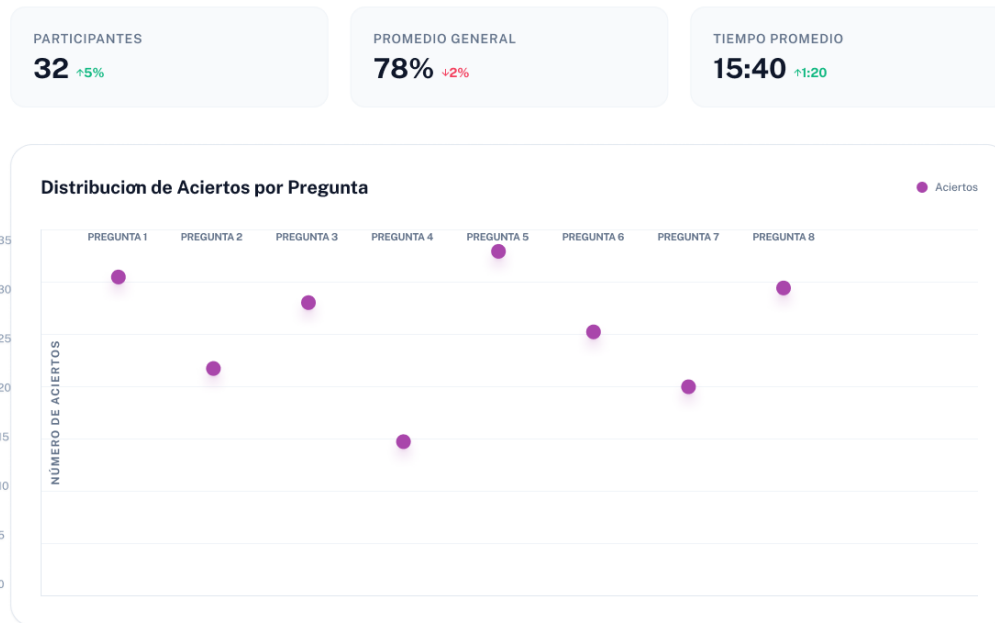


Figura A.18: Distribución global de respuestas.



Figura A.19: Verificación final de la sesión.

B. Manual de usuario

Manual de usuario de quizzie

C. Manual de instalación

Guía de instalación

Bibliografía

- [1] Roger S. Pressman. *Ingeniería del software: Un enfoque práctico*. McGraw-Hill, 7^o edition, 2010. ISBN 9786071503145.
- [2] Ian Sommerville. *Ingeniería de Software*. Pearson, 9^o edition, 2011. ISBN 9786073206037.
- [3] Robert C. Martin and Michael C. Feathers. *Clean code: a handbook of agile software craftsmanship*. Prentice Hall, 2009. ISBN 9780132350884.
- [4] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017. ISBN 9780134494166.
- [5] Dave Farley. *Continuous Delivery Pipelines: How To Build Better Software Faster*. Independently published, 2021. ISBN 979-8517979551.
- [6] Parlamento Europeo y Consejo de la Unión Europea. Reglamento (ue) 2016/679 relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (rgpd), 2016. URL <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32016R0679>.
- [7] Python Software Foundation. Python programming language official website, 2026. URL <https://www.python.org/>.
- [8] Sebastián Ramírez. Fastapi framework documentation, 2026. URL <https://fastapi.tiangolo.com/>.
- [9] MDN Web Docs. Websockets api - communication protocol, 2026. URL https://developer.mozilla.org/es/docs/Web/API/WebSockets_API.
- [10] Sebastián Ramírez. Sqlmodel library documentation, 2026. URL <https://sqlmodel.tiangolo.com/>.
- [11] SQLite Development Team. Sqlite database engine documentation, 2026. URL <https://www.sqlite.org/>.
- [12] Astral Software. uv: An ultra-fast python package installer and resolver, 2026. URL <https://docs.astral.sh/uv/>.
- [13] Meta Open Source. React - a javascript library for building user interfaces, 2026. URL <https://react.dev/>.
- [14] Vite Team. Vite - next generation frontend tooling, 2026. URL <https://vite.dev/>.
- [15] Microsoft Corporation. Typescript - javascript with syntax for types, 2026. URL <https://www.typescriptlang.org/>.

- [16] Remix Software. React router dom documentation, 2026. URL <https://reactrouter.com/>.
- [17] Axios Project. Axios - promise based http client for the browser and node.js, 2026. URL <https://axios-http.com/>.
- [18] Andrew M. St. Laurent. *Understanding Open Source and Free Software Licensing*. O'Reilly Media, 2008.
- [19] Project Management Institute. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. Project Management Institute, 7th edition, 2021.
- [20] Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, and Dave Thomas. *Manifesto for agile software development*, 2001. URL <https://agilemanifesto.org/>.
- [21] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2nd edition, 2014.
- [22] International Organization for Standardization / IEEE. ISO/IEC/IEEE 21839:2019 Systems and software engineering – Systems and software Work Breakdown Structure (WBS). Technical report, International Organization for Standardization / IEEE, 2019.
- [23] James M. Wilson. Gantt charts: A review of the project management tool. *International Journal of Project Management*, 21(7):521–528, 2003.
- [24] Dan Remenyi, Frank Bannister, and Arthur Money. *The Effective Measurement and Management of IT Costs and Benefits*. Butterworth-Heinemann, 2007.
- [25] Junta de Andalucía. Informe final sobre la consulta preliminar del mercado: “perfiles profesionales Ámbito informático”. Informe técnico, Consejería de Agricultura, Pesca y Desarrollo Rural. Secretaría General Técnica. Servicio de Informática, Sevilla, España, 2018. Firmado por D. Juan Luis Ceada Ramos. Publicado en el Perfil del Contratante.
- [26] Lisa M. Ellram. Total cost of ownership: elements and implementation. *International Journal of Purchasing and Materials Management*, 29(4):2–11, 1993.
- [27] IEEE Computer Society. IEEE Std 828-2012 IEEE Standard for Configuration Management in Systems and Software Engineering. Technical report, IEEE Computer Society, 2012.
- [28] Armando Fox and David Patterson. *Engineering Software as a Service: An Agile Approach Using Cloud Computing*. Strawberry Canyon LLC, 2014. ISBN 9780984881246.
- [29] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002. ISBN 9780321127426.
- [30] Eve Porcello and Alex Banks. *Learning React: Modern Patterns for Developing React Apps*. O'Reilly Media, 2nd edition, 2020. ISBN 9781492051725.

- [31] Greg MacDonald. *Practical UI Patterns for Design Systems: Fast-Track Interaction Design for a Seamless User Experience*. Apress, 2019. ISBN 9781484249380.
- [32] Vincent Driessen. A successful git branching model, 2010. URL <https://nvie.com/posts/a-successful-git-branching-model/>.
- [33] Emma Jane Hogbin Westby. *Git for Teams: A User-Centered Approach to Creating Efficient Workflows in Git*. O'Reilly Media, 2015.
- [34] Ian Sommerville. *Engineering Software Products: An Introduction to Modern Software Engineering*. Pearson, 2021. ISBN 9781292376356.
- [35] Jason Gregory. *Game Engine Architecture*. CRC Press, 2nd edition, 2014. ISBN 9781466560017.
- [36] International Organization for Standardization. Iso/iec 25010:2011 systems and software engineering – systems and software quality requirements and evaluation (square) – system and software quality models, 2011.
- [37] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 3rd edition, 2012. ISBN 9780321815736.
- [38] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional, 2004. ISBN 9780321200686.
- [39] Michael Nygard. Documenting architecture decisions. *Cognitect Blog*, 2011. URL <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>.
- [40] Francisco Gago Vázquez. Repositorio oficial de quizzie, 2026. URL <https://github.com/frangago71/quizzie-tfg>.
- [41] Guido van Rossum, Barry Warsaw, and Nick Coghlan. PEP 8 – Style Guide for Python Code, 2001. URL <https://peps.python.org/pep-0008/>. Python Enhancement Proposals.
- [42] Tom Preston-Werner. Semantic Versioning 2.0.0, 2013. URL <https://semver.org/>.
- [43] National Institute of Standards and Technology. Security and privacy controls for information systems and organizations. Technical Report NIST Special Publication 800-53, Revision 5, U.S. Department of Commerce, 2020.
- [44] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, Boston, MA, USA, 1994.
- [45] Patrick Th. Eugster, Pascal A. Felber, Rachid Guerraoui, and Anne-Marie Kermarrec. The many faces of publish/subscribe. *ACM Computing Surveys (CSUR)*, 35(2):114–131, 2003. doi: 10.1145/857076.857078.

- [46] OWASP Foundation. OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks. <https://owasp.org/Top10/>, 2021. Accedido en 2026.
- [47] Michal Zalewski. *The Tangled Web: A Guide to Securing Modern Web Applications*. No Starch Press, San Francisco, CA, USA, 2011.
- [48] Michael Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). RFC 7519, IETF, 2015.